COMP1870

Theoretical Foundations Of Computer Science

Part I

Dillon Mayhew

Contents

1 Introduction **3**
1.1 The maximum-matching problem . . . 3
1.2 A naive approach . . . 5
1.3 A greedy approach . . . 6
1.4 The augmenting paths algorithm . . . 6
1.5 Analysing the augmenting paths algorithm . . . 14

2 Logic **16**
2.1 Propositional logic . . . 16
2.2 Predicate logic . . . 22

3 Proofs **25**
3.1 Fitch diagrams . . . 26
3.2 Direct proofs . . . 34
3.3 Contrapositive proofs . . . 35
3.4 Proof by contradiction . . . 35

4 Inductive proofs **37**

5 Sets **44**
5.1 Subsets . . . 45
5.2 Set operations . . . 47
5.3 Venn diagrams . . . 48
5.4 Cartesian products . . . 49
5.5 Data structures . . . 50

6 Functions **52**
6.1 Composition of functions . . . 54
6.2 Properties of functions . . . 56

7 Relations **64**
7.1 Properties of relations . . . 66
7.2 Equivalence relations and partial orders . . . 69

8 Enumeration **73**
8.1 Cardinality . . . 73
8.2 Addition and multiplication rules . . . 73
8.3 Permutations . . . 75
8.4 Binomial coefficients . . . 76
8.5 The pigeonhole principle . . . 79
8.6 Inclusion-exclusion . . . 80
8.7 Recursive definitions . . . 81

9 Number Theory **84**
9.1 Divisibility . . . 84
9.2 Number representations . . . 85
9.3 Algorithms for arithmetic . . . 89
9.4 Quotients and remainders . . . 91
9.5 Modular arithmetic . . . 94
9.6 Greatest common divisors . . . 98
9.7 Multiplicative inverses . . . 102

1 Introduction

Theoretical computer science is the area of mathematics that provides a foundation for computer science. The main subject of study in theoretical computer science is *algorithms*.

Definition 1.1. An *algorithm* is a finite sequence of instructions. The instructions must be precisely defined so that there is no ambiguity in how they should be carried out. Usually the algorithm is designed to produce a specific output.

Why does computer science need mathematics? Here are some reasons:

- Mathematics can help us to find better algorithms. A naive approach might lead to an algorithm that is slow or inefficient. With mathematical insight perhaps we will be able to find an efficient algorithm. For example, how do we find the greatest common divisor of two large numbers? The naive approach tries factoring those two numbers into primes. This approach works only for small numbers. In Section 9.6 we will see a mathematical idea that lets us solve this problem quickly even for very large numbers.
- Mathematics gives us the tools to prove that our algorithms are correct. If we want to rely on an algorithm, we need a rigorous logical proof that the algorithm works. Mathematics is the science of proofs, so we use mathematics to certify that our algorithms are correct.
- We can use mathematics to analyse the running time of algorithms. Without mathematics, we might be reduced to just experimenting to find out whether one algorithm is faster than another. With mathematics, we can make this analysis rigorous.
- By analysing simple mathematical models of computing machines, we can explore the boundaries of what computers are capable of.

We are going to see some of these applications of mathematics in the first few of the module.

1.1 The maximum-matching problem

Imagine we have five servers and seven tasks to be completed by those servers. Each server is able to process at most one task, and each server is able to carry out only some of the tasks. This situation is illustrated in Figure 1. This diagram shows a *graph*.

Graphs are used to model many situations in computer science. We will spend more time on graphs later in the module. For now we just introduce a few key ideas. In the following discussion we are going to use the word *set* to refer to a collection of objects. We will talk about sets in more detail in Section 5. Every graph has a set of *vertices*, also known as *nodes*. In a drawing of a graph, the vertices are often represented by dots. Every graph also has a set of *edges*. An edge joins a pair of vertices. In a graph drawing, each edge is represented by a curve or a line joining the two dots that represent the pair of vertices. The vertices that are joined by an edge are said to be *incident* with that edge. If two vertices are joined by an edge, then we say that those vertices are *adjacent*, and that they are *neighbours*.

In Figure 1, each server and each task is represented by a vertex. An edge joins a server to a task if that server can carry out that task.

How many tasks can we assign to the servers? Obviously we cannot assign all seven tasks to the servers, since there are only five servers, and each can carry out at most one task. This shows that we can assign *at most* five tasks. But can we assign all five tasks? This is the question we want to solve using an algorithm. The problem is called the *maximum matching problem*.

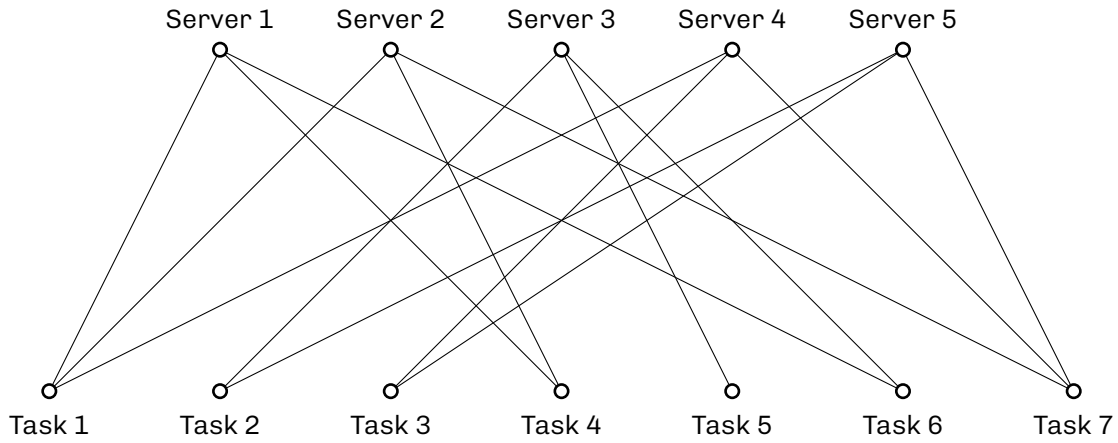


Figure 1: A bipartite graph representing an instance of the maximum matching problem.

Definition 1.2. If the vertices of a graph can be divided into two sets X and Y, where every edge joins a vertex in X to a vertex in Y, then we say that the graph is *bipartite*.

If we let X be the set of vertices representing servers and let Y be the set of vertices representing tasks, then every edge in the graph from Figure 1 joins a vertex in X to a vertex in Y, so this graph is indeed bipartite. The two graphs in Figure 2 are both bipartite, because the vertices are divided into the two sets: the square vertices and the circles. Every edge joins a square to a circle.

Definition 1.3. A *matching* of a graph is a set of edges with the property that every vertex is incident with at most one edge in the matching.

In other words, a matching is a set of edges, call it M, such that there is no vertex that is incident with two or more edges in M. Every vertex is incident with either no edge in the matching, or exactly one edge in the matching. Say that M is a matching. A vertex is *matched* if it is incident with one of the edges in M. The matching M is *perfect* if every vertex is matched. In Figure 2, the example of the left shows a matching. Look at the set of edges that are drawn with thicker lines. Every vertex is incident with at most one of these edges, so this is a matching. In fact, it is a perfect matching, since every vertex is matched. But in the drawing on the right, there is a square vertex that is incident with *two* of the heavier edges. So this set of edges is *not* a matching.



Figure 2: Two bipartite graphs. The thicker edges form a matching in the left graph, but not in the right.

We want to assign tasks to servers. Each task will be assigned to at most one server, and each server will have at most one task assigned to it. So the set of edges that join tasks to the servers they are assigned to will form a matching. We want to assign as many tasks as

possible, so we are looking for a matching that is as large as possible. This is the *(bipartite) maximum-matching problem*.

> BIPARTITE MAXIMUM-MATCHING
> Input: A bipartite graph.
> Problem: Find a matching with the maximum size.

We are going to consider a few approaches to solving this problem.

1.2 A naive approach

You might be able to see that the graph in Figure 1 has a matching with five edges. For example, we could make the following assignment:

Server 1	Task 1
Server 2	Task 4
Server 3	Task 2
Server 4	Task 3
Server 5	Task 7

No matching can have more than five edges, so this is a maximum-sized matching. We have solved the problem just by looking at the graph and luckily seeing a maximum-sized matching. What if this doesn't work?

A simple approach is just to check all possible sets of edges and keep track of the sets that are matchings. At the end of this process, we will have a list of all matchings, so we can check which is largest.

Exercise

Find all matchings in the graph illustrated in Figure 1.

Even for the small graph in Figure 1, this problem is hard, because there are so many sets of edges. Can we be more specific with this analysis? How many sets of edges will we need to check? Say that we want to solve the maximum matching problem in a graph G. Let $|V|$ stand for the number of vertices in G. How many edges could there possibly be? Every edge can be chosen by picking a pair of vertices, and the number of pairs is $|V| \times |V| = |V|^2$. (Actually, this is more than the number of edges, because an edge does not care about the order in which we pick the vertices. But we will ignore this subtlety.) We want to look at every possible set of edges. We will see later in the module that if a set contains k objects, then the total number of possible sets made up of those objects is 2^k. So the number of edge sets we need to inspect could be as large as $2^{|V|^2}$. Here is a table which shows how quickly this number increases.

n	2^{n^2}
1	2
2	16
3	512
4	65536
5	33554432
10	1.26765×10^{30}
20	2.58224×10^{120}

If we want to inspect all the possible sets of edges in a graph with 20 vertices, we potentially need to look at 2.58224×10^{120} sets! Even if we could check a trillion sets every second, it would still take us more than 10^{100} years to look at all of them and this is much more than the lifetime of the universe. So this naive approach can only be useful for very small examples.

1.3 A greedy approach

Let's try something a bit more sophisticated. What if we build a matching edge by edge? At each step we will choose an edge at random, and add it to our current matching as long as the result is another matching. This is what we call the *greedy approach*, since we are greedily adding the edge to the matching if we can, and we are not looking ahead to see what might happen in the future.

Let's use the greedy approach to find a matching in the graph from Figure 1. To simplify things, we use the notation S_iT_j to denote the edge joining Server i to Task j.

Our first randomly chosen edge is S_5T_7. We add this to our matching. So now we have a matching of one edge. Say that our next randomly chosen edge is S_1T_4. We can add this to our matching, so now our matching contains two edges: S_5T_7 and S_1T_4. But what if our next randomly chosen edge is S_1T_6? We cannot add this to our matching, because then S_1 would be incident with two edges. So we move to our next random choice. To make this process more efficient, we could adjust our random choice so that we never select an edge incident with any of the vertices that are already matched. The matched vertices so far are S_1, S_5, T_4, and T_7, so we won't choose any edge incident with any of these. Maybe our next randomly chosen edge is S_4T_1, and then after that S_3T_6. Now our matching contains four edges: S_5T_7, S_1T_4, S_4T_1 and S_3T_6. What edges can we choose after that? We can't choose any edge incident with S_1, S_3, S_4, or S_5, so we are forced to choose an edge incident with S_2. But the only edges incident with S_2 join it to T_1, T_4, and T_7, and these vertices are already matched. So we cannot add any more edges and the greedy algorithm has terminated. But it hasn't found a maximum-sized matching! We already know that the graph has a matching of five edges, so we can do better than four.

The greedy approach is quick, but it may not find the best-possible matching. There are some important applications where a greedy approach works well and provides us with fast algorithms. We will see some of these applications later in the module. But the maximum-matching problem is not one that can be solved by the greedy approach.

1.4 The augmenting paths algorithm

The naive approach of trying every set of edges is far too slow. The approach of greedily building a matching is fast, but doesn't necessarily find a maximum-size matching. What can we do instead? It turns out there is a good algorithm for solving the maximum matching problem, an approach which runs quickly and which is guaranteed to always find a largest-possible matching. This approach requires some mathematical ideas. Our next job is to explain this algorithm.

Say that we have a bipartite graph, which we will call G. We also have a matching, which we call M (we are not saying that M is a largest-possible matching). Remember this means that M is a set of edges in G. Every vertex of G is incident with zero or one edges in M; this is what makes M a matching. The left-hand diagram in Figure 3 shows a bipartite graph and a matching made up of two heavier edges (x_1y_3 and x_3y_2).

A *path* is a sequence of vertices, without any repetitions, where each pair of consecutive vertices is joined by an edge. We think of the edges that join consecutive vertices as being edges in the path. The graph on the left in Figure 3 shows a path, where the edges in the path are

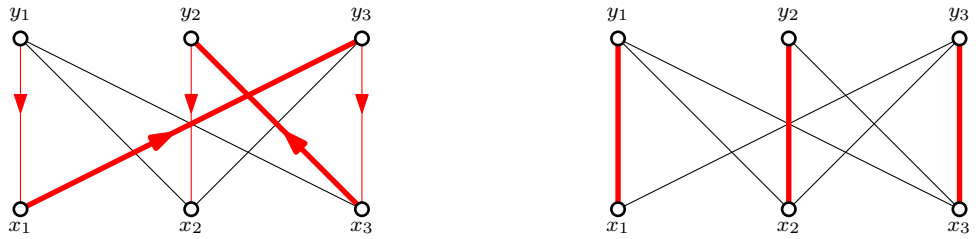


Figure 3: An augmenting path and the matching that we obtain by swapping the edges on the path.

highlighted in red and marked with arrows that show the direction of the path. The sequence of vertices in this path is $y_1, x_1, y_3, x_3, y_2, x_2$.

Remember that M is a matching. If the edges of a path alternate between edges in M and edges not in M, then we say that the path is *alternating*. The path in Figure 3 is alternating, because its edges are y_1x_1 (which is not in the matching), then x_1y_3 (which is in the matching), then y_3x_3 (not in the matching), then x_3y_2 (in the matching), and finally y_2x_2 (not in the matching). On the other hand, the path x_2, y_1, x_1 is not alternating, since the edge x_1y_1 is not in the matching, and neither is the next edge, y_1x_1.

If a path is alternating, and the first and last vertices of an alternating path are not matched by M, then we say that the path is *augmenting*. So every augmenting path is alternating, but not every alternating path is augmenting. In the graph on the left in Figure 3, the red edges make up an augmenting path since y_1 and x_2 are both unmatched.

Augmenting paths are useful for our application because we can use them to increase the size of a matching. Imagine that we change the matching by removing the edges of the matching that are in the augmenting path, and adding the edges of the augmenting path that are not in the matching. This means we flip the status of every edge in the path: edges not in the matching go into the matching, and edges in the matching leave the matching. We can think of this as adding all the edges of the path to the matching, and then removing the edges that are in the path and in the original matching. The graph on the right in Figure 3 has been obtained by performing this swapping operation on the augmenting path in the left diagram: we have gone from a matching with two edges to a matching with three edges.

Exercise

Convince yourself that when we flip the status of edges in an augmenting path we obtain a matching that contains one more edge than the original matching.

Now we have all the ingredients necessary for an algorithm that will find a maximum-sized matching. We call it the *augmenting paths* algorithm. We start with the matching that has zero edges. We look for an augmenting path. If we can find one, then we use it to make the matching larger. Then we repeat this step: we look for an augmenting path, and if we find one, we use it to make a larger matching. If we cannot find an augmenting path, then we stop. This process is illustrated in Figure 4.

There are some important questions that we have not yet addressed:

- Will the algorithm we have just discussed always find a matching with maximum size?
- If so, why?

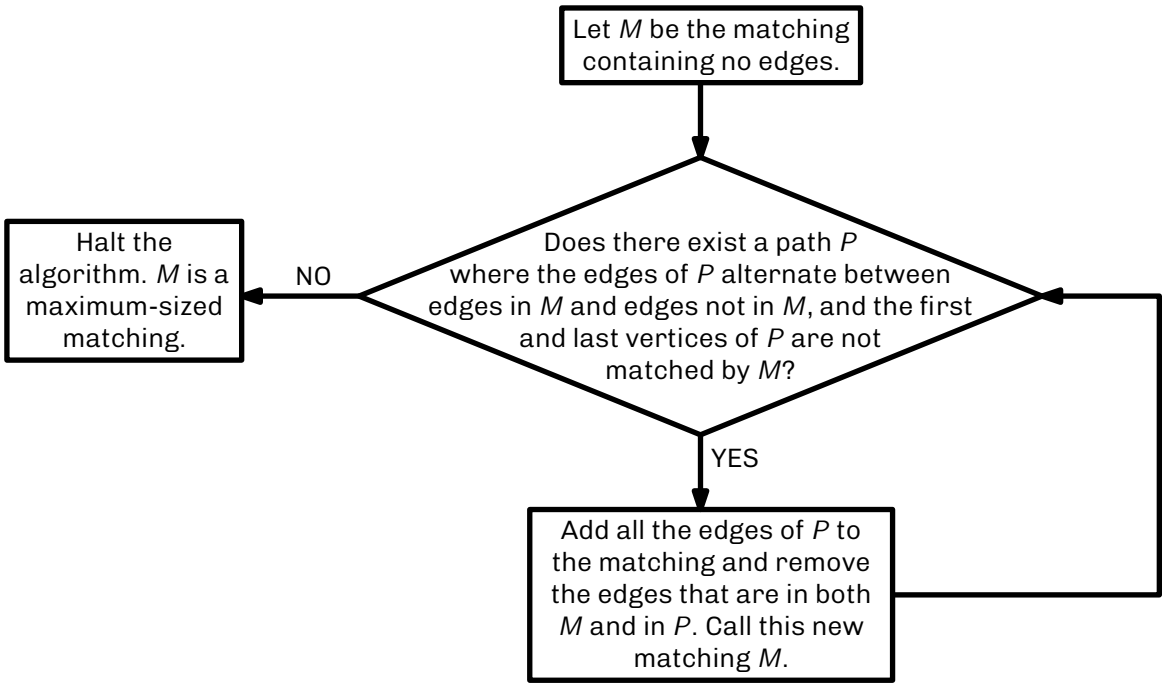


Figure 4: A flow chart for the maximum-matching algorithm.

- How do we actually find an augmenting path?
- How quickly does the algorithm run?

We will answer the first question: yes, the augmenting paths algorithm always finds a maximum-size matching. What about the second question? Why does the algorithm work? The answer is that the algorithm works because of the following mathematical fact.

Theorem 1.4. *Let G be a bipartite graph and let M be a matching of G. If there is no augmenting path, then M is a maximum-sized matching.*

This theorem means that if the algorithm reaches a point where there is no augmenting path, then we can stop because we already have a maximum-sized matching. So how do we know that Theorem 1.4 is true? This is what a mathematical proof is for — it gives us a rigorous argument that the theorem must be true. We won't prove the theorem now: we will leave that for later in the module.

Now for the third question: how do we search for an augmenting path? We will use what we call a *breadth-first search*. This is a procedure which will find an augmenting path, if one exists. Assume that G is a bipartite graph, where the vertices are divided into two sets X and Y, and every edge joins a vertex in X to a vertex in Y. Let M be a matching in G. We write U_X for the set of unmatched vertices in X and let U_Y be the set of unmatched vertices in Y.

Exercise

Explain why any augmenting path must be an alternating path from a vertex in U_X to a vertex in U_Y. Explain why an alternating path from a vertex in U_X to a vertex in U_Y is augmenting.

The breadth-first search will start with the vertices in U_X, and then look at the neighbours of these vertices, and then the neighbours of those vertices, and so on. We make sure we construct alternating paths, so that if the first edge is not in M, then the second edge is in M, etc. If we reach a vertex in U_Y, then we have found an augmenting path. We keep track of the vertices we visit by adding them to a list (sometimes called a *queue*). Every time we add a vertex to the queue, we keep track of the vertex we reach it from. Say that we reach vertex u from v. Then we set $\mathsf{pred}(u) = v$, where pred stands for *predecessor*. This lets us backtrack to find all the vertices in an augmenting path. The *pseudocode* in Algorithm 1 shows how the breadth-first search procedure works. The input is the graph G along with the matching M, and the output is an augmenting path P (assuming one exists). If there is no augmenting path then the output will be empty, and at this point we will know we have a maximum-sized matching.

Algorithm 1 The breadth-first search to find an augmenting path

Input: A bipartite graph, where every edge joins a vertex in the set X to a vertex in the set Y, and a matching M.
Output: An augmenting path P, or an empty output if there is no augmenting path.

```
1: Mark every vertex as unvisited.
2: U_X ← the set of vertices in X that are unmatched by M.
3: L ← the vertices in U_X listed in some order.
4: Mark the vertices in L as visited.
5: while L contains at least one vertex do
6:     a ← the first vertex in L.
7:     Remove the first vertex from L.
8:     if a is in X then
9:         for unvisited vertex b that is joined to a by an edge not in M do
10:            Mark b as visited.
11:            pred(b) ← a
12:            if b is matched by M then
13:                Add b to the end of list L.    //Add b to the list for later processing.
14:            else b is unmatched by an edge in M.    //We have found an augmenting path.
15:                P ← a list containing the single vertex b
16:                c ← b
17:                while c is not in U_X do    //We build the path P by backtracking.
18:                    d ← pred(c)
19:                    c ← d
20:                    Add c to the beginning of the list P.
21:                end while
22:                Halt and return P.    //The path P is augmenting.
23:            end if
24:        end for
25:    else a is in Y    //Note that in this case, a must be matched.
26:        b ← the vertex in X that is joined to a by an edge in M.
27:        Mark b as visited.
28:        pred(b) ← a.
29:        Add b to the end of list L.
30:    end if
31: end while    //If the list becomes empty, there is no augmenting path.
```

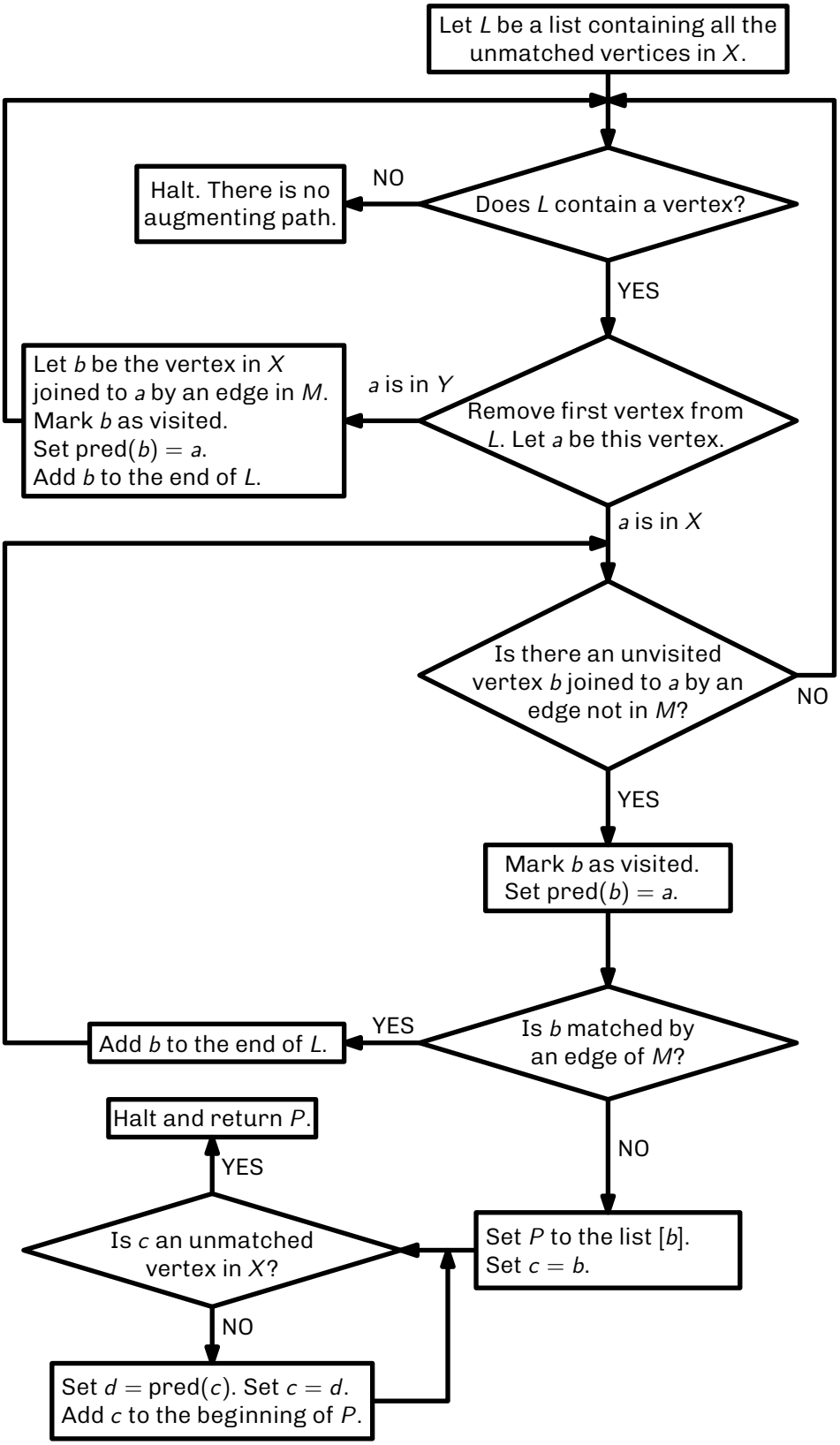


Figure 5: A flow chart for breadth-first search to find an augmenting path.

The flow chart in Figure 5 gives an alternative description of the same breadth-first-search algorithm. We will apply our algorithm to the graph in Figure 6. In this case X is $\{x_1, x_2, x_3, x_4, x_5, x_6\}$ and Y is $\{y_1, y_2, y_3, y_4, y_5\}$.

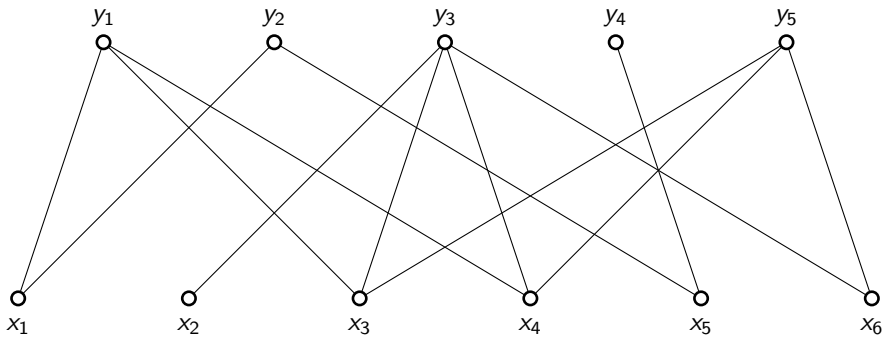


Figure 6: A bipartite graph.

We start with M being equal to the empty matching, the one that contains zero edges. We will use Algorithm 1 to show how we can find an augmenting path. Every vertex in X is unmatched by M, so on line 3 of the algorithm, we set the list L to be $[x_1, x_2, x_3, x_4, x_5, x_6]$. On line 5, we enter the **while** loop, since L contains at least one vertex. On line 6 we set a to be x_1 and then we remove x_1 from L. At this stage the only visited vertices are the ones in X. So on line 9, we will choose the unvisited vertex y_1 to be b, since y_1 is joined to x_1 by an edge that is not in M. Line 11 sets $\mathsf{pred}(y_1) = x_1$. We skip from line 12 to line 14 because b is unmatched by M. In lines 15–22, we produce the path $P = [x_1, y_1]$ as output. Now we increase the size of our matching by flipping the status of every edge in P. There is only one edge in P, and it is not in M, so we change its status to being in the matching. Our new matching contains exactly one edge, the edge x_1y_1. This is the matching shown in Figure 7.

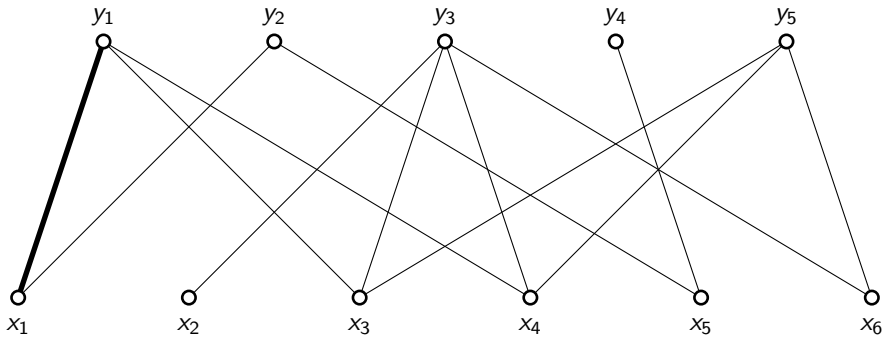


Figure 7: The matching after one application of breadth-first search.

Now we apply Algorithm 1 to the same graph with the matching M containing exactly one edge: x_1y_1. On line 3, we will set L to be the list $[x_2, x_3, x_4, x_5, x_6]$. On line 6 we set a to x_2, and in line 9, we will set b to be y_3. We will then return the path $P = [x_2, y_3]$. We flip the status of every edge in this path, so the edge x_2y_3 goes from not being in the matching to being in the matching. At this point our matching contains two edges: x_1y_1 and x_2y_3. This matching is shown in Figure 8.

The next step will proceed in a similar way: we set L to be $[x_3, x_4, x_5, x_6]$. These are the only visited vertices. At line 6, we set a to be x_3. In line 9, we first choose b to be y_1 and we set $\mathsf{pred}(y_1) = x_3$. Since y_1 *is* matched by M, we enter the **if** statement in line 12 and add y_1 to the

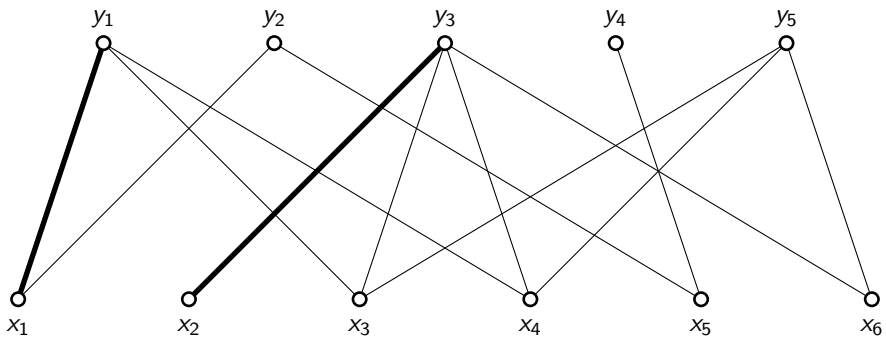


Figure 8: The matching after two applications of breadth-first search.

end of L before skipping to the end of the **if** statement on line 23. Now L is the list $[x_4, x_5, x_6, y_1]$. We then repeat the **for** loop at line 9, with b set to y_3. At the end of the **for** loop, we will have set $\mathsf{pred}(y_3) = x_3$, and the list L will be $[x_4, x_5, x_6, y_1, y_3]$. We repeat the **for** loop at line 9 with b set to y_5. This time y_5 is unmatched by M, so we output the path $P = [x_3, y_5]$. When we flip the status of the edges in this path, we add x_3y_5 to our matching, which now consists of three edges: x_1y_1, x_2y_3, and x_3y_5. This is the matching shown in Figure 9.

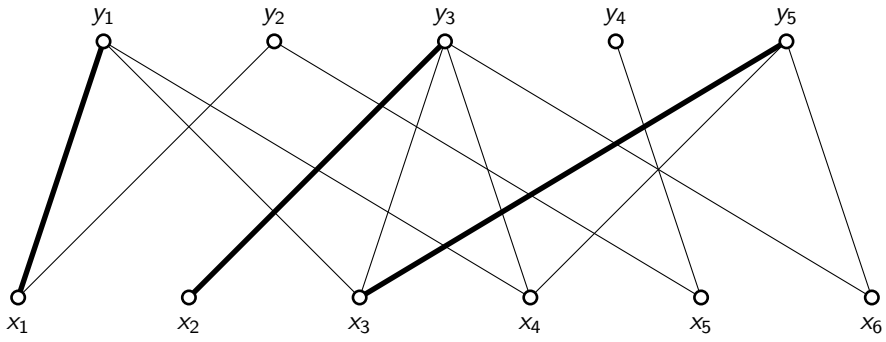


Figure 9: The matching after three applications of breadth-first search.

In the next application of Algorithm 1, we start with M equal to the matching that contains x_1y_1, x_2y_3, and x_3y_5 and L equal to the list $[x_4, x_5, x_6]$. We set a to be x_4 and set b to be y_1. We mark y_1 as visited, set $\mathsf{pred}(y_1) = x_4$, and add y_1 to the end of L. At the next repetition of the **for** loop, we choose b to be y_3. We mark y_3 as visited, set $\mathsf{pred}(y_3) = x_4$, and add y_3 to the list. At the next step, we choose b to be y_5, mark y_5 as visited, set $\mathsf{pred}(y_5) = x_4$, and add y_5 to the list. So now L is the list $[x_5, x_6, y_1, y_3, y_5]$. We have processed all the unvisited vertices joined to x_4 by an edge not in M, so we have completed the **for** loop on line 9. We set a to be the first vertex in L, so a is x_5. We enter the **for** loop by choosing b to be y_2. Since y_2 is unmatched by M, we output the path $P = [x_5, y_2]$. We flip the status of every edge in this path, and obtain the new matching consisting of the edges x_1y_1, x_2y_3, x_3y_5, and x_5y_2. This is the matching in Figure 10.

Now we apply Algorithm 1 with the matching M containing the edges x_1y_1, x_2y_3, x_3y_5, and x_5y_2. We start with the list L set to $[x_4, x_6]$. In line 5, we enter the **while** loop, and we choose a to be x_4. We repeat the **for** loop between lines 9 and 24 three times, with b set to y_1, y_3, and y_5. At the end of this loop, the list L is $[x_6, y_1, y_3, y_5]$. At the next repetition of the **while** loop, a is set to x_6. But the neighbours joined to x_6 by edges not in M are y_3 and y_5. Both of these were visited in the previous **while** loop, so there are no unvisited neighbours of x_6. Therefore we have completed the **while** loop with a set to x_6. At this stage L is $[y_1, y_3, y_5]$. We enter the **while** loop again with a

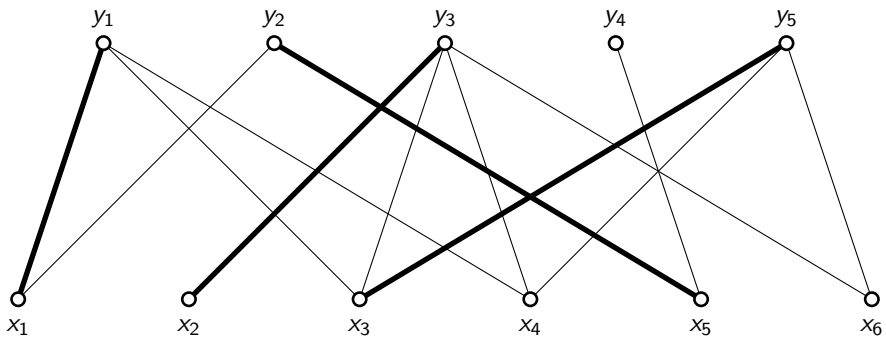


Figure 10: The matching after four applications of breadth-first search.

set to y_1. Since a is in Y, we enter the **else** at line 25. The vertex that is joined to y_1 by an edge of M is x_1. We mark x_1 as visited, and set $\mathsf{pred}(x_1) = y_1$, and then add x_1 to the end of L. The list is now $[y_3, y_5, x_1]$. At the next execution of the **while** loop, we set a equal to y_3, and we add x_2 at the end of L. At the next **while** loop, a is set to y_5 and we add x_3 to L. So now L is $[x_1, x_2, x_3]$. Next we enter the **while** loop with a set to x_1. We set b to y_2 and add y_2 to the end of L. Now L is $[x_2, x_3, y_2]$. At the next **while** loop, a is set to x_2. Since x_2 has no unvisited neighbours, we do not enter the **for** loop. At this stage L is $[x_3, y_2]$. Now we set a to be x_3. All of the neighbours of x_3 have been visited, so we do not enter the **for** loop. Now L only contains one entry: y_2. We set a to be y_2. The vertex that is joined to y_2 by an edge of M is x_5, so we add it to L. Now L contains the single vertex x_5. In the **while** loop, a is set to x_5. The only unvisited neighbour of x_5 is y_4, so we set b to y_4. We mark y_4 as visited and set $\mathsf{pred}(y_4) = x_5$. Because y_4 is unmatched by M, we execute the instruction at line 14. We are going to reconstruct the augmenting path P by executing the **while** loop on line 17. When the algorithm halts, the augmenting path P will be equal to $[x_4, y_1, x_1, y_2, x_5, y_4]$.

We increase the size of the matching by flipping the status of the edges in P. The edge x_4y_1 is placed in the matching, and x_1y_1 is taken out. Next, x_1y_2 is added to the matching and x_5y_2 is taken out. Continuing in this way, we end up with the matching $x_4y_1, x_1y_2, x_5y_4, x_2y_3, x_3y_5$. This is the matching shown in Figure 11.

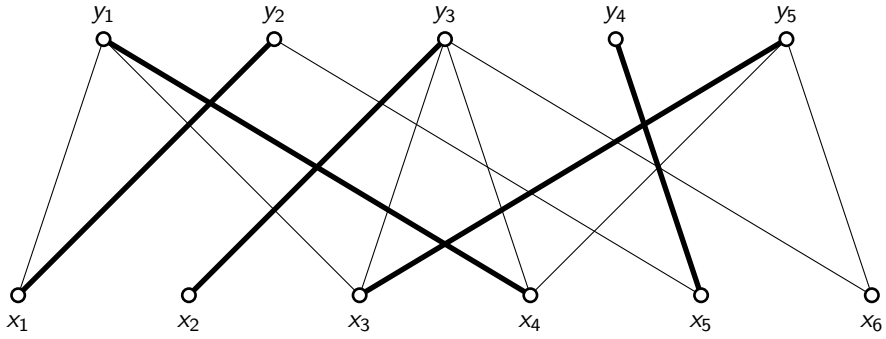


Figure 11: The matching after five applications of breadth-first search.

We can see for ourselves that this is a maximum-sized matching, because every vertex in Y is matched. If we continued to apply the algorithm, it would tell us that there is no augmenting path, and therefore we would halt the algorithm.

1.5 Analysing the augmenting paths algorithm

At this point we have applied the augmenting paths algorithm to an example, and we have talked about the reason it always produces a maximum-sized matching. There is an important question we have not spoken about: how quickly does the algorithm work? Say that we apply it to a graph with $|V|$ vertices. What is the maximum number of steps the algorithm will take?

Exercise

Explain why the number of edges in a matching can never be bigger than $\frac{1}{2}|V|$.

First of all we ask how many times we will need to search for an augmenting path. Each time we find such a path, we add one edge to the current matching. By the previous exercise, we will never carry out this steps more than $\frac{1}{2}|V|$ times.

Now how long does it take to search for an augmenting path? We analyse the pseudocode in Algorithm 1. The **while** loop in lines 5–31 processes each vertex at most once, because once a vertex is marked as visited, it is never visited again and cannot be added to the list L a second time. So this **while** loop is executed at most $|V|$ times. How many loops happen inside the **while** loop? The instructions in lines 25–30 do not involve any loops at all. And the **for** loop in lines 9–24 processes the unvisited neighbours of a. There are at most $|V|$ of these neighbours, so this loop will execute at most $|V|$ times. The only other loop we have not considered is the **while** loop in lines 17–21. When we do enter this loop, it could run $|V|$ times, but we only enter the 17–21 loop once we have found a neighbour that is unmatched by M. This can only happen the final time we enter the **for** loop. So the number of steps it takes to run the **for** loop is at most $(|V|-1)+|V| = 2|V|-1$. When we put all this analysis together, we obtain the maximum number of steps to run the augmenting paths algorithm.

$$\underbrace{\tfrac{1}{2}|V|}_{\text{edges in matching}} \times \underbrace{|V|}_{\text{lines 5–31}} \times \underbrace{2|V|-1}_{\text{lines 9–24}} = |V|^3 - \tfrac{1}{2}|V|^2$$

Now we have an answer to our question: when we run the augmenting paths algorithm on a graph with $|V|$ vertices, we expect it to take at most $|V|^3 - \frac{1}{2}|V|^2$ steps. This answer has too much detail. The most important term in the expression is $|V|^3$, meaning that as $|V|$ gets very large, the function $|V|^3 - \frac{1}{2}|V|^2$ starts behaving more and more like $|V|^3$. We express this by writing that the algorithm takes $\Theta(|V|^3)$ steps to run. You will see more notation like this at the end of the module.

Remark

The analysis we have just done is a typical example of *running-time analysis*. Note that we have ignored the cost of things like marking vertices as visited. The important thing is to count the number of loops. It is not useful or feasible to produce a function which tells us *exactly* how many steps the algorithm will take. Instead we want a rough approximation which will tell us how the algorithm behaves as the input gets larger.

Our first naive algorithm for solving the maximum-matching problem needed something like $2^{|V|^2}$ steps. This function grows incredibly quickly, compared to our new approximate running-time of $|V|^3$. The next definition captures an important difference between these running times.

Informal definition 1.5. Roughly speaking, we say that an algorithm has *polynomial running-time* if there is some number c such that when the input has size n, then the time it takes for the

algorithm to run is at most n^c.

Our first algorithm did not have polynomial running-time, since $2^{|V|^2}$ is larger than any function of the form $|V|^c$. Algorithms with a polynomial running time are generally considered to be efficient.

Exercise

Is an algorithm with polynomial running-time necessarily efficient? What if the running time was something like n^{100}?

Let us summarise the ways that mathematics has appeared in this introduction to the module. The fact that augmenting paths let us grow matchings is a mathematical insight: no-one would have discovered the augmenting paths algorithm without mathematical intuition. In order to prove that the algorithm works, we need a mathematical statement: Theorem 1.4. Proving this statement requires mathematical arguments (which you will see later).

Furthermore, we have analysed the algorithm and compared it to the naive algorithm which simply examines every set of edges. The augmenting paths algorithm has a running time of about $|V|^3$ and the naive approach has a running time of about $2^{|V|^2}$. It certainly seems as though $|V|^3$ is much better, but how would we know this for sure? This requires ideas from calculus and analysis, which we will also see later in the module. In summary, this introduction has demonstrated why mathematical ideas are at the heart of the design and analysis of algorithms.

Summary

Important ideas from Section 1.

- A graph is *bipartite* if the vertices can be divided into two sets X and Y, where every edge joins a vertex in X to a vertex in Y.
- A *matching* is a set of edges, say M, such that no vertex is incident with two or more edges in M.
- A path is *alternating* if its edges alternate between being in the matching and not being in the matching. An alternating path is *augmenting* if its end vertices are both unmatched.
- A matching is maximum-sized if and only if there is no augmenting path.

2 Logic

Logic is a branch of mathematics and philosophy with multiple applications in computer science. The study of logic consists mainly of studying *formal languages* and *logical arguments*. A formal language is in contrast to the *natural languages* that humans use to communicate with each other. Formal languages are constructed with rigorous rules of grammar, unlike natural languages. Any programming language is an example of a formal language.

In a logical argument, we accept certain statements as being true, and from these statements we are forced to accept other statements. This process is called *deduction*, or *inference*: we *deduce* or we *infer* new statements from the statements that we have already accepted. Deductive arguments are the central object of mathematics — they are what makes mathematics distinct from other sciences.

We will start our investigation of logic by looking at a simple formal language.

2.1 Propositional logic

We are going to build a language of *logical statements* by starting with the simplest possible statements, and then joining them together to build more complicated statements. We think of a statement as being something that has one of two *truth values*: it is either *true* or *false*. A *Boolean value* means the same thing as a *truth value*. We use **T** to stand for a value of true, and **F** to stand for a value of false.

Remark

Often you will see truth values represented by 0 and 1, where 0 stands for false, and 1 stands for true.

The simplest possible statements are called *propositions* (sometimes they are called *atomic statements*). Propositions are used to build more complicated statements. The language of statements that we build in this way is called *propositional logic*. We use upper-case letters to represent propositions, such as A, B, P_1, P_2, P_3 and so on. Like any other logical statement, propositions have one of two possible truth values. A proposition is either true or false.

Remark

Most statements in English do not have a definite truth value. Think of the statement "the sky is blue". Maybe this statement is true, maybe it is false, and maybe it is something in between. (Does it depend on the time and place, and on our subjective perception of the colour blue?) In classical logic, we consider only two truth values: true or false. If we want examples of statements that are definitely either true or false, we really need to think about the mathematical universe. Whenever we talk about a proposition, you can imagine a statement like "4 is an even number". However, it may be best to just accept that proposition is a thing that has a value of either true or false: it doesn't have to stand for anything, it just *is*.

To build our language we need ways of combining propositions into more complicated expressions. So now we talk about *logical operators*. These operations take one or two statements, and combine them in some way to produce another statement.

Negation. We start with the simplest logical operation: *negation*. If A is any logical statement, then $\neg A$ stands for the negation of A. The negation of A is another logical statement: it is false if A is true, and it is true if A is false. We can express this compactly with a *truth table*.

A	$\neg A$
T	**F**
F	**T**

This truth table has two rows, and each row corresponds to a possible truth value of A. The first row says that when A is true, $\neg A$ is false, and the second row says that when A is false, $\neg A$ is true.

Remark

Instead of writing $\neg A$, you may see the notations $\overline{A}$, $\sim A$, or NOT(A).

Conjunction. Unlike the negation operation, our next operation combines *two* statements. It is called the *conjunction* operation. If A and B are statements, then $A \wedge B$ stands for the conjunction of A and B. Conjunction roughly corresponds to the English word "and", so instead of saying "the conjunction of A and B", we say "A and B".

We expect the statement "A and B" to be true if both A and B are true, but false if either A or B is false. This is expressed in the following truth table.

A	B	$A \wedge B$
T	**T**	**T**
T	**F**	**F**
F	**T**	**F**
F	**F**	**F**

Note that unlike the truth table we saw for negation, this truth table has four rows. That's because negation operates on only one statement, whereas conjunction operates on two. There are four possible different truth values of A and B, and these are represented in the four rows. (We will use the convention that the four rows are arranged in the order shown above, but you may see this order reversed in some sources.)

Remark

In some textbooks you will see conjunction written as $A\&B$ or A AND B instead of $A \wedge B$.

Disjunction. Our next logical operation is called *disjunction*. If A and B are statements, then $A \vee B$ stands for the disjunction of A and B. The disjunction operation roughly corresponds to the English word "or". We say "A or B" to mean the disjunction $A \vee B$.

We would expect the disjunction $A \vee B$ to be true if one of the two terms in the disjunction are true. We summarise this in the following truth table.

A	B	$A \vee B$
T	**T**	**T**
T	**F**	**T**
F	**T**	**T**
F	**F**	**F**

Remark

Remember that the disjunction operator always corresponds to the use of an *inclusive* "or": it is true if one *or both* of the two terms are true.

Remark

The notation A OR B is sometimes used instead of $A \vee B$.

Example

We still have some more logical operations to introduce, but let's look at an example building a logical statement from propositions using negation, conjunction, and disjunction. Consider the statement $(P_1 \wedge P_2) \vee (\neg P_3)$. (We are using brackets around parts of the statement to make it clearer how to construct it.) We start building this statement by taking the conjunction of P_1 and P_2 to get $P_1 \wedge P_2$. We then negate P_3 to get $\neg P_3$. Finally we take the disjunction of $(P_1 \wedge P_2)$ and $(\neg P_3)$ and obtain $(P_1 \wedge P_2) \vee (\neg P_3)$.
The truth value of this statement will depend on the truth values of P_1, P_2, and P_3. For example, say that P_1 is false, P_2 is true, and P_3 is false. Now $P_1 \wedge P_2$ is false because P_1 is false. However, $\neg P_3$ is true because P_3 is false. Therefore the disjunction $(P_1 \wedge P_2) \vee (\neg P_3)$ is true, because one of the two terms in the disjunction is true. (In particular, $\neg P_3$ is true.)
If we calculated the truth value of the statement for all possible truth values of P_1, P_2, and P_3, we would arrive at the following truth table.

P_1	P_2	P_3	$(P_1 \wedge P_2) \vee (\neg P_3)$
T	**T**	**T**	**T**
T	**T**	**F**	**T**
T	**F**	**T**	**F**
T	**F**	**F**	**T**
F	**T**	**T**	**F**
F	**T**	**F**	**T**
F	**F**	**T**	**F**
F	**F**	**F**	**T**

Implication. Our next operation is called *implication*. Implication roughly corresponds to statements in English of the form "if A is true, then B is true". We write this as $A \Rightarrow B$, which we read as "if A then B" or "A implies B". We say that A is the *antecedent* of $A \Rightarrow B$, and B is the *consequent*.

We will first look at the truth table of $A \Rightarrow B$ and then discuss why it should have this form. The statement $A \Rightarrow B$ is false if A is true and B is false. Otherwise it is true. Here is the truth table.

A	B	$A \Rightarrow B$
T	**T**	**T**
T	**F**	**F**
F	**T**	**T**
F	**F**	**T**

The truth table for $A \Rightarrow B$ is more difficult to understand than the others we have seen. It might seem strange that $A \Rightarrow B$ is true if A is false, but this turns out to be a useful convention.

Think of an if-then statement as being like a promise. If I make the statement, $A \Rightarrow B$, I am promising that if A is true, then B is true. What does it mean for me to break my promise? The promise is only broken if A is true, but B is false. Under any other circumstances, I have kept my promise. In particular, I have kept my promise even if A is false. So we accept $A \Rightarrow B$ as being true when A is false.

Remark

You may see the notation $A \to B$ or A IMPLIES B used instead of $A \Rightarrow B$.

If and only if. Our last logical operation is easy to understand. Suppose that A and B are logical statements. Then $A \Leftrightarrow B$ is a logical statement, and it is true if the truth values of A and B are the same. If they are different, then $A \Leftrightarrow B$ is false.

A	B	$A \Leftrightarrow B$
T	T	T
T	F	F
F	T	F
F	F	T

We read $A \Leftrightarrow B$ as "A if and only if B".

Remark

Sometimes "if and only if" is abbreviated to "iff". Alternative notations for $A \Leftrightarrow B$ include $A \leftrightarrow B$ and A IFF B.

Truth tables. Suppose that we have a statement in propositional logic that is built from the atomic statements $P_1, \ldots, P_n$. If we know the truth values of these atomic statements, then we can build our way up to finding the truth value of the statement. Let's look at a statement that is built from three propositions:

$$(P_1 \Leftrightarrow P_2) \vee (P_3 \Rightarrow P_2).$$

In the column below $P_1 \Leftrightarrow P_2$, we will put the truth value $P_1 \Leftrightarrow P_2$ (using a small font, so we don't mistake this for the truth value of the whole statement). We will do the same for $P_3 \Rightarrow P_2$. The final column contains the truth value of $(P_1 \Leftrightarrow P_2) \vee (P_3 \Rightarrow P_2)$, so it is true exactly when at least one of the previous columns is true.

P_1	P_2	P_3	$(P_1 \Leftrightarrow P_2)$	$\vee$	$(P_3 \Rightarrow P_2)$
T	T	T	T	T	T
T	T	F	T	T	T
T	F	T	F	F	F
T	F	F	F	T	T
F	T	T	F	T	T
F	T	F	F	T	T
F	F	T	T	T	F
F	F	F	T	T	T

Satisfiability, validity, equivalence.

Definition 2.1. Suppose that A is a statement, built from the atomic statements $P_1, \ldots, P_n$. We say that A is *satisfiable* (or *consistent*) if A is true in at least one row of its truth table. If A is false in every row of its truth table, we say it is *unsatisfiable* (sometimes we say such a statement is *inconsistent* or a *contradiction*). If A is true in every row of its truth table, then it is a *tautology*.

Example

Suppose that A, B, and C are statements built from the atomic statements P_1 and P_2, and that the truth tables for A, B, and C are given as follows:

P_1	P_2	A	B	C
T	T	F	F	T
T	F	T	F	T
F	T	T	F	T
F	F	F	F	T

Then A is satisfiable, and B is unsatisfiable. C is a tautology (and is also satisfiable). In fact, a tautology is true in every row, and is therefore true in at least one row. This shows that every tautology is also satisfiable.

It is easy to see that the next result is true.

Proposition 2.2. *Suppose that A is any statement. Then $A \vee (\neg A)$ is a tautology and $A \wedge (\neg A)$ is unsatisfiable.*

The fact that $A \vee (\neg A)$ is always true is the *law of the excluded middle*. It says that if a statement is false, then its negation is true.

Definition 2.3. Suppose that A and B are statements. We say that A and B are *equivalent* if $A \Leftrightarrow B$ is a tautology.

In other words, A and B are equivalent if A is true exactly when B is true.

Example

We want to show that P_1 and $\neg(\neg P_1)$ are equivalent. If we construct the truth table for $P_1 \Leftrightarrow (\neg(\neg P_1))$ we see that it looks as follows:

P_1	$P_1 \Leftrightarrow (\neg(\neg P_1))$
T	T
F	T

Therefore $P_1 \Leftrightarrow (\neg(\neg P_1))$ is a tautology, so P_1 and $\neg(\neg P_1)$ are indeed equivalent.

Proposition 2.4. *Let A and B be any statements. Then $A \Rightarrow B$ is equivalent to $(\neg A) \vee B$.*

Proof. We already know that the truth table for $A \Rightarrow B$ looks as follows:

A	B	$A \Rightarrow B$
T	T	T
T	F	F
F	T	T
F	F	T

On the other hand, it is not difficult to verify that the truth table for $(\neg A) \vee B$ is

A	B	$(\neg A)\vee B$
T	**T**	**T**
T	**F**	**F**
F	**T**	**T**
F	**F**	**T**

Therefore $A \Rightarrow B$ and $(\neg A) \vee B$ have the same truth tables, so $(A \Rightarrow B) \Leftrightarrow ((\neg A) \vee B)$ is a tautology. This means that $A \Rightarrow B$ and $(\neg A) \vee B$ are equivalent. □

The next result is easy to prove.

Proposition 2.5. *Let A be any statement.*

(i) *If B is a tautology, then $A \vee B$ is a tautology, and $A \wedge B$ is equivalent to A.*

(ii) *If C is unsatisfiable, then $A \vee C$ is equivalent to A, and $A \wedge C$ is unsatisfiable*

Next we collect a few more equivalences. They are all easy to confirm, using truth tables.

Proposition 2.6. *Let A, B, and C be any statements. The following pairs of statements are equivalent.*

Double negation. *A and $\neg(\neg A)$.*

Commutative laws.

(i) *$A \wedge B$ and $B \wedge A$.*

(ii) *$A \vee B$ and $B \vee A$.*

Associative laws.

(i) *$A \wedge (B \wedge C)$ and $(A \wedge B) \wedge C$.*

(ii) *$A \vee (B \vee C)$ and $(A \vee B) \vee C$.*

Distributive laws.

(i) *$A \vee (B \wedge C)$ and $(A \vee B) \wedge (A \vee C)$.*

(ii) *$A \wedge (B \vee C)$ and $(A \wedge B) \vee (A \wedge C)$.*

Idempotent laws.

(i) *$A \wedge A$ and A.*

(ii) *$A \vee A$ and A.*

De Morgan's laws.

(i) *$\neg(A \wedge B)$ and $(\neg A) \vee (\neg B)$.*

(ii) *$\neg(A \vee B)$ and $(\neg A) \wedge (\neg B)$.*

It is very useful to be familiar with equivalent forms of negated logical statements. De Morgan's laws tell us how to negate conjunction and disjunctions. The next result tells us how to negate an implication.

Proposition 2.7. *Let A and B be statements. Then $A \Rightarrow B$ is equivalent to $(\neg A) \vee B$. The negation $\neg(A \Rightarrow B)$ is equivalent to $A \wedge (\neg B)$.*

Proof. The first equivalence is Proposition 2.4. Now $\neg(A \Rightarrow B)$ is equivalent to $\neg((\neg A) \vee B)$. We use De Morgan's laws to see that this is equivalent to $(\neg(\neg A)) \wedge (\neg B)$. By double negation, this is equivalent to $A \wedge (\neg B)$. □

2.2 Predicate logic

Propositional logic is probably the simplest logical language there is. It is useful for gaining intuition for logical languages, but it is not expressive enough to say very much. So we will look at a more sophisticated language called *predicate logic* or *first-order logic*. In this logic we have statements called *predicates*, which may be true for some of the objects under discussion, but not for others. For example, suppose we are discussing the *positive integers*, which are the numbers $1, 2, 3, \ldots$. Then we might want a predicate that allows us to say a number is even. We use upper-case letters to stand for predicates, so let's use E to stand for the evenness predicate. Then $E(2)$ stands for the statement "2 is even", so $E(2)$ is true. As another example, $E(5)$ stands for the statement "5 is even", so $E(5)$ is false. If we wanted to talk about prime numbers we might use P to stand for a primeness predicate. Then $P(11)$ would be true, and $P(9)$ would be false.

Remark

Recall that a positive integer is *prime* if it can be divided without remainder by precisely two positive integers. For example, 2 can be divided without remainder by 1 and 2, and no other positive integer. Therefore 2 is prime. On the other hand, 4 can be divided without remainder by 1, 2, and 4, so 4 is not prime. The first few primes are $2, 3, 5, 7, 11, 13, 17, 19, 23, \ldots$. Note that this definition excludes 1 as a prime number, because it cannot be divided by *exactly two* positive integers. The only positive integer that divides 1 is 1 itself. A few centuries ago, it was common to include 1 as a prime number, but the universal modern convention is that 1 is not prime.

Often, we want to use a predicate without having to specify a particular number. For this we need *variables*. We use x, y, z, and symbols such as $x_1, x_2, x_3, \ldots$ as variables. These variables stands for unspecified numbers. So now $E(x)$ stands for the statement "x is even". Note that $E(x)$ is neither true nor false, since the number x is not specified.

The set of objects we want to talk about is called the *domain*. It might be the set of positive integers, or perhaps some other mathematical set. Assume the domain is the set of positive integers. We might want to talk about a relation between two numbers. For this we need a predicate which takes two numbers as input. For example, $G(x, y)$ could stand for the statement "x is greater than y". Then $G(14, 8)$ is true, and $G(2, 9)$ is false. We will normally use the notation $x > y$ instead of $G(x, y)$.

It is possible to use predicates to talk about the relations between three or more variables. However, for our purposes it is enough to use only *unary* predicates, which have a single variable as an input, and *binary* predicates, which take take pairs of variables as inputs.

Quantifiers. The real power of predicate logic is that we can now make statements about *all* the objects in the domain. The symbol $\forall$ is called the *universal quantifier*. We read $\forall$ as "for all" or "every". So $\forall x$ stands "for all objects x in the domain". For example, if our domain

is the set of positive integers then $\forall x\, E(x)$ stands for "for all positive integers x, x is even". We could reword this as "every positive integer is even". Note that even though $E(x)$ does not have a definite truth value, the statement $\forall x\, E(x)$ does have a truth value: it is false, because not every positive integer is even.

Example

We consider the domain of positive integers. Let $E(x)$ stand for "x is even", and let $O(x)$ stand for "x is odd". Then $E(x) \vee O(x)$ is neither true nor false, but

$$\forall x\, (E(x) \vee O(x))$$

is true. It stands for the statement "every positive integer is either even or odd".

We have talked about the universal quantifier. Next we introduce the *existential quantifier*, $\exists$. We read $\exists$ as "there exists" or "for some". So if our domain is the set of positive integers, then $\exists x\, E(x)$ stands for "there exists a positive integer x such that x is even". Therefore $\exists x\, E(x)$ is true. On the other hand, $\exists x\, (E(x) \wedge O(x))$ stands for "there exists a positive integer x such that x is even and x is odd". This is a false statement.

Remark

The domain we are working in makes a difference to the truth value of statements. If our domain is the set of positive integers then $\exists x\, (x > 1 \wedge 2 > x)$ is false, because it can be interpreted as "there is a positive integer that is greater than one, and less than two". No such positive integer exists, so this is false. But if our domain includes rational numbers (fractions), then the statement is true, because it stands for "there is a rational number that is greater than one, and less than two". This is true because there are many rational numbers that satisfy this description, for example $3/2$.

Remark

The order in which we apply quantifiers can make a big difference to a statement. Say our domain is the set of positive integers. Then $\forall x \exists y\, y > x$ says "for every positive integer x there is a positive integer y that is greater than x". Do you think this is true, or is it false? The sentence $\exists y \forall x\, y > x$ says "there exists a positive integer y such that for every positive integer x, y is greater than x". This can be reworded as "there exists a positive integer y that is greater than every positive integer". Is this a true statement?

Example

We consider the domain of positive integers. Let E be the evenness predicate and let P be the primeness predicate. Say we want to express the statement "for every positive integer x, there exists a positive integer y such that y is greater than x and y is prime". We can do so with the formula $\forall x \exists y\, (y > x \wedge P(y))$.
Now look at this formula: $\forall x\, ((E(x) \wedge P(x)) \Rightarrow \neg \exists y\, (x > y \wedge E(y)))$. Let's translate this into English: "for every positive integer x, if x is even and prime, then there does not exists a positive integer y such that x is greater than y and y is even". Do you think this sentence is true or false?

Negating quantifiers. The statement $\forall x\, P(x)$ has a definite truth value, so its negation $\neg\forall x\, P(x)$ also has a definite truth value. Is there another way of expressing this negation?

Many people make the mistake of thinking that $\neg(\forall x\, P(x))$ is equivalent to $\forall x\, (\neg P(x))$, but these statements are different. Say that our domain is the set of all cats, and that the predicate statement $B(x)$ stands for "x is black". Then $\forall x\, B(x)$ is the statement "for every cat x, x is black", or, rewording: "every cat is black". The statement $\forall x\, (\neg B(x))$ stands for "every cat is not black". These two statements cannot be the negation of each other because both are false statements! (Recall that the negation of statement A is true when A is false, and false when A is true.)

Imagine that someone said "every cat is black" and you want to contradict them by saying the negation of this statement. You would not say "every cat is not black". Instead you would say "at least one cat is not black". This statement can be expressed as $\exists x\, (\neg B(x))$. So we see that negating a universal quantifier produces an existential quantifer. This is true in general.

Proposition 2.8. *If A is any formula in predicate logic, then $\neg(\forall x\, A)$ is equivalent to $\exists x\, (\neg A)$.*

A similar thing happens when we negate an existential quantifier. Consider the statement "someone came to the party". We can express this formally as follows: let our domain be the set of all people. Let the predicate statement $P(x)$ stand for "x came to the party". Then the statement above can be expressed as $\exists x\, P(x)$. The negation of this is "no one came to the party", or "every person did *not* come to the party", which can be expressed as $\forall x\, (\neg P(x))$. So negation has changed an existential statement into a universal statement.

Proposition 2.9. *If A is any formula in predicate logic, then $\neg(\exists x\, A)$ is equivalent to $\forall x\, (\neg A)$.*

Summary

Important ideas from Section 2.

- The negation $\neg A$ is true exactly when A is false.
- The conjunction $A \wedge B$ is true exactly when A and B are *both* true. The disjunction $A \vee B$ is true exactly when *at least one* of A and B is true.
- The implication $A \Rightarrow B$ is false exactly when A is true and B is false.
- The equivalence $A \Leftrightarrow B$ is true exactly when A and B have the same truth value.
- Two statements A and B are equivalent if A is true exactly when B is true.
- $\forall x\, Q(x)$ means "for every object x in the domain, $Q(x)$ is true".
- $\exists x\, Q(x)$ means "there exists an object x in the domain such that $Q(x)$ is true".
- $\neg(\forall x\, A)$ is equivalent to $\exists x\, (\neg A)$ and $\neg(\exists x\, A)$ is equivalent to $\forall x\, (\neg A)$.

3 Proofs

We have explored a couple of different formal logical languages. Now it is time to think about logical arguments. For now, we will think only about propositional logic, so we temporarily forget about predicates and variables.

Definition 3.1. An *argument* consists of a collection of statements, called the *premises*, and another statement, called the *conclusion*. If $A_1, A_2, \ldots, A_n$ are the premises, and C is the conclusion, then we write $A_1, A_2, \ldots, A_n \therefore C$ to represent this argument. The $\therefore$ symbol is read as "therefore".

Example

Suppose that the premises of an argument are $P_1 \Rightarrow P_2$ and $P_1 \vee P_2$, and that the conclusion is P_2. We would write this argument as $P_1 \Rightarrow P_2,\ P_1 \vee P_2 \ \therefore P_2$.

Definition 3.2. Suppose that $A_1, A_2, \ldots, A_n \therefore C$ is an argument. We say that this argument is *valid* if the conclusion if true whenever the premises are all true. Otherwise the argument is *invalid*.

We could equivalently say that the argument is valid if

$$(A_1 \wedge A_2 \wedge \cdots \wedge A_n) \Rightarrow C$$

is a tautology. Note that this implication will only ever be false if $A_1 \wedge A_2 \wedge \cdots \wedge A_n$ is true and C is false. Furthermore, $A_1 \wedge A_2 \wedge \cdots \wedge A_n$ is only true if all the premises are true. Therefore, the only way an argument can be invalid is if it is possible for us to assign truth values in such a way that all the premises are true while the conclusion is false.

> *An argument is invalid if there is an assignment of truth values that makes all the premises true while making the conclusion false. Otherwise, the argument is valid.*

Example

Consider the argument $P_1 \vee P_2,\ \neg P_2 \ \therefore P_1$. We can determine whether this is a valid argument by examining the truth tables for all the premises and the conclusion simultaneously.

P_1	P_2	$P_1 \vee P_2$	$\neg P_2$	P_1
T	**T**	**T**	**F**	**T**
T	**F**	**T**	**T**	**T**
F	**T**	**T**	**F**	**F**
F	**F**	**F**	**T**	**F**

There is only one row where the premises are all true: namely the second row. The conclusion is also true in this row. So the truth of the premises forces the truth of the conclusion, and therefore this is a valid argument.

Example

Consider the argument $P_1 \vee P_2,\ P_2\ \therefore P_1$.

P_1	P_2	$P_1 \vee P_2$	P_2	P_1
T	**T**	**T**	**T**	**T**
T	**F**	**T**	**F**	**T**
F	**T**	**T**	**T**	**F**
F	**F**	**F**	**F**	**F**

There is a row in this table where the premises are true but the conclusion is false (the third row). Therefore the argument is invalid.
Next consider the following argument. $P_1,\ \neg P_1\ \therefore P_2$.

P_1	P_2	P_1	$\neg P_1$	P_2
T	**T**	**T**	**F**	**T**
T	**F**	**T**	**F**	**F**
F	**T**	**F**	**T**	**T**
F	**F**	**F**	**T**	**F**

There is no row where both the premises are true. Surprisingly, this means that the argument is valid. To see why, consider the statement

$$(P_1 \wedge (\neg P_1)) \Rightarrow P_2.$$

The antecedent of the implication $P_1 \wedge (\neg P_1))$ is always false. This means that the implication is always true and therefore the argument is valid. Another way of thinking about this is as follows: an argument is invalid only if you can simultaneously make the premises true while the conclusion is false. If you can't do that, then the argument is valid. It is impossible to make P_1 and $\neg P_1$ simultaneously true, so the argument $P_1,\ \neg P_1\ \therefore P_2$ is valid. This last paragraph says that if we accept $P_1 \wedge (\neg P_1)$ as being true, then we are forced to accept P_2. This seems weird, since P_2 has nothing to do with P_1. If we accept $P_1 \wedge (\neg P_1)$, then we are forced to accept a completely arbitrary statement. In other words, if we start from a false statement, then we can prove anything we like. This is known as the *principle of explosion* and it is one of the more counter-intuitive features of classical logic.

3.1 Fitch diagrams

In most cases it is not practical to use a truth table to decide whether an argument is or not. Consider the argument

$$P_1 \Rightarrow P_2,\ P_2 \Rightarrow P_3,\ P_3 \Rightarrow P_4,\ P_4 \Rightarrow P_5,\ P_5 \Rightarrow P_6,\ P_1\ \therefore P_6$$

If we checked the validity of this argument with a truth table, we would need a table with 64 rows! Moreover, if we want to consider the validity of arguments in predicate logic, then we cannot use truth tables. We need another way of testing whether an argument is valid.

A mathematical proof is essentially a confirmation that a particular argument is valid. A proof is a sequence of statements, each one of which can be derived from the previous statements. We are going to study proofs in a formal setting by looking at *rules of inference*.

Assume that we want to use these rules to prove that $A_1, \ldots, A_n \therefore C$ is a valid argument. The idea is that we start with the premises $A_1, \ldots, A_n$, and by applying the rules of inference, we produce a sequence of statements $B_1, B_2, B_3, \ldots$. Each statement in this sequence is produced from previous statements by applying the rules. We choose the rules in such a way that if we accept the premises as being true, then we are forced to accept all of the statements $B_1, B_2, B_3, \ldots$ as being true. If we eventually produce the conclusion C by applying the rules of inference, then we have shown that C is always true as long as $A_1, \ldots, A_n$ are true. In other words, we will have shown that $(A_1 \wedge \cdots \wedge A_n) \Rightarrow C$ is always true, and that therefore the argument $A_1, A_2, \ldots, A_n, \therefore C$ is valid.

The sequence of statements is arranged on a series of numbered lines, as follows.

1. $P_1 \wedge P_2$	Premise
2. $P_2 \Rightarrow P_3$	Premise
3. P_2	$\wedge$Elim 1
4. P_3	$\Rightarrow$Elim 2, 3

The small dash represents the end of the premises, so in this example the premises are $P_1 \wedge P_2$ and $P_2 \Rightarrow P_3$. On the right hand side, we justify each statement by saying that it is either a premise, or by listing the rule we have just used to derive it. This type of diagram is called a *Fitch diagram*.

The rules of inference work as follows: for each of our basic operators there is a rule to introduce a statement involving that operator, and another rule to eliminate that operator from a statement. Our first rule of inference eliminates a conjunction ($\wedge$) operator.

Conjunction elimination rule. Suppose that on line j, the statement $A \wedge B$ appears. This means we have accepted $A \wedge B$ as being true. If we have accepted $A \wedge B$, then we have accepted A. Therefore we can write A on a subsequent line, with the justification $\wedge$Elim j. This stands for "conjunction elimination, using line j". The conjunction elimination rule has two forms: if $A \wedge B$ appears on line j, then we have accepted $A \wedge B$, which means we can put either A or B on a later line, using the same justification.

Example

Here is how we apply the conjunction elimination rule to show that the argument $P_1 \wedge P_2 \therefore P_1$ is valid:

1. $P_1 \wedge P_2$	Premise
2. P_1	$\wedge$Elim 1

We have used the rules of inference to derive the conclusion of the argument, starting from the premises. Therefore the argument is valid. The first statement we accept simply because it is a premise. The second statement is justified by the conjunction elimination rule applied to statement number 1.

Our next rule introduces a conjunction statement.

Conjunction introduction rule. Assume that we have accepted that A is true, and also we have accepted that B is true. This means that we are forced to accept that $A \wedge B$ is true. So if A

appears on line j, and B appears on line k, then on a subsequent line we can put the statement $A \wedge B$, with the justification $\wedge$Intro j, k. This stands for "conjunction introduction using lines j and k".

Remark

It does not matter if line j comes before line k or after. If A is on line j and B is on line k, then we can write $A \wedge B$ with the justification $\wedge$Intro j, k, even if j is a larger number than k.

Example

Let's use the conjunction introduction rule to prove that the argument $P_1,\ P_2\ \therefore\ P_2 \wedge P_1$ is valid.

1. P_1	Premise
2. P_2	Premise
3. $P_2 \wedge P_1$	$\wedge$Intro 2, 1

Implication elimination rule. Assume we have accepted the statement $A \Rightarrow B$. So we have accepted that if A is true, then B is true. If we also accept that A is true, then we are forced to accept B as true. This justifies the following rule. Suppose that the statement $A \Rightarrow B$ appears on line j, and A appears on line k. Then we can put the statement B on a subsequent line, with the justification $\Rightarrow$Elim j, k, because we have eliminated an implication (using lines j and k).

Remark

This rule has a Latin name: *modus ponens*.

Example

We now use the three rules we have introduced so far to prove that the argument $P_1 \Rightarrow (P_2 \wedge P_3),\ P_1\ \therefore\ P_1 \wedge P_2$ is valid.

1. $P_1 \Rightarrow P_2 \wedge P_3$	Premise
2. P_1	Premise
3. $P_2 \wedge P_3$	$\Rightarrow$Elim 1, 2
4. P_2	$\wedge$Elim 3
5. $P_1 \wedge P_2$	$\wedge$Intro 2,4

Remark

When we place the justification $\Rightarrow$Elim j, k, we mean that the implication appears on line j and the antecedent of that implication appears on line k. It good discipline to make sure that the numbers appear in the correct order.

Implication introduction rule. Our next rule is the most complicated. We want to introduce statements of the form $A \Rightarrow B$. We do this by assuming that A is true, and then using this assumption to show that B is true. If we succeed in doing this, then we have proved the following: "if A is true, then B is true". In other words, we have shown that $A \Rightarrow B$ is a true statement. This is called *hypothetical* reasoning.

There are certain traps we need to be aware of. When we make the assumption that is A true, it is only so we can prove that $A \Rightarrow B$ is a true statement. We are not saying A *is* true, we are just making the temporary assumption that A is true in order to see what follows from this assumption. We do not want to make the mistake of thinking this hypothetical assumption is really true.

In order to avoid this mistake, we use indenting to remind us when we are making a hypothetical assumption. If we want to introduce the statement $A \Rightarrow B$, we start an indented Fitch subdiagram which will contain all the lines in our hypothetical argument. So we start the indented Fitch diagram with the statement A (say on line j) and use the justification "Assumption". Then we produce a series of statements using the rules of inference, all contained in the indented Fitch proof, until we produce the statement B (let us say on line k). We then end the indentation, and write the statement $A \Rightarrow B$, with the justification $\Rightarrow$Intro j-k.

After we have ended the indentation, we are subsequently not allowed to use any of the indented statements in our deduction. To say this in another way: as soon as we write down the justification $\Rightarrow$Intro j-k, all the statements on lines j-k become out of bounds. This is very important: the indented statements all relied on the hypothetical assumption that A is true. Once we have finished the proof that $A \Rightarrow B$ is true, we cannot use those hypothetical statements again. Sometimes we say that when we write the justification $\Rightarrow$Intro j-k, we have *discharged* the assumption on line j.

Example

Here is an example of the implication introduction rule in action. We want to show that $(P_1 \wedge P_2) \Rightarrow P_3,\ P_1 \ \therefore\ P_2 \Rightarrow P_3$ is a valid argument.

1. P_1	Premise
2. $(P_1 \wedge P_2) \Rightarrow P_3$	Premise
3. P_2	Assumption
4. $P_1 \wedge P_2$	$\wedge$Intro 1,3
5. P_3	$\Rightarrow$Elim 2,4
6. $P_2 \Rightarrow P_3$	$\Rightarrow$Intro 3-5

Note that if we were to continue this sequence of statements, we would be allowed to use statement 6, since the implication $P_2 \Rightarrow P_3$ is a true statement (assuming that all the premises are true). However, we would not be allowed to use any of the statements 3, 4, or 5. This is because as soon as we write the justification $\Rightarrow$Intro 3-5, we discharge the assumption on line 3, and all the statements on lines 3-5 become out of bounds.

Example

Sometimes it is necessary to have one indented environment inside another. This occurs when we require more than one assumption. Let's consider an example. We want to verify that $(P_1 \wedge P_2) \Rightarrow P_3 \ \therefore \ P_1 \Rightarrow (P_2 \Rightarrow P_3)$ is a valid argument.

1. $(P_1 \wedge P_2) \Rightarrow P_3$	Premise
2. P_1	Assumption
3. P_2	Assumption
4. $P_1 \wedge P_2$	$\wedge$Intro 2,3
5. P_3	$\Rightarrow$Elim 1,4
6. $P_2 \Rightarrow P_3$	$\Rightarrow$Intro 3-5
7. $P_1 \Rightarrow (P_2 \Rightarrow P_3)$	$\Rightarrow$Intro 2-6

Note that on line 4 we have used the assumption P_1, even though we are on a different level of indentation. This is allowed, because when we write line 4, we have not yet discharged the assumption P_1. (We don't do that until line 7.)

Disjunction introduction rule. If we have accepted A as being true, then we have also accepted $A \vee B$ as being true, no matter what B is. This may seem strange, because there does not need to be a connection between A and B. But nonetheless, if A is true, then so is $A \vee B$. This justifies the disjunction introduction rule. If A appears on line j, then we can put the statement $A \vee B$ on a subsequent line, where B is any statement at all. We give the justification $\vee$Intro j. This rule appears in two forms: if A appears on line j, then we can put either $A \vee B$ or $B \vee A$ on a subsequent line.

Example

We have now discussed the first five rules of inference. We will use them to show that $(P_1 \vee P_2) \Rightarrow P_3,\ P_3 \Rightarrow (P_4 \wedge P_5) \ \therefore P_1 \Rightarrow (P_3 \wedge P_5)$ is a valid argument.

1. $(P_1 \vee P_2) \Rightarrow P_3$	Premise
2. $P_3 \Rightarrow (P_4 \wedge P_5)$	Premise
3. P_1	Assumption
4. $P_1 \vee P_2$	$\vee$Intro 3
5. P_3	$\Rightarrow$Elim 1,4
6. $P_4 \wedge P_5$	$\Rightarrow$Elim 2,5
7. P_5	$\wedge$Elim 6
8. $P_3 \wedge P_5$	$\wedge$Intro 5,7
9. $P_1 \Rightarrow (P_3 \wedge P_5)$	$\Rightarrow$Intro 3-8

Disjunction elimination rule. Assume that we have accepted the statement $A \vee B$ as being true. If we also accept that $A \Rightarrow C$ and $B \Rightarrow C$ are true, then we must accept C, because either A or B must be true, and either one of these statements implies C. This justifies the disjunction

elimination rule. If $A \vee B$ appears on line j, $A \Rightarrow C$ appears on line k, and $B \Rightarrow C$ appears on line l, then we can write C on a subsequent line with the justification $\vee$Elim j, k, l.

Example

We consider the argument $P_1 \vee P_2,\ P_1 \Rightarrow P_2,\ P_2 \Rightarrow P_3\ \therefore P_3$.

1. $P_1 \vee P_2$	Premise
2. $P_1 \Rightarrow P_2$	Premise
3. $P_2 \Rightarrow P_3$	Premise
4. P_1	Assumption
5. P_2	$\Rightarrow$Elim 2,4
6. P_3	$\Rightarrow$Elim 3,5
6. $P_1 \Rightarrow P_3$	$\Rightarrow$Intro 4-6
7. P_3	$\vee$Elim 1,6,3

Negation introduction rule. So far none of our rules allow us to negate a statement. The next rule takes care of this. Imagine that we prove that A implies $B \wedge (\neg B)$. In other words, we accept $A \Rightarrow (B \wedge (\neg B))$ as being true. But $B \wedge (\neg B)$ is a contradiction: it is never true. Can A be true? No, because then A would be true, but $B \wedge (\neg B)$ would be false, so $A \Rightarrow (B \wedge (\neg B))$ would be false. This is impossible since we have accepted $A \Rightarrow (B \wedge (\neg B))$ as being true. So if we accept $A \Rightarrow (B \wedge (\neg B))$ as being true, then we must accept that A is false. That is, we accept $\neg A$ as being true. This justifies the negation introduction rule: if $A \Rightarrow (B \wedge (\neg B))$ appears on line j, then we can write $\neg A$ on a subsequent line with the justification $\neg$Intro j.

We can think of the contradiction $B \wedge (\neg B)$ as being something impossible. So what we are saying with this rule is that if A implies something impossible, then A cannot be true, and therefore $\neg A$ is true. This is connected to the idea of a *proof by contradiction*. We assume that A is true, and from this assumption we prove a contradiction, that is, something impossible. This tells us that $\neg A$ must be true.

Example

We will prove the validity of the argument $P_1 \Rightarrow (\neg P_2)\ \therefore P_2 \Rightarrow (\neg P_1)$.

1. $P_1 \Rightarrow (\neg P_2)$	Premise
2. P_2	Assumption
3. P_1	Assumption
4. $\neg P_2$	$\Rightarrow$Elim 1,3
5. $P_2 \wedge (\neg P_2)$	$\wedge$Intro 2,4
6. $P_1 \Rightarrow (P_2 \wedge (\neg P_2))$	$\Rightarrow$Intro 3-5
7. $\neg P_1$	$\neg$Intro 6
8. $P_2 \Rightarrow (\neg P_1)$	$\Rightarrow$Intro 2-7

Remark

Be careful with the negation introduction. It does *not* mean that we can simply apply a negation operator to any statement we see and then justify it with the negation introduction rule. We can *only* apply the negation operator to A if A implies something impossible. That is, we can only justify $\neg A$ if we see a statement of the form $A \Rightarrow (B \wedge (\neg B))$.

Remark

We could justify the introduction of $\neg A$ if A implies any impossible thing. But when we apply the negation introduction rule, we insist that the impossible thing has the specific form $B \wedge (\neg B)$. This is because it is very easy to recognise this statement as a contradiction: something that can never be true. Other statements that are never true are harder to recognise.

Double negation rule. Our final rule uses the logical equivalence of the statements A and $\neg(\neg A)$. If we accept $\neg(\neg A)$ as being true, then obviously we must accept A, since they are logically equivalent. So if $\neg(\neg A)$ appears on line j, then on a subsequent line we can put A, with the justification $\neg\neg\, j$. Similarly, if A appears on line j, then we can introduce a double negation, and put $\neg(\neg A)$ on a subsequent line, with the same justification, $\neg\neg\, j$. We call this the *double negation* rule.

Example

Let us prove that $(\neg P_1) \Rightarrow P_2$, $\neg P_2 \ \therefore P_1$ is a valid argument.

1. $(\neg P_1) \Rightarrow P_2$	Premise
2. $\neg P_2$	Premise
3. $\neg P_1$	Assumption
4. P_2	$\Rightarrow$Elim 1,3
5. $P_2 \wedge (\neg P_2)$	$\wedge$Intro 4,2
6. $(\neg P_1) \Rightarrow (P_2 \wedge (\neg P_2))$	$\Rightarrow$Intro 3-5
7. $\neg(\neg P_1)$	$\neg$Intro 6
8. P_1	$\neg\neg$7

This completes the description of our rules of inference.

Remark

We have not discussed rules to introduce or eliminate the equivalence operator. But it is easy to check that $A \Leftrightarrow B$ is equivalent to

$$(A \Rightarrow B) \wedge (B \Rightarrow A).$$

Therefore every statement involving equivalence can be restated using only implication ($\Rightarrow$) and conjunction ($\wedge$). For this reason we will omit rules about equivalence.

Example

In our last example we will consider the argument $P_1 \vee P_2$, $\neg P_2 \ \therefore P_1$. The following proof shows that this is a valid argument.

1. $P_1 \vee P_2$	Premise
2. $\neg P_2$	Premise
3. P_1	Assumption
4. $\neg P_1$	Assumption
5. $P_1 \wedge (\neg P_1)$	$\wedge$Intro 3,4
6. $\neg P_1 \Rightarrow (P_1 \wedge (\neg P_1))$	$\Rightarrow$Intro 4-5
7. $\neg(\neg P_1)$	$\neg$Intro 6
8. P_1	$\neg\neg$ 7
9. $P_1 \Rightarrow P_1$	$\Rightarrow$Intro 3-8
10. P_2	Assumption
11. $\neg P_1$	Assumption
12. $P_2 \wedge (\neg P_2)$	$\wedge$Intro 10,2
13. $(\neg P_1) \Rightarrow (P_2 \wedge (\neg P_2))$	$\Rightarrow$Intro 11-12
14. $\neg(\neg P_1)$	$\neg$Intro 13
15. P_1	$\neg\neg$ 14
16. $P_2 \Rightarrow P_1$	$\Rightarrow$Intro 10-15
17. P_1	$\vee$Elim 1, 9, 16

The following table summarizes our rules of inference.

Name	**Previous statement(s)**	**New statement**	**Justification**
Conjunction Elimination	$j.\ A \wedge B$	A	$\wedge$Elim j
	$j.\ A \wedge B$	B	$\wedge$Elim j
Conjunction Introduction	$j.\ A$ $k.\ B$	$A \wedge B$	$\wedge$Intro j,k
Implication Elimination	$j.\ A \Rightarrow B$ $k.\ A$	B	$\Rightarrow$Elim j,k
Implication Introduction	$j.A$ $\vdots$ $k.B$	$A \Rightarrow B$	$\Rightarrow$Intro j-k
Disjunction Introduction	$j.\ A$	$A \vee B$	$\vee$Intro j
	$j.\ A$	$B \vee A$	$\vee$Intro j
Disjunction Elimination	$j.\ A \vee B$ $k.\ A \Rightarrow C$ $l.\ B \Rightarrow C$	C	$\vee$Elim j,k,l
Negation Introduction	$j.\ A \Rightarrow (B \wedge (\neg B))$	$\neg A$	$\neg$Intro j
Double Negation	$j.\ \neg(\neg A)$	A	$\neg\neg\ j$
	$j.\ A$	$\neg(\neg A)$	$\neg\neg\ j$

Remark

We know certain logical equivalences, such as the equivalence of $\neg(P_1 \wedge P_2)$ and $(\neg P_1) \vee (\neg P_2)$. It may be tempting to simply substitute in such an equivalence when constructing a proof using the rules of inference. But an equivalence like this is not included in our rules. If we want to use such an equivalence, then we need to justify it by applying the rules. The point of this exercise is to use only the rules we have agreed upon, and to strictly use only these rules.

Remark

The rules of inference are not unique. The list of rules we have presented is relatively short and simple so it is easy to memorise and implement, but there are other valid choices of rules we could have used.

Remark

We have now developed a *proof system*, which consists of a logical language (in our case propositional logic), along with rules of inference. Our system is *consistent* (or *sound*), which means starting from the premises it is not possible to derive both a statement B and its negation $\neg B$ (assuming that the premises are not contradictory). In other words, if it is possible for all the premises to be simultaneously true, then we cannot derive a contradiction $B \wedge (\neg B)$. Our system is also *complete*, meaning that if an argument is valid, then we can demonstrate that it is valid using our rules. In other words, these rules are sufficient to demonstrate the validity of any valid argument.
Completeness and consistency are both extremely desirable properties of a proof system. A fundamental theorem by Kurt Gödel, which he proved in 1929, tells us that it is usually impossible to have both. If a proof system is sufficiently powerful, meaning that the logic is capable of expressing statements about arithmetic, then it cannot be both consistent and complete. The idea of the proof is that we use arithmetic to encode a statement that says "this statement cannot be proved". If this statement can be proved, then it is false, so our system has proved a false statement, meaning that the system is not consistent. On the other hand, if the statement cannot be proved then it is true but cannot be proved, so the system is not complete. Our system of propositional logic is not powerful enough for Gödel's theorem to apply, so we can enjoy the properties of consistency and completeness. Gödel's theorem means that if a system is consistent then it must contain statements which are true but which cannot be proved. This fact completely revolutionised the philosophy of mathematics.

3.2 Direct proofs

We move away from the formal setting of a rules of inference, and talk more about proofs as humans read and write them. A *direct proof* starts from its premises and proceeds to the conclusion by producing a sequence of statements, as in the next example.

Proposition 3.3. *Let a and b be integers. If a and b are even, then so is $a + b$.*

Proof. Since a is even, it can be written as $2m$, where m is an integer. Since b is even, it can be written as $2n$, where n is an integer. So $a + b$ can be written as $2m + 2n$. This is equal to $2(m + n)$.

As both m and n are integers, so is $m + n$. So $a + b$ can be written as $2(m + n)$, which is 2 times an integer. Therefore $a + b$ is even. □

3.3 Contrapositive proofs

Imagine that we want to prove that X implies Y. So for some statements X and Y, we want to prove that $X \Rightarrow Y$ is true. As we have already said, a *direct proof* of $X \Rightarrow Y$ will start from the assumption X, and then prove a sequence of statements, ultimately arriving at Y. Now we will talk about two other very common styles of proof.

A *contrapositive proof* uses the fact that $X \Rightarrow Y$ and $(\neg Y) \Rightarrow (\neg X)$ are equivalent (check this using a truth table). If we can prove that $(\neg Y) \Rightarrow (\neg X)$ is true, then we have also proved that $X \Rightarrow Y$ is true.

We give an example of a contrapositive proof. Recall that the positive integer p is a *prime* if there are exactly two positive integers that divide p. The biggest known primes are all of the form $2^z - 1$, where z is a positive integer. Primes of this type are called *Mersenne* primes.

Proposition 3.4. *If $2^z - 1$ is prime, then z is prime.*

Proof. We let X be the premise "$2^z - 1$ is prime", and let Y be the conclusion "z is prime".

We assume that $\neg Y$ is true, and use this assumption to prove that $\neg X$ is true. The statement $\neg Y$ stands for "z is not prime". Since z is not prime, there is a positive integer x that divides z where x is not equal to 1 or z. Since x divides z, we can write z as $z = kx$, where k is a positive integer. Since $x \neq 1$ and $x \neq z$, we see that the positive integer $2^x - 1$ is not equal to 1 or $2^z - 1$. Now

$$(2^x - 1)(1 + 2^x + 2^{2x} + 2^{3x} + \cdots + 2^{(k-1)x}) = (2^x + 2^{2x} + 2^{3x} + \cdots + 2^{kx}) - (1 + 2^x + 2^{2x} + \cdots + 2^{(k-1)x}) = 2^{kx} - 1 = 2^z - 1.$$

This shows that $2^x - 1$ divides $2^z - 1$. Since $2^x - 1$ is not 1 or $2^z - 1$, this means that $2^z - 1$ is not prime. Therefore $\neg X$ is true.

We have shown that $\neg Y$ implies $\neg X$. Therefore X implies Y, and the proof is complete. □

3.4 Proof by contradiction

We discussed this style of proof when we were talking about the negation introduction rule. Suppose that we want to prove the statement X. We prove that $\neg X$ implies $P \wedge (\neg P)$, for some statement P. That is, we prove that

$$(\neg X) \Rightarrow (P \wedge (\neg P))$$

is true. Since $P \wedge (\neg P)$ is definitely false, there is only one way for the implication $(\neg X) \Rightarrow (P \wedge (\neg P))$ to be true: $\neg X$ must also be false. As $\neg X$ is false, X is true. To be more informal, a proof by contradiction works by showing that $\neg X$ implies something impossible (namely $P \wedge (\neg P)$). Therefore $\neg X$ can't be true, so X must be true.

We will illustrate proof by contradiction with one of the most famous proofs in mathematics.

Theorem 3.5. *There is an infinite number of primes.*

Proof. Let X be the statement "there is an infinite number of primes". We assume $\neg X$ is true, and show that this implies something impossible. Since $\neg X$ is the statement "there are finitely many primes", we assume that $p_1, p_2, \ldots, p_n$ is the complete list of prime numbers. Let z be $p_1 p_2 \cdots p_n + 1$. Every integer greater than 1 has a prime factor (we will prove this later, in Theorem 4.4), so there is a prime p_i on our list such that p_i divides z. This means that z/p_i is an integer. On the other hand, when we divide z by p_i, we get

$$\frac{z}{p_i} = \frac{p_1 p_2 \cdots p_n + 1}{p_i} = \frac{p_1 p_2 \cdots p_n}{p_i} + \frac{1}{p_i} = p_1 p_2 \cdots p_{i-1} p_{i+1} \cdots p_n + \frac{1}{p_i}.$$

Now $p_1 p_2 \cdots p_{i-1} p_{i+1} \cdots p_n$ is a product of integers, so it is certainly an integer. But $1/p_i$ is not an integer, because p_i is greater than one, meaning that $1/p_i$ is not an integer. This shows that z/p_i is not an integer. If we let P be the statement "z/p_i is an integer", we have proved that $\neg X$ implies the impossible statement $P \wedge (\neg P)$. Therefore X must be true and the proof is complete. □

Summary

Important ideas from Section 3.

- An argument $A_1, A_2, \ldots, A_n, \therefore C$ consists of premises $A_1, A_2, \ldots, A_n$ and conclusion C.
- An argument is valid if the premises being true forces the conclusion to be true.
- If the premises are never simultaneously true, then the argument is valid. This is the principle of explosion.
- A direct proof starts from the premises, and then produces a sequence of statements, each of which is derived from the previous statements, ending with the conclusion.
- A contrapositive proof proves that $X \Rightarrow Y$ is true by proving that $(\neg Y) \Rightarrow (\neg X)$ is true.
- A proof by contradiction proves that X is true by showing that $\neg X$ implies something impossible.

4 Inductive proofs

We have already discussed direct proofs, contrapositive proofs, and proofs by contradiction. In this chapter we will discuss a fourth, very powerful, method of proof called *proof by induction*. We also refer to a proof by induction as an *inductive proof*. Inductive proofs are subtle, and it can take some time to feel comfortable with them. But it is worth investing effort in understanding this proof strategy, because it is so useful and so widely used in mathematics and theoretical computer science.

When we use an inductive proof, we often (but not always) are trying to prove something about the non-negative integers. (Remember that these are the numbers $0, 1, 2, 3, \ldots$.) Suppose that we want to prove that a statement holds for every non-negative integer. In other words, we have a predicate statement $P(x)$, and we would like to prove that the sentence $\forall x\, P(x)$ is true, where our domain is the set of non-negative integers. We start by proving that $P(0)$ is true. (This is often quite easy.) This step of the process is called the *base case* for the inductive argument.

Next comes the subtle part. We prove that if $P(0), P(1), \ldots, P(k)$ are all true, then $P(k+1)$ is also true. Here k is standing in for an arbitrary non-negative integer. In other words, *we prove that if P is true for all the non-negative integers up to an arbitrary point, then it is true for the next integer*. Equivalently, we prove that

$$(P(0) \wedge P(1) \wedge \cdots \wedge P(k)) \Rightarrow P(k+1)$$

is a true implication for every non-negative integer k. It may not be clear how we prove this step, and indeed, it can sometimes be very difficult. For now, let's not worry about how we prove this step. The important point is that, if we succeed in this step, then we have proved that $P(x)$ is true for all non-negative integers x. This is not immediately obvious, so we will spend some time trying to understand why it is true.

To see why, assume that $P(x)$ is false for some non-negative integer x. Then there is a smallest-possible non-negative integer x such that $P(x)$ is false. So now we choose x so that it is the smallest-possible non-negative integer such that $P(x)$ is false. We think of x as being a *smallest counterexample*. We have checked that $P(0)$ is true, so x cannot be 0, since $P(x)$ is false. Therefore x is greater than 0, which means that $0, 1, \ldots, x-1$ are non-negative integers that are less than x. Since x is the smallest-possible non-negative integer such that $P(x)$ is false, it follows that $P(0), P(1), \ldots, P(x-1)$ are true. But we know that if P is true for all non-negative integers up to $x-1$, then it is true for x. (We used k before rather than $x-1$, but that isn't important: the symbol is just a placeholder. The important point is that we proved that if P is true for all non-negative integers up to an arbitrary point, then it is true for the next number.) OK, so $P(0), P(1), \ldots, P(x-1)$ are all true, meaning that $P(x)$ is true as well. But we assumed that $P(x)$ was false, so we have a contradiction. What created this contradiction? It was the assumption that $P(x)$ was false for at least one non-negative integer x. Therefore we must reject this assumption: this means that $P(x)$ is true for every non-negative integer x, and our inductive proof is complete.

Now let's return to the question of how we prove that when $P(0), P(1), \ldots, P(k)$ are true, so is $P(k+1)$. We want to prove the implication

$$(P(0) \wedge P(1) \wedge \cdots \wedge P(k)) \Rightarrow P(k+1).$$

Remember how we introduced an implication in Section 3.1? We made an *assumption* and derived statements from that assumption. That's what we will do here as well. We *assume* $P(0), P(1), \ldots, P(k)$ are all true, and use this assumption to prove that $P(k+1)$ is true as well. This

will establish that $(P(0) \wedge P(1) \wedge \cdots \wedge P(k)) \Rightarrow P(k+1)$ is a true implication when k is an arbitrary non-negative integer. The assumption that $P(0), P(1), \ldots, P(k)$ are all true is called the *inductive assumption*.

So far, the description of inductive proof strategy has been very vague — we haven't said anything about what $P(x)$ might actually stand for. This is because induction is a very broad proof strategy — it can be applied to many different types of problems. Now we will leave the abstract description of induction behind, and move to some concrete examples of proofs that use induction.

The description we have given applies to the situation where we want to prove that a statement holds for every non-negative integer. In the next proof, and in many other situations, we want to prove that a statement holds for the positive integers. The proof strategy is exactly the same, except that we start with the base case $P(1)$ instead of $P(0)$. For the inductive step, we assume that $P(1) \wedge P(2) \wedge \cdots \wedge P(k)$ is true, and we use this assumption to prove that $P(k+1)$ is true.

Proposition 4.1. *Let n be a positive integer. Then*

$$1 + \cdots + n = \frac{n(n+1)}{2}.$$

Proof. First we prove the base case by checking that the statement holds for 1. The sum of all the positive integers up to 1 is just 1. Also, $1(1+1)/2 = 1$, so the base case holds.

Now we need to prove that if the statement is true up to an arbitary point, then it is true for the next positive integer. To do this, we make the inductive assumption that the statement holds for the numbers $1, 2, 3, \ldots, k$, and we will use this assumption to prove that it holds for $k+1$. Since the statement holds for k, we see that

$$1 + 2 + \cdots + k = \frac{k(k+1)}{2}.$$

Therefore

$$1 + 2 + \cdots + k + (k+1) = [1 + 2 + \cdots + k] + (k+1) = \frac{k(k+1)}{2} + (k+1).$$

Note how we used the inductive assumption when we replaced the contents of the square brackets with $k(k+1)/2$. This is the last time we use the inductive assumption, and at this point we have essentially completed the inductive proof! The rest of the proof is merely algebraic rearrangement. You may not feel confident with this algebraic manipulation, but don't confuse the difficulty of the algebra with the difficulty of the inductive proof. The inductive portion of the proof is now behind us. Let's continue:

$$\begin{aligned} 1 + 2 + \cdots + k + (k+1) &= [1 + 2 + \cdots + k] + (k+1) \\ &= \frac{k(k+1)}{2} + (k+1) \\ &= \frac{k(k+1)}{2} + \frac{2(k+1)}{2} \\ &= \frac{k(k+1) + 2(k+1)}{2} \\ &= \frac{(k+1)(k+2)}{2} \end{aligned}$$

The last expression is exactly the expression we get when we replace every occurrence of n with $k+1$ in the statement of the proposition. Therefore we have proved that when the statement holds for $1, 2 \ldots, k$, it also holds for $k+1$. In other words, if the equation is true up to an arbitrary positive integer, then it is true for the next positive integer. This is exactly what we need to show in order to complete the inductive proof. □

Remark

Very frequently, we do not need to know that the statement we are trying to prove inductively holds for all integers $1, 2, \ldots, k$. This is the case in the previous example. When we proved that the statement held for $k+1$, we used the fact that it held for k, but we did not use the fact that it held for $1, 2, \ldots, k-1$. Sometimes we will express this subtlety by saying that we have used *weak induction*, as opposed to *strong induction*, which really does use the fact that the statement holds for all of $1, 2, \ldots, k$.

When n is a positive integer, we define $n!$, which is called n *factorial*, to be $n \cdot (n-1) \cdots 2 \cdot 1$. So, for example, $3! = 3 \cdot 2 \cdot 1 = 6$. Recall also that the notation $x \geq y$ means x is greater than or equal to y.

Proposition 4.2. *Let n be a positive integer. Then $n! \geq 2^{n-1}$.*

Proof. Does the statement hold for 1? The left hand side is $1! = 1$, and the right hand side is $2^{1-1} = 2^0 = 1$, so we have checked that the base case holds. Now we make the inductive assumption that the statement holds for the positive integers $1, 2, \ldots, k$. We will use this assumption to prove that the statement holds for $k+1$.

$$\begin{aligned}(k+1)! &= (k+1) \cdot k \cdot (k-1) \cdots 2 \cdot 1 \\ &= (k+1)[k \cdot (k-1) \cdots 2 \cdot 1] \\ &= (k+1)k! \\ &\geq (k+1)2^{k-1}\end{aligned}$$

In the last step we used the inductive assumption that $k!$ is greater than or equal to 2^{k-1}. Since k is a positive integer, it is at least equal to 1. Therefore $k+1 \geq 2$. Now we have

$$(k+1)! \geq (k+1)2^{k-1} \geq 2 \cdot 2^{k-1} = 2^k.$$

This inequality is what we get when we substitute $k+1$ for n in the equation given by the proposition. Therefore the inductive assumption that the statement holds for $1, 2, \ldots, k$ implies that it holds for $k+1$. This completes the inductive proof. □

Here is one more example.

Proposition 4.3. *Let n be a non-negative integer. Then*

$$1 + 2 + 4 + 8 + \cdots + 2^n = 2^{n+1} - 1.$$

Proof. We check the statement holds when $n = 0$. The left hand side is just 1, and the right hand side is $2^{0+1} - 1 = 2^1 - 1 = 2 - 1 = 1$, so the base case holds. We make the inductive assumption that the statement holds for $0, 1, 2, \ldots, k$. Then

$$1 + 2 + 4 + \cdots + 2^k + 2^{k+1} = [1 + 2 + 4 + \cdots + 2^k] + 2^{k+1}$$

$$
\begin{aligned}
&= (2^{k+1} - 1) + 2^{k+1} \\
&= 2 \times 2^{k+1} - 1 \\
&= 2^{k+2} - 1
\end{aligned}
$$

This shows that the statement is true for $k + 1$. Therefore the inductive assumption that the statement holds for $0, 1, 2, \ldots, k$ implies that it holds for $k + 1$, and the proof is complete. □

Very often students are introduced to induction using examples like the one in the last few propositions. This can give the false impression that induction is only used for proving equalities and inequalities between expressions involving positive integers. In fact, we can use induction in many many different contexts. You are going to see several inductive proofs by the end of the module. The next proof is one of the fundamental facts of arithmetic. We are going to prove it by induction, but the inductive proof is phrased in a slightly different way.

Theorem 4.4. *Any integer greater than* 1 *is either prime or is equal to a product of prime numbers.*

Proof. We are trying to prove that $P(x)$ is true for every positive integer (except for 1), where $P(x)$ is the statement "x is either prime or is equal to a product of prime numbers". We assume that the theorem is false. Then $P(x)$ is false for at least one positive integer x that is greater than 1. We choose x so that it is as small as possible amongst all such positive integers. Therefore x is an integer greater than 1, but x is not prime, and it cannot be written as a product of primes. Furthermore, if y is a positive integer greater than 1 and y is smaller than x, then $P(y)$ is true, so either y is prime, or it can be written as a product of primes.

We have said that x is not prime. Therefore it can be expressed as $x = yz$, where y and z are positive integers and neither y nor z is equal to 1 or x. This means that y and z are both greater than 1, since they are positive integers and they are not equal to 1. If y were larger than x, then yz would be larger than x, and this is impossible, since yz is *equal* to x. Therefore y is not greater than x. It is also not equal to x, so it is smaller than x. By using exactly the same arguments, we can prove that z is smaller than x.

Now here is the important part: the part where we use the inductive argument. Since y is a positive integer that is greater than 1, and y is smaller than x, it follows that $P(y)$ is true, because we chose x to be the smallest positive integer such that $P(x)$ was false. Since $P(y)$ is true, y is either prime, or it can be written as a product of prime numbers. Let y be equal to $p_1 p_2 \cdots p_m$, where $p_1, \ldots, p_m$ are prime numbers. If y is prime, then there is only one number in this sequence, so $y = p_1$. By using exactly the same argument, we can show that $P(z)$ is true, so we can write z as $z = q_1 q_2 \cdots q_n$, where $q_1, \ldots, q_n$ are all prime. Now $x = yz = p_1 \cdots p_m q_1 \cdots q_n$, so x can be expressed as a product of primes. We have reached a contradiction. We said that $P(x)$ was false, but we have just shown that x is a product of primes, so $P(x)$ is true. We started by assuming that P is not true for every positive integer, and we just reached the contradiction $P(x) \wedge (\neg P(x))$. Therefore our assumption must have been false: P *is* true for every positive integer. This is exactly what we wanted to prove. □

You may see Theorem 4.4 stated in the following way: *every positive integer is a product of prime numbers*. This relies on the convention that the product of a sequence of numbers is equal to 1 if that sequence is empty, and if the sequence contains a single number, then the product of that sequence is just that number. So 1 is therefore the product of an empty sequence of prime numbers. And a prime, like 2, is the product of the sequence that contains 2 and no other number.

We can prove a stronger fact: *every positive integer can be expressed as a product of primes in a* unique *way (if we ignore reordering the primes)*. This is known as the *Fundamental Theorem of Arithmetic*. We will not prove the uniqueness of the product.

Remark

The proof of Theorem 4.4 doesn't look the same as the inductive proofs we have seen up to this point. There is no inductive assumption that the theorem is true for the integers $1, 2, 3, \ldots, k$. However, this *is* an inductive proof — it's just disguised. Instead of making the standard inductive assumption, we instead assume that the theorem fails, and we select a *smallest possible counterexample*. We then obtain a contradiction from the fact that this is the smallest counterexample. Every inductive proof can be phrased in this way — the two types of argument are equivalent.

Inductive proofs are often used in computer science to verify the correctness of an algorithm. We will illustrate this by looking at a *sorting algorithm*. Such an algorithm receives as input a list of numbers, and outputs the same list of numbers, but sorted into increasing order. There is a huge number of sorting algorithms, and they are an important area of study in computer science. We will look at an algorithm called *bubble sort*. This is not an especially good sorting algorithm — others run faster. But it is simple enough to easily understand, which makes it useful for our purposes.

We are given a list of numbers $a_1, a_2, \ldots, a_n$. When we say that we will *swap* a_i and a_{i+1}, we mean that we change the list so that values assigned to a_i and a_{i+1} are swapped. For example, if

$$a_1 = 5 \qquad a_2 = 4 \qquad a_3 = 6 \qquad a_4 = 2 \qquad a_5 = 3$$

then swapping a_3 and a_4 produces

$$a_1 = 5 \qquad a_2 = 4 \qquad a_3 = 2 \qquad a_4 = 6 \qquad a_5 = 3.$$

In bubble sort, we repeatedly look at consecutive pairs in the list, and swap them if the first number is larger. If we can make our way through the list without performing any swaps, then we know that we have correctly sorted the numbers into increasing order. We keep track of whether we have performed a swap using a *Boolean variable* named `Swap`. A Boolean variable is one that takes on the value of either true or false.

The pseudocode for bubble sort is in Algorithm 2.

In the next result we will use induction to prove that the bubble sort algorithm correctly orders the numbers in the input list.

Proposition 4.5. *After the while loop in the bubble sort algorithm has been carried out k times, the k largest numbers in the list $a_1, a_2, \ldots, a_n$ are in the correct position (when the list is sorted in increasing order).*

Proof. We prove this statement by induction on k. If $k = 0$, then no **while** loops have been performed at all, and the statement does not say that any number is in its correct position. So the base case of the inductive argument is trivial to verify.

Now we will make the inductive assumption: if the **while** loop has been carried out k times then the largest k numbers are correctly positioned. We will use this assumption to prove that when the **while** loop has been carried out $k+1$ times, the largest $k+1$ numbers are in the correct position.

Algorithm 2 The bubble sort algorithm

Input: A list of numbers $a_1, a_2, \ldots, a_n$.
Output: A list of the same numbers sorted into increasing order.

```
Swap ← T
while Swap = T do
    Swap ← F
    for i = 1, 2, ..., n − 1 do
        if a_i > a_{i+1} then
            swap a_i and a_{i+1}
            Swap ← T
        end if
    end for
end while
```

Say that after k repetitions of the **while** loop, the list has the form $a_1, \ldots, a_{n-k}, a_{n-k+1}, \ldots, a_n$, where $a_{n-k+1}, \ldots, a_n$ are the largest numbers in the list, correctly ordered. (Note that there are k numbers in $a_{n-k+1}, \ldots, a_n$.) Now let us say that the largest number in $a_1, \ldots, a_{n-k}$ is a_j. When i reaches the value j in the **for** loop, a_i will be larger than a_{i+1}, unless $j = n - k$. This is because a_j is the largest value in $a_1, \ldots, a_{n-k}$. So if j is not $n - k$, then the algorithm will carry out a swap when $i = j$. After this swap, the largest value in $a_1, \ldots, a_{n-k}$ will be a_{j+1}. By the same reasoning, a_{j+1} will be larger than a_{j+2}, so another swap will be carried out. These swaps will continue until i reaches the value $n - k$. At this point, a_i will not be greater than a_{i+1}, because the largest value in $a_1, \ldots, a_{n-k}$ is no greater than the values in $a_{n-k+1}, \ldots, a_n$. From this point forward, no swaps will be carried out, because $a_{n-k+1}, \ldots, a_n$ are correctly ordered. So at the completion of the **while** loop, the largest value in $a_1, \ldots, a_{n-k}$ will be a_{n-k}. This means that $a_{n-k}, a_{n-k+1}, \ldots, a_n$ will be in their correct positions, so the largest $k + 1$ numbers will be in the correct positions. This completes the proof by induction. □

The last result proves that the bubble sort algorithm is correct, because after n executions of the **while** loop, every number will be in its correct position.

Another use of induction in computer science is to analyse the running time of algorithms. Say that we have an algorithm which takes as input a list of n numbers. We would like to find a function for $T(n)$, the running time of this algorithm. We do not explicitly know this function, but we do know that $T(1) = 1$, $T(2) = 2$, and $T(n) = T(n - 1) + 2T(n - 2)$ when $n \geq 3$. From this information we can calculate the following running times.

n	$T(n)$
1	1
2	2
3	$T(2) + 2T(1) = 2 + 2 \cdot 1 = 4$
4	$T(3) + 2T(2) = 4 + 2 \cdot 2 = 8$
5	$T(4) + 2T(3) = 8 + 2 \cdot 4 = 16$

Now we strongly suspect that $T(n) = 2^{n-1}$ for any n. But how can we prove this fact? The natural strategy is to use induction. We will present the proof by using the minimal counterexample framing. Assume that $T(n) \neq 2^{n-1}$ for some positive integer n. Amongst all such counterexamples, let n be the smallest possible counterexample. Now $n \neq 1$, because $T(1) = 1 = 2^0 = 2^{1-1}$. Furthermore, $n \neq 2$, since $T(2) = 2 = 2^1 = 2^{2-1}$. So $n \geq 3$. Because n is the smallest

counterexample and $n-1$ and $n-2$ are smaller than n, it follows that $T(n-1) = 2^{(n-1)-1}$ and $T(n-2) = 2^{(n-2)-1}$. In summary,

$$T(n) = T(n-1) + 2T(n-2) = 2^{(n-1)-1} + 2 \cdot 2^{(n-2)-1} = 2^{n-2} + 2 \cdot 2^{n-3} = 2^{n-2} + 2^{n-2} = 2 \cdot 2^{n-2} = 2^{n-1}.$$

So n is not a counterexample after all. We choose n to be a counterexample, but now we see that it is not. This contradiction was created by our assumption that $T(n) \neq 2^{n-1}$ for some positive integer n. Therefore this situation can never arise, and hence $T(n) = 2^{n-1}$ for every positive integer n.

Summary

Important ideas from Section 4.

- An inductive proof often involves these two steps:
  (1) Check that the base case holds by testing $P(0)$ is true.
  (2) Assume that $P(0), P(1), \ldots, P(k)$ are all true for an arbitrary non-negative integer k, and using this inductive assumption to prove that $P(k+1)$ is true.

  This applies if we want to prove $P(n)$ is true for all non-negative integers n. If we want to prove it for positive integers, then we start with the base case $P(1)$, and make the inductive assumption $P(1), P(2), \ldots, P(k)$.

5 Sets

The idea of a set is very simple, but very important.

Informal definition 5.1. A *set* is a collection of objects, called the *elements* of the set.

It turns out that this informal definition can lead to serious problems (see Russell's paradox, for example). However it is enough to let us build some intuition for the idea of a set.

We usually use upper-case letters such as A and B to denote sets, and lower-case letters such as a, b, x, and y, to denote elements of sets. Sometimes we can list the elements in a set. In this case we represent the set by surrounding the elements of set with the curly brackets $\{$ and $\}$.

Example

The notation $\{1, 2, 3\}$ stands for the set that has 1, 2, and 3 as its elements.

Sets are *unordered* data structures, meaning that when we list the elements of a set order does not matter. We also ignore repetition, so

$$\{1, 2, 3\}, \quad \{1, 1, 1, 2, 2, 3\}, \quad \{3, 2, 2, 1\}, \quad \{2, 3, 1\}$$

all represent the same set.

Sometimes it is not possible to list all the elements of a set, because there are too many of them. In this case we may be able to use dots ... to indicate that the elements continue in a predictable fashion, as in the next definition.

Definition 5.2. The set of *integers* is denoted $\mathbb{Z}$, and is equal to

$$\{\ldots, -2, -1, 0, 1, 2, \ldots\}.$$

For another example, $\{1, 2, \ldots, 100\}$ is the set of integers between 1 and 100.

Remember that a number is *positive* if it is greater than zero, and it is *negative* if it is less than zero. Zero itself is neither positive nor negative. The set of *positive integers* is $\{1, 2, 3, \ldots\}$ and the set of *non-negative integers* is $\{0, 1, 2, 3, \ldots\}$. We use $\mathbb{Z}^+$ to denote the set of positive integers. You may have heard the term *natural* numbers used in this context, but this term is ambiguous: some people use the term natural numbers to refer to the positive integers, and other people use it to refer to the non-negative integers.

Remark

We frequently use the notation $[n]$ to denote the set $\{1, 2, \ldots, n\}$ when n is a positive integer. In the Python programming language, `range`(n) stands for $\{0, 1, 2, \ldots, n-1\}$.

If A is a set, then the notation $x \in A$ means that x is one of the elements in A. We read $x \in A$ as "x is in A", "x is an element of A", or "x is contained in A". The notation $x \notin A$ means that x is not an element of A.

Example

If $A = \{-1, 0, 1\}$, then $0 \in A$, and $2 \notin A$.

Often it is not possible to list all the elements of a set. Instead, we use *set-builder* notation, which looks like this

$$\{x \mid \text{conditions on } x\}.$$

This stands for the set of all elements x which satisfy the conditions after the vertical bar $|$. The next definition shows an example of set-builder notation.

Definition 5.3. The set of *rational numbers* is denoted $\mathbb{Q}$, and is equal to

$$\{p/q \mid p \in \mathbb{Z},\ q \in \mathbb{Z},\ q \neq 0\}.$$

So $\mathbb{Q}$ is equal to the set of all numbers that can be written as a fraction p/q, where p and q are integers, and q is not equal to zero. (We have to insist that q is not zero, because division by zero is not defined.) Note that every integer p is a rational number, since it can be written as $p/1$.

Sometimes we use the notation $\{x \in S \mid \text{conditions on } x\}$ rather than $\{x \mid \text{conditions on } x\}$ if we want to explicitly say that x must come from the set S. This notation stands for the set that contains all elements in S that satisfy the conditions.

Example

The set $\{x \in \mathbb{Z} \mid x \geq 0\}$ contains all the integers that are greater than or equal to zero, so it is the set of non-negative integers.

Remark

In some sources you may see a colon : used instead of a bar $|$ in set-builder notation, as in $\{x \in \mathbb{Q} : 0 < x < 1\}$.

Definition 5.4. Two sets are *equal* if they have the same elements.

There is a special set which does not contain any elements at all. We call this the *empty set*, and denote it with the symbol $\emptyset$, or sometimes $\{\}$.

We have defined the sets of integers and rational numbers. The set of *real* numbers is denoted by $\mathbb{R}$. The real numbers are all those numbers that have a decimal representation, like this

$$a_n a_{n-1} \cdots a_1 a_0 \,.\, b_1 b_2 b_3 \cdots$$

where the digits a_i and b_i are integers between 0 and 9. All the rational numbers can be written with a decimal representation, so every rational number is a real number. For example, $2 = 2.00000\cdots$, $1/5 = 0.20000\cdots$, $1/3 = 0.33333\cdots$, and $1/7 = 0.142857142857\cdots$. There are also *irrational* real numbers, which cannot be written as fractions. Some examples of irrational real numbers are

$$\sqrt{2} = 1.414213562\cdots \quad \pi = 3.141592653\cdots \quad \text{and} \quad e = 2.718281828\cdots$$

5.1 Subsets

It is obvious that some sets are contained in others. For example, every number in $\mathbb{Z}^+$ is also contained in $\mathbb{Z}$. If A and B are sets, and every element in A is also an element of B, then we say that A is a *subset* of B. We use the notation $A \subseteq B$ or $B \supseteq A$ to express the fact that A is a subset of B. Therefore $\mathbb{Z}^+ \subseteq \mathbb{Z} \subseteq \mathbb{Q} \subseteq \mathbb{R}$. If A is not a subset of B, then we write $A \not\subseteq B$.

Example

If $A = \{1, 3, 5\}$, then the subsets of A are

$$\emptyset, \quad \{1\}, \quad \{3\}, \quad \{5\}, \quad \{1, 3\}, \quad \{1, 5\}, \quad \{3, 5\}, \quad \{1, 3, 5\}.$$

The following statements are all true.

$$\{1\} \subseteq \{1, 3, 4\}, \quad \{3, 4\} \subseteq \{3, 4\}, \quad \{1, 2, 3\} \nsubseteq \{1, 2, 4, 5\}.$$

The statement $A \nsubseteq B$ means that there is some element in A that is not an element of B. But what if A is the empty subset? Then there are no elements of A, so $A \nsubseteq B$ cannot occur. This means that when A is the empty set, then $A \subseteq B$ is always true. So the empty set is a subset of every set. It is also true that every element of B is an element of B, so every set is a subset of itself. In other words, we have the following.

Proposition 5.5. *If B is any set, then $\emptyset \subseteq B$, and $B \subseteq B$.*

Definition 5.6. If A is a subset of B, and A is not equal to B, then we say that A is a *proper* subset of B. The notations $A \subset B$, $A \subsetneq B$, and $A \subsetneqq B$ are all used to indicate that A is a proper subset of B. Note the difference between $A \subsetneq B$ and $A \nsubseteq B$. The first piece of notation means that A is a proper subset of B, and the second means that A is *not* a subset of B.

Example

If $X = \{a, b\}$, then the proper subsets of A are $\emptyset$, $\{a\}$, and $\{b\}$. Also $\{a\} \subsetneq \{a, b\}$ and $\{a\} \nsubseteq \{b\}$ are both true statements.

So far, we have mainly considered sets that contain numbers. But a set can contain just about any type of mathematical object. In particular, a set can contain other sets. So $\{\emptyset, \{1, 2\}, \{1, 3\}\}$ is a set of sets.

Definition 5.7. If A is a set, then the *power set* of A, written $\mathcal{P}(A)$, is

$$\{X \mid X \subseteq A\}.$$

In other words, $\mathcal{P}(A)$ is the set of all subsets of A.

Example

If $A = \{a, b\}$, then $\mathcal{P}(A) = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$.

Remark

Sometimes the notation 2^A or $\mathbf{pow}(A)$ is used instead of $\mathcal{P}(A)$.

Remark

What is the power set of the empty set? The only subset of the empty set is the empty set, so $\mathcal{P}(\emptyset) = \{\emptyset\}$. Note that $\{\emptyset\}$ is not the empty set, even though it seems as though it contains nothing. In fact $\{\emptyset\}$ contains one element: $\emptyset$. If this confuses you, think of the

following analogy. A set is like a box, and the elements in that set are objects contained in the box. Now $\emptyset$ is an empty box, but $\{\emptyset\}$ is not an empty box, it is a box containing an empty box.

5.2 Set operations

Definition 5.8. If A and B are sets, then $A \cup B$, the *union* of A and B, is the set

$$\{x \mid x \in A \text{ or } x \in B\}.$$

The *intersection* of A and B, written $A \cap B$, is

$$\{x \mid x \in A \text{ and } x \in B\}.$$

So $A \cup B$ is the set of all elements that are in either A or B, and $A \cap B$ is the set of all elements that are in both.

Example

If $A = \{1, 3, 5, 7\}$ and $B = \{2, 3, 4, 5, 6\}$, then

$$A \cup B = \{1, 2, 3, 4, 5, 6, 7\} \quad \text{and} \quad A \cap B = \{3, 5\}.$$

Note that even though 3 appears in both A and B, we only list it once in $A \cup B$, as we never repeat elements in sets.

Definition 5.9. If A and B are sets, then the *relative complement* of B in A is denoted $A \backslash B$, and is equal to $\{x \mid x \in A,\ x \notin B\}$. In other words, $A \backslash B$ is the set of elements that are in A but not in B.

Example

Let $A = \{1, 3, 5, 7, 9\}$ and let $B = \{0, 1, 2, 3, 4\}$. Then $A \backslash B = \{5, 7, 9\}$.

Remark

The notation $A - B$ is sometimes used instead of $A \backslash B$.

Definition 5.10. Let A and B be sets. The *symmetric difference* of A and B is denoted by $A \triangle B$ and is equal to $\{x \mid x \in A \backslash B \text{ or } x \in B \backslash A\}$. Therefore $A \triangle B$ is equal to $(A \backslash B) \cup (B \backslash A)$.

Example

Let $A = \{2, 5, 6, 8, 9, 10\}$ and let $B = \{1, 3, 5, 8, 9, 11\}$. Then $A \triangle B = \{1, 2, 3, 6, 10, 11\}$.

Quite frequently we consider sets that are all subsets of some *universal* set. For example, we might consider subsets of the positive integers, and in this case the universal set would be $\mathbb{Z}^+$. If we are working with a universal set, U, and A is a subset of the universal set, then the *complement* of A is $U \backslash A$. We denote the complement of A with $\overline{A}$. (Some people use A^c or $\sim A$ to denote the complement of A.)

Example

Suppose that we are working with sets of integers, so our universal set is $\mathbb{Z}$. If $A = \{\ldots, -4, -2, 0, 2, 4, \ldots\}$ is the set of even integers, then $\overline{A}$ is $\{\ldots, -3, -1, 1, 3, \ldots\}$, the set of odd integers.

5.3 Venn diagrams

When we are trying to visualise set operations it is often helpful to draw a *Venn diagram*. In these diagrams, sets are represented by circles or other shapes. The box around the circles represents the universal set U. The shaded area in Figure 12 represents the intersection of A and B. In Figure 13, the shaded area represents $\overline{A}$, the complement of A.

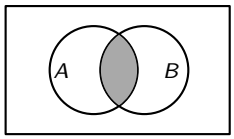


Figure 12: The intersection of A and B.

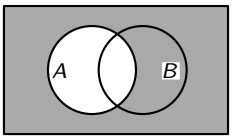


Figure 13: The complement of A.

By using Venn diagrams, we can discover important facts about set operations. For example, $\overline{A \cap B}$, the complement of $A \cap B$, is shown in Figure 14. On the other hand, Figure 15 (i)

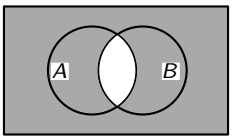
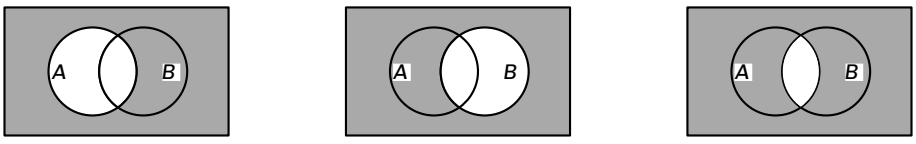


Figure 14: The complement of $A \cap B$.

shows $\overline{A}$, (ii) shows $\overline{B}$, and (iii) shows $\overline{A} \cup \overline{B}$. Obviously $\overline{A} \cup \overline{B}$ is the same as $\overline{A \cap B}$. We have proved



(i) (ii) (iii)

Figure 15:

the following result.

Proposition 5.11. *If A and B are subsets of the universal set U, then $\overline{A} \cup \overline{B} = \overline{A \cap B}$.*

You should confirm the following facts by drawing Venn diagrams.

Proposition 5.12. *Let A, B, and C be subsets of a universal set U.*

Double complements. $A = \overline{\overline{A}}$.

Commutative laws.

(i) $A \cap B = B \cap A$.
(ii) $A \cup B = B \cup A$.

Associative laws.

(i) $A \cap (B \cap C) = (A \cap B) \cap C$.
(ii) $A \cup (B \cup C) = (A \cup B) \cup C$.

Distributive laws.

(i) $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$.
(ii) $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$.

Idempotent laws.

(i) $A \cap A = A$.
(ii) $A \cup A = A$.

De Morgan's laws.

(i) $\overline{A \cap B} = \overline{A} \cup \overline{B}$.
(ii) $\overline{A \cup B} = \overline{A} \cap \overline{B}$.

You will recognise that these laws of set algebra are very similar to the laws of logical equivalences in Proposition 2.6. This is not a coincidence: we can use the logical laws to prove the set laws.

5.4 Cartesian products

An *ordered pair* is a pair of elements (a, b). Note that (a, b) is different from the set $\{a, b\}$. In a set, we ignore repetition and order. So $\{a, a\} = \{a\}$, and $\{a, b\} = \{b, a\}$. In an ordered pair we pay attention to order, and we allow repetition. So (a, a) is a legitimate ordered pair, and (a, b) and (b, a) are different ordered pairs. We use round brackets around ordered pairs instead of curly brackets to draw attention to these differences. We call the elements in an ordered pair the *coordinates* of that pair.

Definition 5.13. Let A and B be sets. Then the *Cartesian product* of A and B is denoted $A \times B$, and is equal to $\{(a, b) \mid a \in A,\ b \in B\}$.

In other words, $A \times B$ consists of all ordered pairs where the first element comes from A and the second element comes from B.

Example

If $A = \{0, 1\}$ and $B = \{x, y, z\}$, then

$$A \times B = \{(0, x), (0, y), (0, z), (1, x), (1, y), (1, z)\}$$

Example

If $A = \{x, y\}$, then $\mathcal{P}(A) = \{\emptyset, \{x\}, \{y\}, \{x, y\}\}$, so $A \times \mathcal{P}(A)$ is

$$\{(x, \emptyset), (x, \{x\}), (x, \{y\}), (x, \{x, y\}), (y, \emptyset), (y, \{x\}), (y, \{y\}), (y, \{x, y\})\}.$$

More generally, we say that a *tuple* is an ordered sequence of objects. An *n-tuple* is an ordered sequence of n objects. We use rounded brackets to write tuples, so $(a_1, a_2, \ldots, a_n)$ is an n-tuple. Note that this is different from the unordered set $\{a_1, a_2, \ldots, a_n\}$. In tuples, order matters, and repetition is allowed. If $A_1, \ldots, A_n$ are sets, then $A_1 \times \cdots \times A_n$ is the set of all n-tuples, where the object in position i comes from the set A_i.

Example

Let $A = \{1, 3, 5\}$, let $B = \{\circ, \triangle\}$, and let $C = \{0, 1\}$. Then

$$A \times B \times C = \{(1, \circ, 0), (1, \circ, 1), (1, \triangle, 0), (1, \triangle, 1), (3, \circ, 0), (3, \circ, 1),$$
$$(3, \triangle, 0), (3, \triangle, 1), (5, \circ, 0), (5, \circ, 1), (5, \triangle, 0), (5, \triangle, 1)\}$$

5.5 Data structures

A *data structure* is a method or a format for internally storing data in the architecture of a computer. The way in which we store data can make a significant difference to the speed of an algorithm, so the study of data structures is of great importance. We will pay more attention to data structures later in the module, but for now we will have a brief discussion of sets and tuples, considered as data structures.

There are certain operations we always want to perform on a data structure. For instance, we want to be able to check whether a certain data value is stored in the data structure. We will likely want to be able to alter the data structure by adding or removing values.

A set is an unordered data structure, meaning that there is no first or last element in the data structure. We would normally expect to have operations such as

`is_element_of(x,S)` and `is_empty(S)`

in an implementation of the set data structure. These two commands will return a value of either *true* or *false* (we call such a value a *Boolean value*). The first command returns true if the element x is in the set S. The second command will return true if S is the empty set.

Other operations we might expect to see are

`add(x,S)` and `remove(x,S)`.

These operations will return another set. The first command returns S with the element x added. If x is already in S, then the output set will be exactly the same as the input. In other words

the output is $S \cup \{x\}$. The second operation will return $S \backslash \{x\}$. Notice that if x is not in S then the output will be exactly the same as the input. We might also see commands such as

`union(S,T)` and `intersection(S,T)`

which will perform the corresponding set operations on their input.

Let us now turn to ordered data structures. These basically correspond to tuples, since order is taken into account when we work with tuples. Ordered data structures are often called *lists*. If a list contains n elements, then these elements will typically be indexed with the integers $0, 1, \ldots, n-1$. We expect to be able to access the item in the list in any specified position. For example, if `L` is a list, then a command such as `L[i]` will return the item in `L` that is indexed by i. A command like `L[i]=x` will replace the item index by i with x. The command `len(L)`, or some equivalent, will return the number of elements in the list. We also encounter the following operations:

`prepend(L,x)` and `append(L,x)`

which alter the list L by adding the element x as either the first element or the last element. In the `prepend` command, the element x is added as the first element, and the other elements are shifted by adding one to their index. In either case the length of the list will increase by one.

Summary

Important ideas from Section 5.

- We list the items in a set inside curly brackets $\{\ \}$.
- Repetition and order are ignored when we list the elements of a set.
- The notation $a \in A$ means that a is an element in the set A.
- The empty set $\emptyset$ is the unique set that has no elements.
- $\mathbb{Z}$ is the set of integers, $\mathbb{Q}$ is the set of rational numbers, and $\mathbb{R}$ is the set of real numbers.
- The notation $A \subseteq B$ means that A is a subset of B.
- Every set is a subset of itself, the empty set is a subset of every set.
- $\mathcal{P}(A)$ is the power set of A, the set of all subsets of A.
- $A \cup B$ is the union of A and B, the set of all elements that are in A or B.
- $A \cap B$ is the intersection of A and B, the set of all elements that are in A and B.
- $A \backslash B$ is the relative complement of B in A, the set of all elements that are in A but not in B.
- $A \triangle B = (A \backslash B) \cup (B \backslash A)$.
- $\overline{A}$ is the complement of A, the set of all elements that are not in A.
- $A \times B$ is the Cartesian product of A and B, the set of all ordered pairs with the first element from A and the second element from B.

6 Functions

You have very likely encountered the idea of a function before. Probably you have drawn the graph of a function such as $f(x) = x^2$ or $f(x) = \sin x$. It is helpful to think of functions as machines which take numbers as input and then produce numbers as output. So the $f(x) = x^2$ function is a machine which takes any real number as input, and then produces the square of that number as output. Figure 16 shows the *graphs* of the functions $f(x) = x^2$ or $f(x) = \sin x$. In a graph such as this, we fill in every point with coordinates of the form $(x, f(x))$: that is, we colour any point where the horizontal coordinate is an input and the vertical coordinate is the corresponding output.

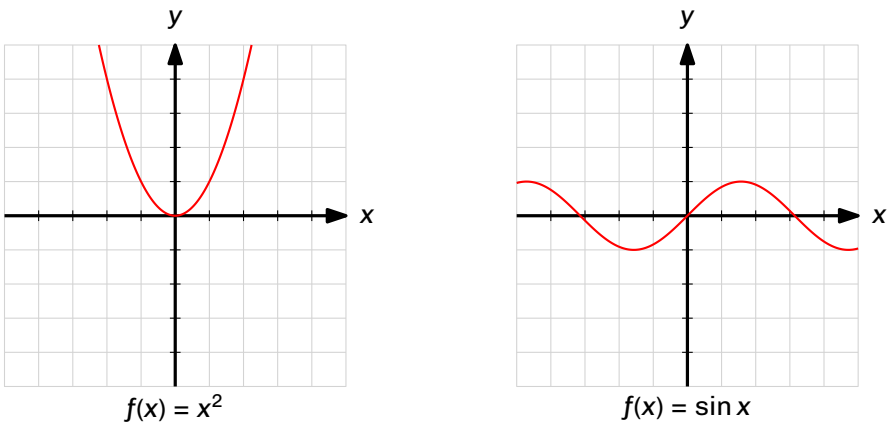


Figure 16: The graphs of two functions.

The modern definition of a function is more abstract: there does not need to be a nice rule which determines how the function operates. Furthermore, the input and the output do not need to be numbers. They could be just about any mathematical object.

A function is always associated with a set of inputs, which we call the *domain* of the function. There is also a set that contains all outputs. We call this set the *codomain*. In the familiar examples $f(x) = x^2$ and $f(x) = \sin x$ the domain is $\mathbb{R}$, the set of real numbers, because we can use any real number as an input. The codomain is also $\mathbb{R}$ because every output will be a real number. We express this by saying that in either of these cases, f is a *function from the real number to the real numbers*. We may abbreviate this by saying that f is a function *on the real numbers*. Most of the functions you will have encountered up to this point will be functions on the real numbers.

Notice that some elements of the codomain may never appear as an output. For example, if $f(x)$ is the function taking x to x^2, then no negative real number will ever appear as an output, because the square of any real number is non-negative. So the set of outputs is a subset of the codomain, possibly a proper subset. The set of all outputs is known as the *image* or the *range* of the function. The image of $f(x) = x^2$ is $\{r \in \mathbb{R} \mid r \geq 0\}$ and the image of $f(x) = \sin x$ is $\{r \in \mathbb{R} \mid -1 \leq r \leq 1\}$.

Remark

If the image is the set of all possible outputs, why do we need the idea of the codomain? Why don't we just always take the codomain to be the set of outputs, instead of some seemingly arbitrary set which contains all possible outputs? The answer is that determ-

ining the image may not always be easy, or possible. For example, if $f(x)$ is defined to be something like

$$\sin x - \tfrac{1}{2!}\cos 2x + \tfrac{1}{3!}\sin 3x - \tfrac{1}{4!}\cos 4x + \cdots$$

then I may not immediately be able to see what the set of outputs is. But I do know for sure that every output is a real number. So I am comfortable saying that the codomain is the set of real numbers, even if I don't know what the image is.

You may have heard of the *vertical line test*, which says that if we draw the graph of any function from $\mathbb{R}$ to $\mathbb{R}$, then any vertical line will intersect this graph in at most one point. So the set of points that satisfy the equation $x^2 + y^2 = 1$ is not the graph of a function, because that equation describes a circle and a vertical line may intersect the circle in two points. This test reflects the fact that for every input there should be *exactly one* output. We want to think of a function as a machine which receives input and produces output, and we insist that each input produces exactly one output.

Now that we have covered some of the basic intuition for functions, we can move towards the more abstract definition.

Definition 6.1. Let A and B be sets. A *function from A to B* is a set of ordered pairs with the first element in A and the second element in B. In other words, a function is a subset of the Cartesian product $A \times B$. In order to be a function the following condition must be satisfied: for every element $a \in A$ there is exactly one ordered pair in the function with a as a first element. We say that A is the *domain* of the function and B is the *codomain* of the function.

We write $f\colon A \to B$ to express the fact that f is a function from A to B. If $a \in A$, and b is the unique element in B such that (a, b) is in the function, then we write $f(a) = b$ and we say that b is the *image* of a. We also say that a is *mapped* to b. We sometimes use the notation $f\colon a \mapsto b$ to mean that f maps a to b. Note the difference between the notations $f\colon A \to B$ and $f\colon a \mapsto b$. The former is used to show that f is a function with domain A and codomain B. The latter is used to show that the function f maps the element a to the element b.

The *image* of the function f is $\{b \in B \mid \text{there exists } a \in A \text{ such that } f(a) = b\}$.

Example

If $f(x)$ is the function which takes each real number x to x^2, then we think of f as the collection of ordered pairs $\{(x, x^2) \mid x \in \mathbb{R}\}$. This is a subset of the Cartesian product $\mathbb{R} \times \mathbb{R}$, which contains every possible ordered pair of real numbers. Note that for every real number x there is exactly one real number equal to the square of x. So for each $x \in \mathbb{R}$ there is exactly one ordered pair with x as a first element. Thus f really is a function.

Remark

As we have said, most functions you have seen are functions from $\mathbb{R}$ to $\mathbb{R}$ given by nice rules such as $f(x) = e^x \sin x$. But the definition we have just given allows for the domain and codomain to be completely arbitrary sets. They don't need to be sets of numbers at all. Furthermore, there does not need to be a consistent rule which determines the output produced from any given input.

Remark

Sometimes people will say something like "the square root of four is either two or negative two". The more standard convention is to say that the square root of four is two. When x is a non-negative real number, the convention says that $\sqrt{x}$ is the non-negative real number that satisfies $(\sqrt{x})^2 = x$. Under this convention, $f(x) = \sqrt{x}$ is a function with domain and codomain equal to the set of non-negative real numbers.

We draw functions $f\colon \mathbb{R} \to \mathbb{R}$ by drawing graphs in the Cartesian coordinate grid. If we have a more abstract function $f\colon A \to B$, we may still be able to illustrate it with a diagram that represents the domain A by a collection of the dots on the left-hand side of the diagram and the codomain B by a collection of dots on the right-hand side of the diagram. If a is in the domain and b is its image then we draw an arrow from a to b. We call a diagram of this type an *arrow diagram*. In order for this diagram to represent a function, we must have exactly one arrow coming out of each dot in the domain, which reflects the fact that each input in the domain must have exactly one output in the codomain.

Example

Look at the diagrams in Figure 17. The first diagram represents a function from the domain $\{1, 2, 3, 4\}$ to the codomain $\{1, 2, 3, 4\}$. Notice that each dot in the domain has exactly one arrow coming out of it. The function shown in (i) is the set of ordered pairs $\{(1, 3), (2, 4), (3, 1), (4, 4)\}$. Each ordered pair corresponds to an arrow in the diagram.
On the other hand, diagram (ii) does not represent a function because 2 has two arrows coming out of it. Diagram (iii) does not represent a function because 3 has zero arrows coming out of it.

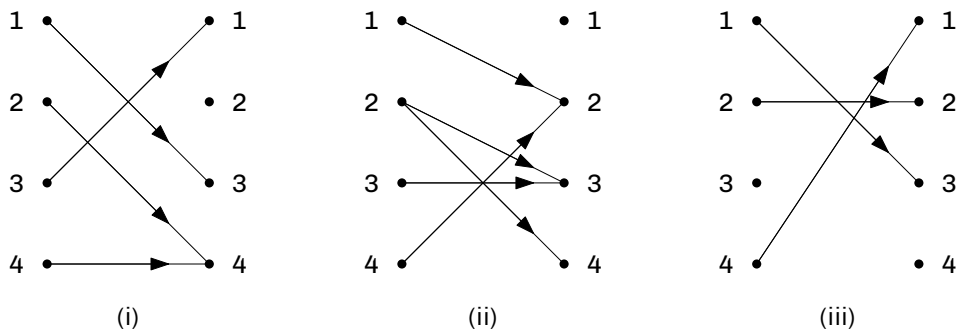


Figure 17: Three arrow diagrams

6.1 Composition of functions

Often we need to *compose* functions. This means we apply one function to an input, and then use the output from that function as the input to another function. You will have already seen this happen in examples such as $f(x) = \sin(x^2)$. In this case we can think of f as being made up of two functions. The first function squares the number x, and the second function uses this as input to the sine function. We can think of this composition as wiring up two machines, so that the output from the first machine becomes the input for the second machine.

Definition 6.2. Given functions $f : A \to B$ and $g : B \to C$, the *composition* of f and g is denoted by $g \circ f$. This is a function from A to C. The composition is defined in the following way: if $a \in A$ is the input then the output is $g(f(a))$. That is, we use a as input for f, and then we take the output and use it as input for g. We are able to do this because the codomain of f is the domain of g. So every output from f is an input for g. Notice that we read $g \circ f$ as *g follows f* — we first apply f and *then* apply g. Be careful — we read $g \circ f$ from right to left, meaning that we apply f first.

Remark

Notice that the composition $g \circ f$ only makes sense if the codomain of f is a subset of the domain of g. We need every output of f to be a possible input for g.

Example

Let f be the function from $\{w, x, y, z\}$ to $\{r, s, t\}$ and let g be the function from $\{r, s, t\}$ to $\{a, b, c\}$ as defined by the diagram in Figure 18. Because the codomain of f is the same as the domain of g we can define the composition $g \circ f$.
To find the image of, say, x under the function $g \circ f$, we can follow the arrow from x to r in the f diagram. We then follow the arrow from r to c in the g diagram, and we deduce that $g \circ f$ takes x to c. By making these calculations we see that

$$g \circ f \text{ takes } w \text{ to } (g(f(w)) = g(s) = c$$
$$g \circ f \text{ takes } x \text{ to } (g(f(x)) = g(r) = c$$
$$g \circ f \text{ takes } y \text{ to } (g(f(y)) = g(t) = b$$
$$g \circ f \text{ takes } z \text{ to } (g(f(z)) = g(r) = c$$

The function $g \circ f$ is shown in Figure 19.

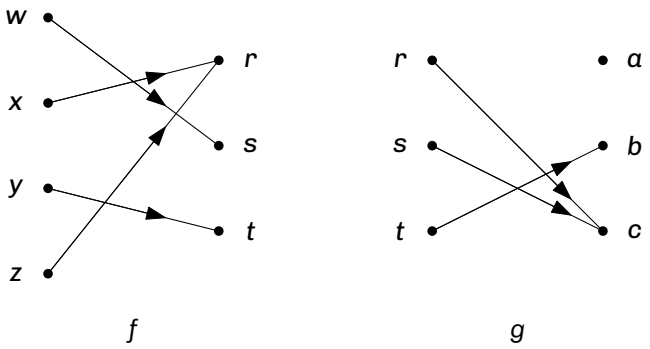


Figure 18: Functions f and g such that $g \circ f$ is defined.

Example

Let $f : \mathbb{R} \to \mathbb{R}$ and $g : \mathbb{R} \to \mathbb{R}$ be the functions defined by $f(x) = 2x$ and $g(x) = x + 1$. Then $g \circ f$ takes $x \in \mathbb{R}$ to $g(f(x)) = g(2x) = 2x + 1$. On the other hand, $f \circ g$ takes $x \in \mathbb{R}$ to $f(g(x)) = f(x + 1) = 2(x + 1) = 2x + 2$. So $g \circ f$ takes 1 to 3 whereas $f \circ g$ takes 1 to 4. Now we

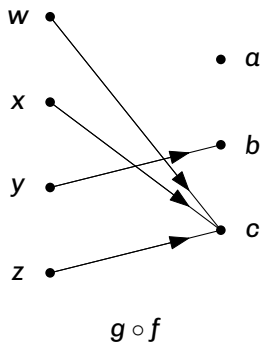


Figure 19: The composition $g \circ f$.

see that $g \circ f$ and $f \circ g$ are different functions. This shows that the order matters when we compose functions. We can express this by saying that composition of functions is not *commutative*.

Although composition is not commutative, we will prove in the next result that it is *associative*. This means that the way we group function composition does not matter. For example, addition is associative because $a + (b + c) = (a + b) + c$ for all a, b, and c. Before we prove composition is associative, we need to say what it means for two functions to be equal. Functions f and g are equal if they have the same domains, same codomains, and map each element of the domain to the same image. That is, for every x in the shared domain of f and g, we must have $f(x) = g(x)$.

Proposition 6.3. *If $f : A \to B$, $g : B \to C$, and $h : C \to D$ are functions, then $h \circ (g \circ f) = (h \circ g) \circ f$.*

Proof. The function $g \circ f$ has domain A and codomain C, so the function $h \circ (g \circ f)$ has domain A and codomain D. The function $h \circ g$ has domain B and codomain D, so the function $(h \circ g) \circ f$ also has domain A and codomain D.

Now take any $a \in A$. Let b be $f(a)$, let c be $g(b)$, and let d be $h(c)$. Then $g \circ f$ takes a to c and $h \circ (g \circ f)$ takes a to d. On the other hand, $h \circ g$ takes b to d, so $(h \circ g) \circ f$ takes a to d. This shows that $h \circ (g \circ f)$ maps any element of A to the same element of D as $(h \circ g) \circ f$. Therefore the two functions are the same. □

6.2 Properties of functions

We now explore fundamental properties that some functions possess.

Injective functions.

Definition 6.4. A function $f : A \to B$ is *one-to-one* or *injective* if, for all $a_1, a_2 \in A$, if $f(a_1) = f(a_2)$, then $a_1 = a_2$.

That is, $f(a_1) = f(a_2)$ implies $a_1 = a_2$. This is often stated in the contrapositive: $a_1 \neq a_2$ implies $f(a_1) \neq f(a_2)$. In other words, "different elements are mapped to different elements". If we are able to draw the arrow diagram of a function, we can check it is one-to-one by checking that no two arrows arrive at the same location.

Example

Consider the left diagram in Figure 20. Does it describe a one-to-one function? It certainly defines a function, as every element in the domain is mapped to exactly one element in the codomain. However, it is *not* one-to-one, as 1 and 3 are both mapped to 3.
On the other hand, we can see that in the right diagram in Figure 20, no two elements in the domain are mapped to the same element of the codomain. Therefore this function is one-to-one.

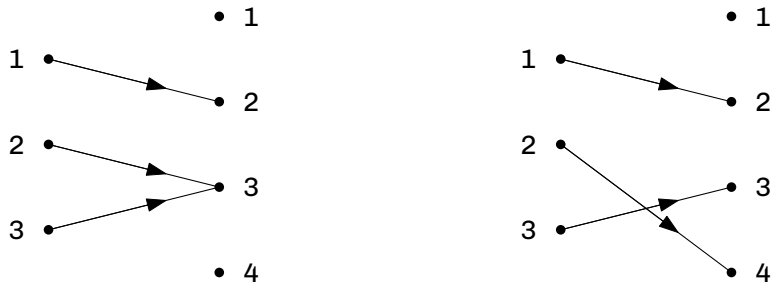


Figure 20: Two functions, one non-injective and the other injective.

To show that a function is *not* one-to-one, all we need is a single counterexample. Consider the function $f(x) = x^2$. Then $f(-1) = f(1) = 1$, so -1 and 1 are mapped to the same element of the codomain. Therefore f is not one-to-one.

To show that a function is one-to-one we assume that $f(a_1) = f(a_2)$ for some elements a_1 and a_2 in the domain. We then prove that $a_1 = a_2$. If we are able to complete this step then we have proved that the function is one-to-one.

Example

Let $f : \mathbb{R} \to \mathbb{R}$ be defined by $f(x) = 2x - 1$. Assume that $f(x_1) = f(x_2)$. Then $2x_1 - 1 = 2x_2 - 1$, which implies that $2x_1 = 2x_2$ and therefore $x_1 = x_2$. This proves that f is a one-to-one function.

Remark

To check whether a function $f\colon \mathbb{R} \to \mathbb{R}$ is one-to-one we can try examining its graph. The function is one-to-one if and only if it passes the *horizontal line test*. This means that any horizontal line can intersect the graph in at most one point. If it fails this test then it is not one-to-one. To see this imagine that some horizontal line intersects the graph at two different points, a_1 and a_2. Then the function takes a_1 and a_2 to the same value, so it is not one-to-one. Can you use this test to decide which of the following functions is one-to-one?

$$f\colon x \mapsto x^3 \qquad g\colon x \mapsto e^x \qquad h\colon x \mapsto \sin x$$

Surjective functions.

Definition 6.5. A function is *onto* or *surjective* if the codomain is equal to the image of the function. That is, if f is a function from A to B, then f is onto if for every element $b \in B$ there exists

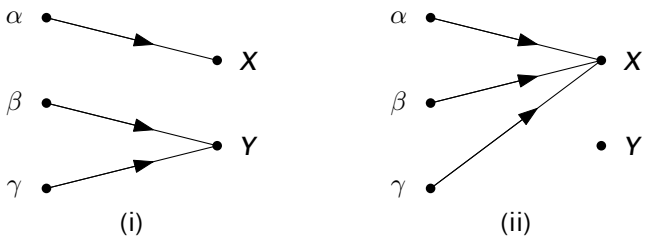


Figure 21:

some $a \in A$ such that $f(a) = b$. This means that every element in the codomain B appears in the image of f.

If we can draw an arrow diagram of a function then we can test whether it is onto by checking that every element in the codomain has an arrow pointing to it.

Example

Consider the diagrams in Figure 21. It shows two functions from the domain $\{\alpha, \beta, \gamma\}$ to the codomain $\{X, Y\}$. The function in (i) is onto, because every element of the codomain has at least one arrow pointing to it. Notice that the image and the codomain are both equal to $\{X, Y\}$. The function in (ii) is not onto because Y has no arrow pointing to it. The image is $\{X\}$.

Example

The function $f : \mathbb{R} \to \mathbb{R}$ defined by $f(x) = 2x + 1$ is onto. For any $y \in \mathbb{R}$ we can find some $x \in \mathbb{R}$ such that $f(x) = y$. To see that this is true, we solve the equation $y = 2x + 1$ to make x the subject of the equation.

$$\begin{aligned} y &= 2x + 1 \\ \Rightarrow y - 1 &= 2x \\ \Rightarrow \tfrac{y-1}{2} &= x \end{aligned}$$

Now we see that an arbitrary $y \in \mathbb{R}$ is the image of the real number $\frac{y-1}{2}$ because

$$f\left(\tfrac{y-1}{2}\right) = 2\left(\tfrac{y-1}{2}\right) + 1 = (y - 1) + 1 = y.$$

So every real number is in the image of f and therefore f is onto.

Example

The function $f : \mathbb{R} \to \mathbb{R}$ defined by $f(x) = x^2$ is *not* onto because the image of f is the set of non-negative real numbers, a proper subset of $\mathbb{R}$.

Remark

If $f\colon A \to B$ is not onto then its image is a proper subset of B, call it X. If we redefine f so that it becomes a function from A to X then it becomes onto. In this way we can make any function onto by restricting its codomain. In the previous example, we could redefine f so that its codomain is the set of non-negative reals, and then it would be onto. Bear in mind though that explicitly determining the image of a function can be difficult.

If $f\colon \mathbb{R} \to \mathbb{R}$ is onto, then it means that every horizontal line will intersect the graph of f. So $f(x) = \ln x$ is an onto function, but $f(x) = x^2$ is not.

It is easy to become muddled with the definitions of functions, injective functions, and surjective functions. The next table summarises the definitions in various different ways.

Property	**Definition**	**Characterisation via arrows**	**Characterisation via ordered pairs**
$f\colon A \to B$ is a function.	Each input is mapped to exactly one output.	Each element of A has exactly one arrow coming out out of it.	For every $a \in A$ there is exactly one ordered pair in f with a as the first element.
$f\colon A \to B$ is injective.	$f(a_1) = f(a_2)$ implies $a_1 = a_2$.	Different arrows go to different locations.	If (a_1, b) and (a_2, b) both belong to f then $a_1 = a_2$.
$f\colon A \to B$ is surjective.	The image of f is equal to B.	Every element of B has an arrow pointing to it.	For every $b \in B$ there exists an $a \in A$ such that (a, b) belongs to f.

Bijective functions.

Definition 6.6. A function $f : A \to B$ is a *bijective function*, or a *bijection*, if it is both injective and surjective. A function is injective if for every b in the codomain there is *at most* one element a in the domain that is mapped to b. A function is surjective if for every b in the codomain there is *at least* one element a in the domain that is mapped to b. A bijective function combines these two properties, so for each $b \in B$ there is *exactly one* $a \in A$ such that $f(a) = b$.

Remark

Contrast the definition of a bijective function with the definition of a function, which says that for each element a in the domain there is exactly one element b in the codomain such that $f(a) = b$.

Example

Figure 22 shows two functions on the set $\{A, B, C, D\}$. The function in (i) is a bijection. We can see this because every dot in the codomain has exactly one arrow pointing to it. On the other hand, the function shown in (ii) is not injective because A has two arrows pointing to it. (So does C.) It also fails to be surjective because B has no arrows pointing to it. (The same statement applies to D.) So the function in (ii) is not a bijection.

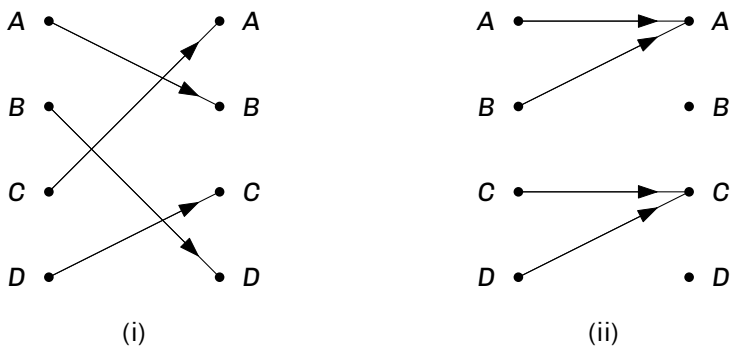


Figure 22:

Identities and inverses. We will now work towards proving an important theorem about bijective functions. First we need to define the identity function.

Definition 6.7. Let A be any set. The *identity function* $\mathrm{id}_A : A \to A$ is the function such that id_A maps every element of A to itself. This means that $\mathrm{id}_A(a) = a$ for every element $a \in A$.

Exercise

Consider the identity function $\mathrm{id}_{\mathbb{R}}$. What does its graph look like?

Remark

Let $f : A \to B$ be a function. Convince yourself that

$$f \circ \mathrm{id}_A = f = \mathrm{id}_B \circ f.$$

The idea of an inverse function is that it undoes the work done by the function.

Definition 6.8. Let $f : A \to B$ be a function. An *inverse* to f is a function $g : B \to A$ such that $g \circ f = \mathrm{id}_A$ and $f \circ g = \mathrm{id}_B$.

Notice that we have symmetry in this definition — if g is an inverse of f, then f is an inverse of g. We frequently write f^{-1} to denote the inverse of the function f.

Assume that g is the inverse of the function $f : A \to B$. Assume also that f takes $a \in A$ to the element $b \in B$ so $f(a) = b$. Now $g \circ f$ takes a to itself. So $g(f(a)) = a$, which means that $g(b) = a$. In other words, f takes a to b and g takes b to a. This means that we can obtain the arrow diagram of g by reversing all the arrows in the arrow diagram of f.

Example

Consider the arrow diagram in Figure 23 (i). This shows a function from $\{A, B, C, D\}$ to itself. In (ii) we have reversed all the arrows in this diagram to obtain a representation of the inverse function. But the convention is that arrows should go from left to right. So in (iii) we have redrawn the arrow diagram of the inverse function so that the arrows follow this convention.

We can think of reversing the arrows in the following way. Remember that we have defined a function to be a collection of ordered pairs. An arrow from a to b corresponds to the ordered

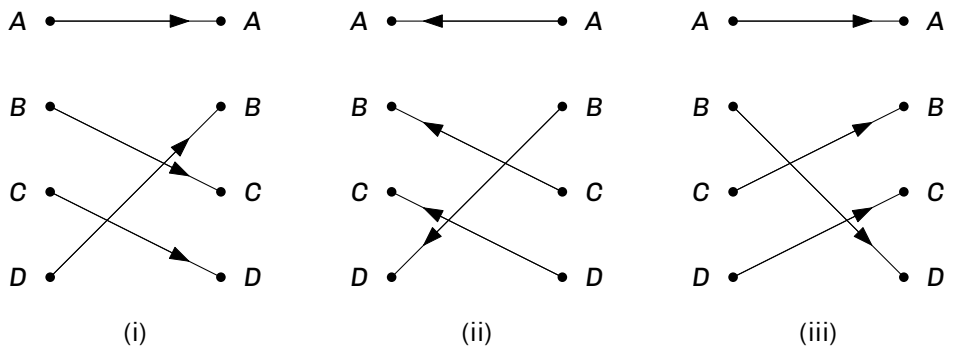
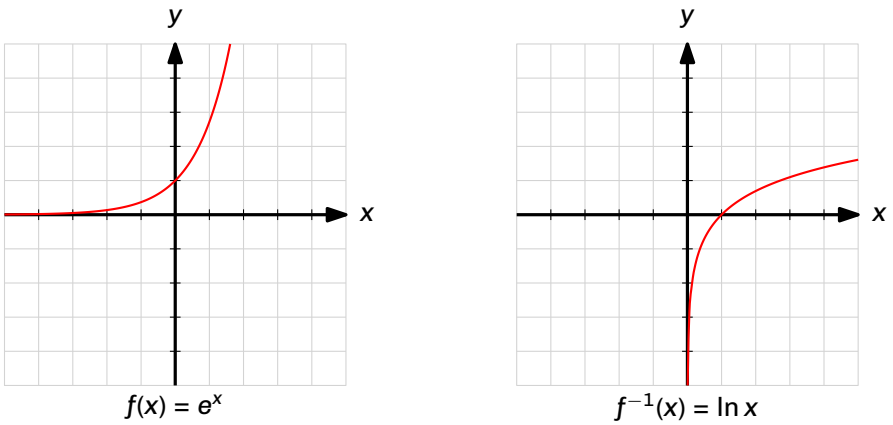


Figure 23:

pair (a, b). To reverse this arrow we simply replace the ordered pair (a, b) with (b, a). This means that if f^{-1} is an inverse of f then $f^{-1} = \{(b, a) \mid (a, b) \in f\}$.

Remark

You already know many inverse functions. A familiar example is provided by the function $f(x) = e^x$ and its inverse $f^{-1}(x) = \ln x$. Sometime the exponential function is written as $\exp(x)$ and its inverse may be written as $\log_e x$. The graphs of these functions are shown in Figure 24. You will notice the similarity of these graphs. In fact, if $f\colon \mathbb{R} \to \mathbb{R}$ is a function, then we can obtain the graph of f^{-1} by reflecting through the diagonal line consisting of the points $\{(a, a) \mid a \in \mathbb{R}\}$. Convince yourself that this means that a function has to pass the horizontal line test in order to have an inverse.
The function $f(x) = x^2$ does not pass the horizontal line test. But isn't the square root function its inverse? Actually no, the square root function is the inverse of the function we obtain from $f(x) = x^2$ by restricting its domain to the non-negative numbers. We do a similar thing with functions like $\sin$ and $\cos$. These function fail the horizontal line test, but we restrict their domains to obtain functions which do have inverses.



$f(x) = e^x$ $f^{-1}(x) = \ln x$

Figure 24: The exponential function and its inverse.

Typically, if we reverse the arrows in an arrow diagram, we will not even obtain a function. This is because there may be two arrows pointing to the same element in the codomain. When we reverse the arrows, there will now be two arrows leaving the same element, which is not

allowed in a function. So we arrive at the question, when does a function have an inverse? This is answered by the next theorem.

Theorem 6.9. *Let $f\colon A \to B$ be a function. Then f has an inverse if and only if it is a bijection.*

Proof. We start by assuming that f is a bijection. We will prove that f has an inverse function. Let g be the collection $\{(b,a) \mid \text{the function } f \text{ contains the ordered pair } (a,b)\}$. We will prove that g is a function and that it is the inverse of f. Note that if (b,a) is an ordered pair in g then b is in B and a is in A. To prove that g is a function we must show that for every element $b \in B$ there is exactly one ordered pair in g with the first element equal to b. Let b be an arbitrary element of b. Because f is a bijection it is surjective, so there is some element $a \in A$ such that $f(a) = b$. Therefore (a,b) is in f, meaning that (b,a) is in g. Assume that there is some other pair (b,a') in g. Then (a',b) is in f, so $f(a') = b = f(a)$. But f is injective, so $f(a') = f(a)$ implies $a' = a$. Therefore (b,a) is the only ordered pair in g with b as the first element. This means that there is exactly one ordered pair of g with b as a first element. Therefore g is a function, as we claimed.

Now we must prove that f and g are inverses. Assume that a is an arbitrary element of A. If f maps a to b then g maps b to a. This means that $g \circ f$ takes a to $g(f(a)) = g(b) = a$. This is true for every element $a \in A$, so $g \circ f$ is the identity function id_A. It is also easy to see that $f \circ g$ is the identity function id_B, by using a very similar argument. This shows that f and g are inverse functions.

For the other direction, we assume that f has an inverse function $g\colon B \to A$. Let b be an arbitrary element of b. Now $f \circ g$ is the identity function id_B. Therefore $f(g(b)) = b$. Let us denote $g(b)$ by a, so that a is an element of A. Now $f(a) = b$, so there is an arrow in f which points to b. Since b was an arbitrary element of B there is an ordered pair of f which contains any given element of B as a second element. This means that f is surjective.

We can finish the proof by showing that f is injective. Assume that $f(a_1) = f(a_2)$. Let b stand for the element $f(a_1) = f(a_2)$ of B. Now $g \circ f$ is the identity function id_A. This means $g(b) = g(f(a_1)) = a_1$. We also have $g(b) = g(f(a_2)) = a_2$. But $g(b)$ is a single element of A, since g is a function. Therefore $g(b) = a_1 = a_2$. This shows that f is an injective function and the proof is complete. □

Summary

Important ideas from Section 6.

- A function $f\colon A \to B$ with domain A and codomain B is a subset of the Cartesian product $A \times B$. For every element $a \in A$, there must be exactly one element $b \in B$ such that (a,b) is in f. We write $f(a) = b$ to mean that (a,b) is in f.
- The image of f is the set of outputs of f, $\{f(a) \mid a \in A\}$.
- If $f\colon A \to B$ and $g\colon B \to C$ are functions, then $g \circ f$ is the composition of f and g. It takes $a \in A$ to $g(f(a))$.
- A function $f\colon A \to B$ is one-to-one (injective) if for all $x, y \in A$, $f(x) = f(y)$ implies $x = y$.
- a function $f\colon A \to B$ is onto (surjective) if for every $b \in B$, there is an element $a \in A$ such that $f(a) = b$.
- A function is bijective if it is injective and surjective.
- Let A be a set. The identity function $\mathrm{id}_A\colon A \to A$ is the function such that $\mathrm{id}_A(a) = a$ for every $a \in A$.

- Let $f\colon A \to B$ be a function. An inverse of f is a function $g\colon B \to A$ such that $g \circ f = \mathrm{id}_A$ and $f \circ g = \mathrm{id}_B$.
- The function $f\colon A \to B$ has an inverse if and only if it is a bijection.

7 Relations

A relation is like a more general version of a function. Recall that a function has exactly one output for each input. In a relation we relax this condition, so that there may be zero outputs for a given input, or there could arbitrarily many outputs. For this reason, a relation is sometimes called a *one-to-many* function. A function from A to B can be thought of as a set of ordered pairs such that for each $a \in A$ there is exactly one ordered pair that contains a as the first element. Now we just dispose of this condition to obtain the definition of a relation.

Definition 7.1. Let A and B be sets. A *relation* from A to B is a set of ordered pairs of the form (a, b), where $a \in A$ and $b \in B$.

Remember that the Cartesian product $A \times B = \{(a, b) \mid a \in A,\ b \in A\}$ is the set of *all* ordered pairs with the first element from A and the second from B. Thus a relation from A to B is a subset of the Cartesian product $A \times B$.

Example

If $A = \{0, 1\}$ and $B = \{a, b, c\}$, then $\{(0, a), (0, b), (1, a), (1, c)\}$ is a relation from A to B. So is $\{(1, a), (1, b), (1, c)\}$, and so is the empty set, $\emptyset$. (We would call this the empty relation.)

Sometimes we can illustrate relations with arrow diagrams, just as we did with functions. The elements of A and B are represented by dots. An ordered pair (a, b) is represented by an arrow going from the dot representing a to the dot representing b. So in the previous example, the relation $\{(0, a), (0, b), (1, a), (1, c)\}$ would be represented by the diagram in Figure 25 (i). The diagram (ii) represents $\{(1, a), (1, b), (1, c)\}$, and (iii) represents the empty relation $\emptyset$.

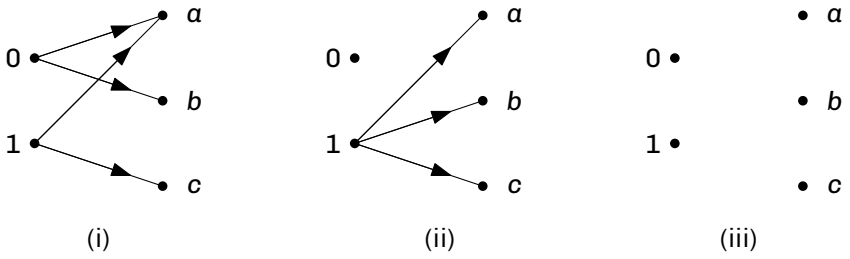


Figure 25: Arrow diagrams of relations

Let A and B be sets, and let R be a relation from A to B. That is, R is a subset of $A \times B$. If (a, b) is an ordered pair in $A \times B$, then either (a, b) belongs to R or it does not. If it does, then we sometimes write aRb to express this fact. If it does not, then we would write $a \not R b$.

Example

If $A = \{a, b, c, d\}$ and $B = \{1, 2, 3\}$, and R is the relation in Figure 26, then $(a, 2)$ is in R, so we write $aR2$. On the other hand, $(d, 1)$ is not in R, so we write $d \not R 1$.

In these examples, A and B have been different sets. In many important cases, A and B are the same set. If R is a relation from A to A, we say that it is a relation *on* A. So a relation on A is a subset of the Cartesian product $A \times A$.

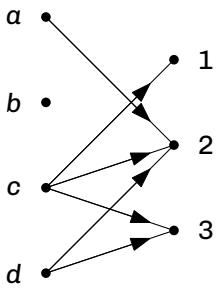


Figure 26:

Example

If $A = \{x, y, z\}$, then $\{(x, x), (x, y), (y, z), (z, x), (z, y), (z, z)\}$ is a relation on A.
When we draw a diagram illustrating a relation on a set, we sometimes use only one set of dots to represent the elements of that set. An arrow from a to b still represents the ordered pair (a, b). An ordered pair of the form (a, a), where the first and second elements are the same, is represented by a loop at the dot representing a. Therefore the relation above is represented by the diagram in Figure 27. Such a diagram is called a *directed graph*.

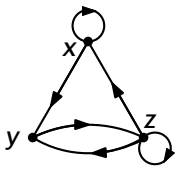


Figure 27:

Example

Relations are very common in the data structures that we use in computer science. In particular, a *dictionary*, *hash*, *map*, or *hashmap*, are terms that are all used to refer to the same type of data structure. Such a structure is a relation that maps *keys* to *values*. In particular, each key is mapped to at most one value. So if K is the set of all possible keys, and V is the set of all possible values, then the dictionary is a relation from K to V with the property that each key in K appears in at most one ordered pair. This mapping is almost a function, because a key cannot be mapped to more than one value, but it may be that some keys are not mapped to any value. So technically the dictionary may not be a function, but it is certainly a subset of $K \times V$ and therefore is a relation. We might sometimes say that the dictionary is a *partial function* from K to V.
As an example, K could be the set of all possible student ID numbers and V could be the set of all possible names. (In reality, the set of all possible names for humans is probably not well defined — is there an upper bound on the length of a name, or the number of alphabets that people use to name themselves?) A university will maintain a dictionary $D \subseteq K \times V$ which maps each key to the name of at most one student. (Some student ID numbers may not have been used and will therefore not be mapped to a name.)

7.1 Properties of relations

In this section we are going to consider some properties of relations. There are four important properties we are going to talk about: *reflexivity*, *symmetry*, *anti-symmetry*, and *transitivity*.

Reflexive relations.

Definition 7.2. Let A be a set, and let R be a relation on A, so that $R \subseteq A \times A$. If (a, a) is contained in R for every element a in A, then we say that R is a *reflexive* relation.

We can check whether a relation on A is reflexive by looking at its arrow diagram. If every dot in the diagram has a loop attached, then the relation is reflexive. Otherwise, it is not reflexive.

Example

The diagrams in Figure 28 show relations on the set $\{x, y, z\}$. The relation in (i) is $\{(x, x), (y, y), (z, z), (x, y), (z, y)\}$. This is a reflexive relation, since every element in $\{x, y, z\}$ has a loop attached. The relation in (ii) is not reflexive, since x and y do not have a loop attached.

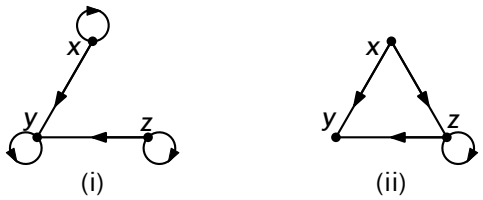


Figure 28:

Example

Let us consider a relation, R, on $\mathbb{Z}$, where (x, y) is in R if and only if x and y are equal. If we could draw the diagram of R, every element of $\mathbb{Z}$ would have a loop, and these would be the only arrows. Therefore this is a reflexive relation.

Symmetric relations.

Definition 7.3. Let A be a set, and let R be a relation on A, so that $R \subseteq A \times A$. If, whenever (a, b) is in R, then (b, a) is in R also, we say that R is *symmetric*.

In the diagram of a symmetric relation, every arrow is a two-way arrow. (We may draw a two-way arrow as two different arrows joining the same pair of dots but going in opposite directions.) Note that loops are automatically two-way arrows, since reversing a loop just gives the same arrow.

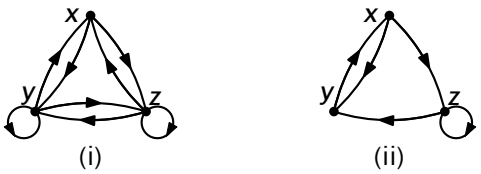


Figure 29:

Example

In Figure 29, the relation in (i) is

$$\{(x,y),(y,x),(x,z),(z,x),(y,z),(z,y),(y,y),(z,z)\}.$$

This is a symmetric relation. We can see this by noting that every arrow in the diagram is a two-way arrow (in other words, every arrow is accompanied by an arrow going in the opposite direction). The relation in (ii) is $\{(x,y),(y,x),(x,z),(z,y),(z,z)\}$ and this is not a symmetric relation, since (z,y) is not accompanied by an arrow going in the opposite direction. That is, (z,y) is in the relation, but (y,z) is not.

Remark

Consider the relation with no ordered pairs. This is the empty relation. Is the empty relation symmetric? It seems strange, but it is. To understand this, think of it this way. A relation fails to be symmetric if we can find an arrow that does not have an accompanying arrow going in the opposite direction. If we can't find such an arrow, then the relation is symmetric. Can we find such an arrow in the empty relation? Certainly not, because there are no arrows at all. Therefore the empty relation is symmetric.

Anti-symmetric relations.

Definition 7.4. Let A be a set, and let R be a relation on A, so that $R \subseteq A \times A$. If, whenever (a,b) and (b,a) are in R, then $a = b$, we say that R is *anti-symmetric*.

The previous definition essentially says that an anti-symmetric relation has no two-way arrows, except that we allow loops (which are always two-way). We can test whether a relation is anti-symmetric by looking at its diagram, and checking that it has no two-way arrows (apart from loops).

Example

In Figure 30, the relation in (i) is $\{(x,x),(y,x),(y,y),(y,z),(z,x)\}$. This is an anti-symmetric relation. We can see this by noting that the diagram has no two-way arrows (apart from loops). The relation in (ii) is $\{(x,y),(x,z),(y,x),(z,z)\}$ and this is not anti-symmetric. To see this note that (x,y) and (y,x) are both in the relation, even though x and y are not equal.

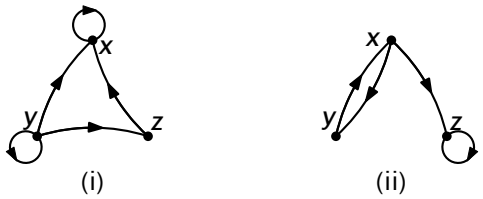


Figure 30:

Remark

Is the empty relation anti-symmetric? A relation fails to be anti-symmetric if there is a two-way arrow that is not a loop. Otherwise the relation is anti-symmetric. Can we find such a two-way arrow in the empty relation? Certainly not, since the empty relation has no arrows at all. Therefore the empty relation is anti-symmetric. It is also symmetric, so this illustrates the surprising fact that a relation can be simultaneously symmetric and anti-symmetric.

Next we define our final property of relations.

Transitive relations.

Definition 7.5. Let A be a set, and let R be a relation on A, so that $R \subseteq A \times A$. We say that R is *transitive* if whenever (a, b) and (b, c) are in R, then (a, c) is also in R.

This definition says that if R is transitive, then whenever we can get from a to c in two steps, we should be able to get there in a single step.

Example

In Figure 31, the relation in (i) is

$$\{(x, x), (x, z), (y, x), (y, y), (y, z), (z, x), (z, z)\}.$$

This is transitive, although it can take a bit of work to see that this is true. We will check a few cases, and leave the rest as an exercise. Note that (y, z) and (z, x) are in the relation, so (y, x) should be in the relation also, and so it is. Also, (x, z) and (z, x) are both in the relation, so (x, x) should also be in the relation. Indeed it is. Similarly, (z, x) and (x, z) are both in the relation, so (z, z) should be in the relation also.
On the other hand, the relation in (ii), which is

$$\{(x, x), (x, z), (y, x), (y, y), (y, z), (z, x), (z, y)\},$$

is not transitive. For example, (x, z) and (z, y) are in the relation, so (x, y) should also be in the relation (assuming it is transitive). Since (x, y) is not in the relation it is not transitive. We could also note that (z, x) and (x, z) are in the relation, but (z, z) is not.

The following table summarises the four properties of relations we have discussed. In each case we assume that $R \subseteq A \times A$ is a relation on the set A.

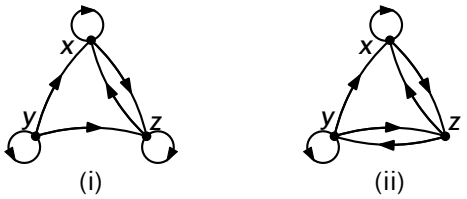


Figure 31:

Property	**Definition**	**Characterisation via arrows**
Reflexive	$(a, a) \in R$ for all $a \in A$	Every element has a loop
Symmetric	$(a, b) \in R$ implies $(b, a) \in R$	Every arrow is two-way
Anti-symmetric	$(a, b) \in R$ and $(b, a) \in R$ implies $a = b$	The only two-way arrows are loops
Transitive	$(a, b) \in R$ and $(b, c) \in R$ implies $(a, c) \in R$	If we can get from a to c in two steps, we can get there in one step

7.2 Equivalence relations and partial orders

Some types are relations are so common that they have special names. The next definition provides an example.

Definition 7.6. Let R be a relation on the set A. If R is reflexive, symmetric, and transitive, then R is an *equivalence relation*.

Example

Consider the relation on $\{a, b, c, d, e\}$ that is shown in Figure 32. Obviously this is a reflexive relation, since every element has a loop attached. It is symmetric, since every arrow is a two-way arrow. It takes a little more work to check that it is transitive, but this is also true. Therefore this is an equivalence relation.

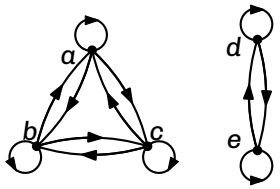


Figure 32:

Note that in the previous example, the equivalence relation divided $\{a, b, c, d, e\}$ into two sets, $\{a, b, c\}$ and $\{d, e\}$. These are called *equivalence classes*. We are going to see that every equivalence relation on the set A divides A into classes in a similar way.

Definition 7.7. Let A be a set. Let $\mathcal{P}$ be a collection of non-empty subsets of A that satisfies the following property: every element of A is in exactly one subset in $\mathcal{P}$. Then $\mathcal{P}$ is a *partition* of A.

Let A be the set $\{a, b, c, d, e\}$. Let $\mathcal{P}$ be $\{\{a, b, c\}, \{d, e\}\}$. Thus $\mathcal{P}$ is a collection of two non-empty subsets of A. Every element of A is in exactly one subset in $\mathcal{P}$. So $\mathcal{P}$ is a partition of A. Note that this is exactly the partition of the equivalence classes in Example 7.2. We will show that in general the equivalence classes of an equivalence relation on A give us a partition of A.

Definition 7.8. Let R be an equivalence relation on the set A. Let a be an element of A. The set $\{x \in A \mid aRx\}$ is said to be an *equivalence class* of R. We denote this equivalence class by $[a]$. Note that $[a]$ is the set containing any element that is the destination of an arrow starting from a.

In Example 7.2 we see that $[a] = \{a, b, c\}$ because there are arrows from a to each of a, b, and c. Note that $[a] = [b] = [c] = \{a, b, c\}$, and $[d] = [e] = \{d, e\}$.

Proposition 7.9. *Let R be an equivalence relation on the set A. Then aRb holds if and only if $[a] = [b]$.*

Proof. We need to prove that if aRb is true then $[a] = [b]$, and we also need to prove the *converse* direction: if $[a] = [b]$ then aRb. We start by assuming that aRb is true. We will use this assumption to prove $[a] = [b]$. We prove that two sets are equal by proving that they have exactly the same elements, so we let z be an arbitrary element of $[a]$. Then aRz holds. We have assumed that aRb is true, and because R is symmetric we deduce that bRa is also true. Because bRa and aRz both hold, transitivity implies bRz. Therefore z is in $[b]$. Now we know that an arbitrary element of $[a]$ is in $[b]$, so $[a] \subseteq [b]$ is true. On the other hand, if z is in $[b]$ then bRz holds, and as aRb is also true, we have aRz by transitivity, so z is in $[a]$. Now $[b] \subseteq [a]$ is also true. Since $[a]$ and $[b]$ have exactly the same elements, we conclude that $[a] = [b]$. So we have proved that aRb implies $[a] = [b]$.

For the other direction we will assume $[a] = [b]$. This means that $[a]$ and $[b]$ have the same elements. Now bRb is true because R is reflexive. This means that b is in $[b]$. Because $[b] = [a]$, it follows that b is in $[a]$. By definition of $[a]$ this means that aRb holds, which is what we needed to prove. This completes the proof. □

Theorem 7.10. *Let R be an equivalence relation on the set A. Then $\{[a] : a \in A\}$ is a partition of A.*

Proof. To prove the result we must show that every element of A is in exactly one set of the form $[a]$. So let x be an arbitrary element of A. Then xRx is true because R is reflexive. Therefore x is in $[x]$. This shows that x is in *at least one* equivalence class.

We can prove that x is in exactly one equivalence class by assuming that x is in two equivalence classes, call them $[a]$ and $[b]$, and then proving that $[a] = [b]$. Because x is in $[a]$ we know that aRx holds by definition. Similarly bRx holds. Now xRb also holds because R is symmetric. Because aRx and xRb are both true, aRb is true because R is transitive. Now Proposition 7.9 implies that $[a] = [b]$, which is what we wanted to prove. □

Here is another important type of relation.

Definition 7.11. Let R be a relation on the set A. If R is reflexive, anti-symmetric, and transitive, then R is a *partial order*. Sometimes a partial order is called a *poset*, meaning a *partially-ordered set*.

Proposition 7.12. *Let A be any set. Consider the following relation R on the power set of A. If X and Y are subsets of A then we declare (X, Y) to be in R if and only if X is a subset of Y. Then R is a partial order.*

Proof. We must prove that R is reflexive, anti-symmetric, and transitive. Let X be an arbitrary subset of A. Then X is a subset of itself, so (X, X) is in R. Therefore R is reflexive.

Assume that (X, Y) and (Y, X) are both in R. We must show that $X = Y$. Now $X \subseteq Y$ and $Y \subseteq X$ both hold. So every element of X is an element of Y and vice versa. Therefore $X = Y$ and R is anti-symmetric.

Finally, we must show that R is transitive. Assume that (X, Y) and (Y, Z) are both in R. Then every element of X is an element of Y, and every element of Y is an element of Z. It follows that every element of X is an element of Z, so (X, Z) is in R. Therefore R is transitive and the proof is complete. □

The next result will let us draw diagrams that represent partial orders. It says that the directed graph representing a partial order can never contain a *directed cycle*.

Proposition 7.13. *Let R be a partial order on the set A. Let $a_1, a_2, \ldots, a_n$ be a sequence of distinct elements of A, where n is at least 2. Assume that $a_1 R a_2, a_2 R a_3, \ldots, a_{n-1} R a_n$ are all true. Then $a_n \not R a_1$.*

Proof. We give a proof by contradiction. Let us assume that we can find a sequence of distinct elements $a_1, a_2, \ldots, a_n$ such that $a_1 R a_2, a_2 R a_3, \ldots, a_{n-1} R a_n$ are all true. Let us assume that we also have $a_n R a_1$. We will show that this leads to an impossible situation, and therefore we will have to conclude that $a_n \not R a_1$. Assume that we have chosen this sequence of elements so that it is as short as possible. If $n = 2$ then we have $a_1 R a_2$ and $a_2 R a_1$, but this is impossible since R is anti-symmetric and $a_1 \neq a_2$. Now assume n is greater than 2. Now $a_{n-1} R a_n$ and $a_n R a_1$ are true, and since R is transitive, it follows that $a_{n-1} R a_1$. There we know that $a_1 R a_2, a_2 R a_3, \ldots, a_{n-2} R a_{n-1}$ are all true, and so is $a_{n-1} R a_1$. But $n - 1$ is at least 2 since n is greater than 2. So why did we not choose the sequence $a_1, a_2, \ldots, a_{n-1}$ instead of $a_1, a_2, \ldots, a_n$? We have contradicted our choice of the shortest possible sequence. This impossible situation means that we must reject the idea that such a sequence exists. Therefore no such sequence exists and the proof is complete. □

We often represent partial orders with a special type of diagram called a *Hasse diagram*. These are like the directed graph of any relation, but we modify them to make them simpler. First of all, a partial order is reflexive, so every element has a loop attached to it. We don't really need to draw the loops — we can just assume that every element is related to itself. Next, we leave out the arrow marks and just draw the diagram in such a way that every arrow points upwards. This is possible because of Proposition 7.13. If there were some directed cycle in the directed graph, it would not be possible to orient every arrow upwards. Finally, we know that the relation is transitive. This means that if a is related to b and b is related to c, then we can leave out the arrow from a to c, because it is implied by the other two arrows, and the fact that the relation is transitive.

Example

Consider the subsets of $\{1, 2, 3\}$. Proposition 7.12 says that the relation R is a partial order, where (X, Y) is in R if and only if $X \subseteq Y$. The Hasse diagram of this order is shown in Figure 33.

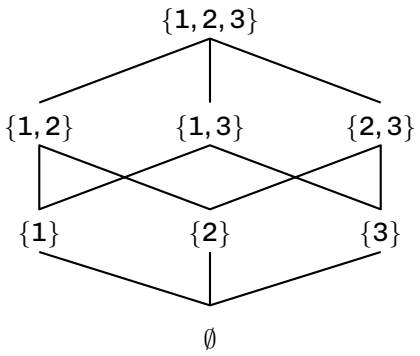


Figure 33: The Hasse diagram of the partial order of subsets of $\{1, 2, 3\}$.

Summary

Important ideas from Section 7.

- Let A and B be sets. A relation from A to B is a subset of the Cartesian product $A \times B$. If $A = B$ then we say that it is a relation on A.
- If $R \subseteq A \times A$ is a relation on A, then A is:
  - (i) reflexive if $(a, a) \in R$ for every $a \in A$,
  - (ii) symmetric if $(a, b) \in R$ implies $(b, a) \in R$,
  - (iii) anti-symmetric if $(a, b) \in R$ and $(b, a) \in R$ implies $a = b$,
  - (iv) transitive if $(a, b) \in R$ and $(b, c) \in R$ implies $(a, c) \in R$.
- A relation on A is an equivalence relation if it is reflexive, symmetric, and transitive.
- If $R \subseteq A \times A$ is an equivalence relation on A, then $[a] = \{b \in A \mid (a, b) \in R\}$ is an equivalence class.
- Let A be a set. A partition of A is a set $\mathcal{P}$ of subsets of A such that every element of A belongs to exactly one subset in $\mathcal{P}$.
- The equivalence classes of an equivalence relation on A form a partition of A.
- Let A be a set. A partial order on A is a relation that is reflexive, anti-symmetric, and transitive.

8 Enumeration

In this section we will discuss some techniques for counting things. The branch of mathematics that studies these techniques is called *enumeration*. Enumeration is very important in computer science, since we often use it to do things like count the number of loops an algorithm will need to execute before it halts.

8.1 Cardinality

The number of elements in a set is called its *cardinality*, or its *size*. If A is a set, then $|A|$ denotes its cardinality. Many sets have infinite cardinality, but we will restrict our attention to finite sets.

Example

If $A = \{4, 8, 12\}$ then $|A| = 3$. The cardinality of $\emptyset$ is 0.

Recall that a bijection is a function that is injective and surjective. We will use the fundamental fact that two sets have the same cardinality if and only if there is a bijection between them.

8.2 Addition and multiplication rules

If A and B are sets, what is the cardinality of $A \cup B$? It is tempting to say $|A \cup B| = |A| + |B|$, since we first count the elements in A, and then count the elements in B. But this is not right, as there might be some elements in $A \cap B$, and these will have been counted twice. Therefore to find $|A \cup B|$, we take the size of A, add the size of B, and then subtract the number of elements in both sets. In other words, we have the following result.

Proposition 8.1. *If A and B are finite sets then $|A \cup B| = |A| + |B| - |A \cap B|$.*

Example

If $A = \{a, c, e, f, i\}$, and $B = \{b, c, d, g, i, j, k\}$, then $|A| = 5$, $|B| = 7$. Also, $A \cap B = \{c, i\}$, so $|A \cap B| = 2$. Therefore

$$|A \cup B| = 5 + 7 - 2 = 10.$$

Since $A \cup B = \{a, b, c, d, e, f, g, i, j, k\}$, we can confirm that this is correct.

Definition 8.2. Two sets are *disjoint* if they do not have any elements in common. In other words, A and B are disjoint if $A \cap B = \emptyset$.

Proposition 8.1 implies the next result.

Proposition 8.3. *Let A and B be two disjoint sets. Then $|A \cup B| = |A| + |B|$.*

Sometimes we refer to Proposition 8.3 as the *addition rule*. More generally, suppose that $A_1, \ldots, A_m$ are sets, and each pair of these sets is disjoint. (We sometimes express this by saying that $A_1, \ldots, A_m$ are *pairwise disjoint*.) Then

$$|A_1 \cup \cdots \cup A_m| = |A_1| + |A_2| + \cdots + |A_m|.$$

Remember that if A and B are sets, then $A \times B$ is the Cartesian product: the set of all ordered pairs (a, b) where a is in A and b is in B. If the set A has m elements and B has n elements, then how many pairs are there in the Cartesian product $A \times B$? This is answered by our second fundamental counting result.

Proposition 8.4. *Suppose that A and B are finite sets. Then $|A \times B| = |A| \times |B|$.*

Proof. Let the elements of A be $a_1, \ldots, a_m$, so that $|A| = m$. For each $i \in \{1, \ldots, m\}$, let A_i be the set of ordered pairs in $A \times B$ where the first coordinate is a_i. That is,

$$A_i = \{(a_i, b) \mid b \in B\}.$$

Note that $A_1, \ldots, A_m$ are pairwise disjoint, since if two pairs come from A_i and A_j (where $i \neq j$), then those pairs differ in their first coordinates. Moreover, every pair in $A \times B$ belongs to one of the sets $A_1, \ldots, A_m$, so $A_1 \cup \cdots \cup A_m = A \times B$.

There is a bijection from A_i to B. Each pair (a_i, b) in A_i is mapped to the element $b \in B$. This argument shows that A_i and B have exactly the same size. Now the addition rule implies that

$$|A \times B| = |A_1 \cup \cdots \cup A_m| = |A_1| + \cdots + |A_m| = m \times |B| = |A| \times |B|$$ □

Example

If $A = \{1, 2\}$ and $B = \{x, y, z\}$, then $A \times B = \{(1, x), (1, y), (1, z), (2, x), (2, y), (2, z)\}$, so $|A \times B| = 6$, which is equal to $|A| \times |B| = 2 \times 3 = 6$.

Remember that if $A_1, A_2, \ldots, A_m$ are sets, then the Cartesian product $A_1 \times A_2 \times \cdots \times A_m$ is the set of all tuples $(a_1, a_2, \ldots, a_m)$ where each a_i is an element of A_i. Generally, if $A_1, \ldots, A_m$ are finite sets, then

$$|A_1 \times \cdots \times A_m| = |A_1| \times \cdots \times |A_m|.$$

This is the *multiplication rule*: if we have n choices to make and for the first choice we can choose from i_1 options, for the second choice we can choose from i_2 options, and so on, then the total number of ways we can make the choices is $i_1 \cdot i_2 \cdots i_n$. The multiplication rule is sometimes called the *rule of product*. We can visualise the multiplication rule with a diagram like the one in Figure 34. We make a sequence of three choices, starting at the top of the tree. At the first choice we have two options (left or right). At the second choice we have four options, and at the third choice we have three options. The multiplication rule tells us that the number of paths from the top of the tree to the bottom is $2 \cdot 4 \cdot 3 = 24$.

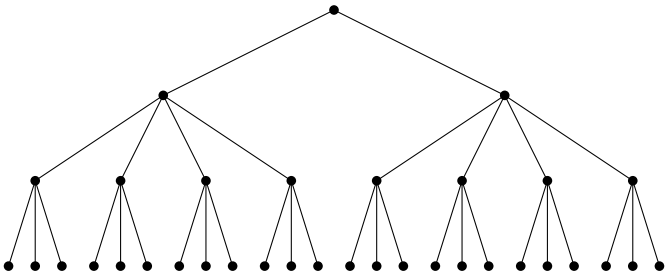


Figure 34: An illustration of the multiplication rule.

Here is an important consequence of the multiplication rule.

Proposition 8.5. *Suppose that A is a finite set and that $|A| = n$. The number of subsets of A is 2^n.*

Proof. Suppose that $A = \{a_1, \ldots, a_n\}$. How many ways are there of choosing a subset A' of A? For a_1 we have two options: either a_1 is in A' or it isn't. We have to decide between these options for each of the elements in A. Therefore we must make n choices, and for each of these we have 2 options. Therefore the total number of options is

$$\underbrace{2 \times 2 \times \cdots \times 2}_{n \text{ times}} = 2^n$$

□

So if A is a finite set, then the size of the power set of A is $2^{|A|}$. This explains why we sometimes see the notation 2^A for the power set of A.

The next exercises involve further applications of the multiplication rule.

Exercise

Let G be a *complete bipartite graph*, where the vertices are partitioned into sets X and Y, where every edge joins a vertex in X to a vertex in Y, and whenever x is in X and y is in Y, then there is an edge joining x to y. If $|X| = m$ and $|Y| = n$ then what is the number of edges in G?

Exercise

Find the number of n-letter words we can make out of an alphabet of q letters.

8.3 Permutations

Let A be a set. A *permutation* of A is an ordering of the elements of A.

Example

Assume that $A = \{x, y, z\}$. Then there are six permutations of A, and they are:

$$xyz, \quad xzy, \quad yxz, \quad yzx, \quad zxy, \quad zyx.$$

If the set A has n elements, how many ways are there of ordering the elements of A? In other words, how many ways can we choose an ordering of the elements in A? We have n options for the first element. We then have $n - 1$ options for the second element, since we cannot reuse the first element. In the same way, we have $n - 2$ options for the third element, and so on until we reach the final element (we only have one option for the final element, since all the other elements have already been used). The multiplication rule now tells us that the total number of ways we can choose an ordering is $n \cdot (n-1) \cdot (n-2) \cdots 2 \cdot 1$. This number is called n *factorial*. We write it using the notation $n!$.

Proposition 8.6. *The number of ways of ordering a set of n elements is $n!$.*

Example

If $A = \{x, y, z\}$, then $|A| = 3$, so there are $3! = 3 \cdot 2 \cdot 1$ permutations of the elements in A. We also have $4! = 4 \cdot 3 \cdot 2 \cdot 1 = 24$. Note that $n! = n \cdot (n-1)!$, so $5! = 5 \cdot 4! = 5 \cdot 24 = 120$.

Remark

Note that $1! = 1$. It turns out to be convenient to define $0!$ to be equal to 1 also, for reasons that we will discuss soon.

The multiplication rule tells us the number of ways we can make an ordered selection of elements from a set. When we talk about an ordered selection, we mean that the order in which we select the elements is significant. As an example, imagine that we have a class of twenty students, and we going to select three of them to form a committee: one president, one vice-president, and one treasurer. We first select the president, then the vice-president, and then the treasurer. The order in which we select the students is significant. Selecting Alice, then Bob, then Che is different from selecting Bob, then Che, then Alice, because in the first scenario Alice is president, and in the second scenario Bob is president. How many ways can we make our selection? The next result answers this question.

Proposition 8.7. *Let A be a set with cardinality n. Let k be an integer satisfying $0 \leq k \leq n$. Imagine that we want make an ordered selection of k distinct elements from A. The number of ways we can do so is*

$$n \cdot (n-1) \cdot (n-2) \cdots (n-k+1) = \frac{n!}{(n-k)!}.$$

Proof. The number of options we have for choosing the first element is n. The number of options we have when choosing the second element is $n-1$, since we cannot reuse the first element. The number of options we have for the third element is $n-2$, and so on. Note that the number of options at the first choice is $n = n-(1-1)$, the number at the second is $n-1 = n-(2-1)$, at the third is $n-2 = n-(3-1)$, and so on. In general, at choice number i, we have $n-(i-1)$ options. At choice number k we have $n-(k-1) = n-k+1$ options. The multiplication rule tells us that the total number of choices we can make is

$$n \cdot (n-1) \cdot (n-2) \cdots (n-k+1).$$

To see that this is equal to $n!/(n-k)!$ we observe that this number is

$$\frac{n \cdot (n-1) \cdots (n-k+1) \cdot (n-k) \cdot (n-k-1) \cdots 2 \cdot 1}{(n-k) \cdot (n-k-1) \cdots 2 \cdot 1}.$$

We cancel out the numbers in the denominator and we obtain $n \cdot (n-1) \cdot (n-2) \cdots (n-k+1)$. □

8.4 Binomial coefficients

Proposition 8.5 tells us that when A has n elements, there are a total of 2^n subsets of A. What about if we ask about the number of subsets of a particular size?

Example

If $A = \{a, b, c, d\}$, then A has $2^4 = 16$ subsets. But the subsets of size two are $\{a, b\}$, $\{a, c\}$, $\{a, d\}$, $\{b, c\}$, $\{b, d\}$, and $\{c, d\}$, so there are six such subsets.

For example, suppose A has 10 elements, and we want to know the number of subsets of size 3. Proposition 8.7 tells us that the number of ways we can make an *ordered selection* of 3 elements is $10 \cdot 9 \cdot 8$. But this is an overcount, because when we are counting subsets, the

order of the elements is not significant. Each subset of size 3 can be ordered in 3! = 6 ways. We have counted these 6 orderings separately. To get the correct answer, we need to divide by 6 to compensate for this. So the number of 3-element subsets is $(10 \cdot 9 \cdot 8)/3!$. Our final answer is

$$\frac{10!}{(10-3)!3!} = \frac{3628800}{5040 \cdot 6} = 120.$$

Therefore there are 120 3-element subsets of A when A has 10 elements.

In general, if A has size n, and we wish to count the number of subsets of size k, then we can repeat the argument above and see that the number of k-element subsets is

$$\frac{n!}{(n-k)!k!}.$$

We use the notation

$$\binom{n}{k}$$

and we say "n choose k" to represent this number, since it is the number of ways we can choose k elements from a set of size n. We summarise this in the next result.

Proposition 8.8. *The number of k-element subsets of a set of size n is*

$$\binom{n}{k} = \frac{n!}{(n-k)!k!}.$$

Example

The number of subsets of size 4 from a set of size 12 is

$$\frac{12!}{(12-4)!4!} = \frac{12!}{8!4!} = \frac{479001600}{40320 \cdot 24} = 495.$$

Remark

The notation ${}^nC_k$ is sometimes used instead of $\binom{n}{k}$.

Remark

How many subsets of size zero are there in a set of size n? There is only one empty subset, so the answer should be one. So we would like $\binom{n}{0}$ to be equal to one. Now

$$\binom{n}{0} = \frac{n!}{(n-0)!0!} = \frac{n!}{n! \cdot 0!}.$$

This is equal to one if we set 0! to be equal to 1. This is our main motivation for defining 0! to be 1.

Exercise

Calculate $\binom{n}{n}$. Can you justify the answer you obtain?

Numbers of the form $\binom{n}{k}$ are called *binomial coefficients*. This is because they appear in expressions of the form

$$(a+b)^n.$$

This is a *binomial* expression because it has *two* variables in it: a and b. You probably know that

$$(a+b)^2 = a^2 + 2ab + b^2 = 1a^2 + 2ab + 1b^2.$$

(We wouldn't normally explicitly write the coefficients that are equal to 1, but we want to draw attention to the pattern here.) The coefficients of this expression are $1, 2, 1$. We can calculate that

$$1 = \binom{2}{0}, \quad 2 = \binom{2}{1}, \quad 1 = \binom{2}{2}.$$

What about $(a+b)^3$? This is equal to $1a^3 + 3a^2b + 3ab^2 + 1b^3$. The coefficients are $1, 3, 3, 1$. We can also calculate that

$$1 = \binom{3}{0}, \quad 3 = \binom{3}{1}, \quad 3 = \binom{3}{2}, \quad 1 = \binom{3}{3}.$$

This is not a coincidence. The following result tells us that it is part of a pattern.

Theorem 8.9 (The Binomial Theorem)**.** *Let $n \geq 0$ be an integer. Then*

$$(a+b)^n = \binom{n}{0}a^n b^0 + \binom{n}{1}a^{n-1}b^1 + \cdots + \binom{n}{n-1}a^1 b^{n-1} + \binom{n}{n}a^0 b^n.$$

We could also use *summation* notation to express this result:

$$(a+b)^n = \sum_{i=0}^{n} \binom{n}{i} a^{n-i} b^i.$$

This notation tells us to evaluate the expression $\binom{n}{i}a^{n-i}b^i$ as i ranges over the values $0, 1, 2, \ldots, n$, and then add all the evaluations together.

We often arrange the binomial coefficients in a triangular array as follows.

$$\begin{array}{ccccccccccccccc}
 & & & & & & 1 & & & & & & & & \\
 & & & & & 1 & & 1 & & & & & & & \\
 & & & & 1 & & 2 & & 1 & & & & & & \\
 & & & 1 & & 3 & & 3 & & 1 & & & & & \\
 & & 1 & & 4 & & 6 & & 4 & & 1 & & & & \\
 & 1 & & 5 & & 10 & & 10 & & 5 & & 1 & & & \\
1 & & 6 & & 15 & & 20 & & 15 & & 6 & & 1 & & \\
 & 7 & & 21 & & 35 & & 35 & & 21 & & 7 & & 1 &
\end{array}$$

This is called *Pascal's Triangle*. The top row contains the number $\binom{0}{0}$. The second row contains $\binom{1}{0}$ and $\binom{1}{1}$. The third row contains $\binom{2}{0}$, $\binom{2}{1}$, and $\binom{2}{2}$, and so on. We notice that each number in the triangle is the sum of the two numbers above it. This is because of the following result.

Proposition 8.10. *Suppose that n is a positive integer and that k is a positive integer that is less than n. Then*

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

The equation in Proposition 8.10 is sometimes called *Pascal's identity*.

Exercise

Prove Proposition 8.10.

8.5 The pigeonhole principle

The *pigeonhole principle* receives its name because of the following problem: imagine that there are n pigeons distributed into k pigeonholes. There are many such distributions. In each case, we look at the maximum number of pigeons in a pigeonhole.

Example

Let's say we are distributing 18 pigeons into five pigeonholes. Here are five possible distributions.

$$|3|0|4|10|1| \qquad |0|0|0|0|18| \qquad |1|2|3|4|8| \qquad |1|1|14|1|1| \qquad |3|3|4|4|4|$$

In these distributions, the maximum number of pigeons in a pigeonhole are: 10, 18, 8, 14, and 4, respectively.

Can we say anything about the maximum number of pigeons in a pigeonhole? In particular, can we give a lower bound on this maximum number? That is, can we calculate some number N such that the maximum number of pigeons in a pigeonhole will be at least N, in every possible distribution?

Definition 8.11. If x is any real number, then $\lfloor x \rfloor$ (called the *floor* of a) is the largest integer that is less than or equal to x, and $\lceil x \rceil$ (called the *ceiling* of x) is the smallest integer that is greater than or equal to x.

So $\lfloor x \rfloor$ is produced by rounding x down to the nearest integer, and $\lceil x \rceil$ is produced by rounding up to the nearest integer. If x is an integer, then $\lfloor x \rfloor = \lceil x \rceil = x$.

Example

$$\lfloor \pi \rfloor = 3 \qquad \lceil 4/3 \rceil = 2 \qquad \lfloor 7 \rfloor = 7 \qquad \lfloor 1.999999 \rfloor = 1 \qquad \lceil 1.999999 \rceil = 2$$

Proposition 8.12. *If n pigeons are distributed into k pigeonholes, the maximum number of pigeons in the same pigeonhole is at least $\lceil n/k \rceil$.*

Proof. Assume for a contradiction that every pigeonhole contains fewer than $\lceil n/k \rceil$ pigeons. Since $\lceil n/k \rceil$ is an integer, this means that each pigeonhole contains at most $\lceil n/k \rceil - 1$ pigeons. Because $\lceil n/k \rceil$ is produced by rounding n/k up to the nearest integer, the difference between $\lceil n/k \rceil$ and n/k is less than one. In other words, $\lceil n/k \rceil - n/k < 1$. Rearranging, we have $\lceil n/k \rceil - 1 < n/k$. Now we have k pigeonholes, each one contains at most $\lceil n/k \rceil - 1$ pigeons, and this number is less than n/k. So the total number of pigeons is less than $k \cdot (n/k) = n$, a contradiction since there are n pigeons. □

Example

If we distribute 18 pigeons into 5 pigeonholes, the maximum number of pigeons in a pigeonhole will be at least $\lceil 18/5 \rceil = \lceil 3.6 \rceil = 4$.

A distribution of n pigeons into k pigeonholes is like a function from a domain of n pigeons to a codomain of k pigeonholes. So we can restate the pigeonhole principle in terms of functions.

Proposition 8.13. *Let f be a function from A to B, where $|A| = n$ and $|B| = k$. Then there is an element $b \in B$ such that at least $\lceil n/k \rceil$ elements in A are mapped to b by f.*

Exercise

The idea of a *hash function* is that we have a set of data values, but it may be hard to search through this set to find whether a particular piece of data is present. So instead we have a function h which maps the set of data values to a codomain of *hash values*. The hash values are easier to search, so instead of searching for a value v, we will search the list of hash values for $h(v)$.
This approach cannot work if two data values have the same hash value. This phenomenon is known as a *collision*. Collisions are serious problems in hash functions.
Say that we have a hash function from a set of 1024 data values. Assume that the number of possible hash values is 100. Prove that there is a hash value that has at least 11 values mapped to it.

8.6 Inclusion-exclusion

In Proposition 8.1 we noted that if A and B are sets, then the number of elements in $A \cup B$ is $|A| + |B| - |A \cap B|$. How can we generalize this to larger collections of sets?

Consider the sets $A_1 = \{1, 4, 7, 8\}$, $A_2 = \{1, 2, 4, 5, 6, 9\}$, and $A_3 = \{1, 3, 5, 6, 8, 9\}$. We want to find the cardinality of $A_1 \cup A_2 \cup A_3$. A logical place to start is by adding together the cardinalities of A_1, A_2, and A_3. This gives us $4 + 6 + 6 = 16$. However, we have now counted the elements in $A_1 \cap A_2$, $A_1 \cap A_3$, and $A_2 \cap A_3$ twice, so we subtract the cardinalities of these three sets. This gives us

$$16 - |\{1, 4\}| - |\{1, 8\}| - |\{1, 5, 6, 9\}| = 8.$$

But this is still not the answer, because the elements in $A_1 \cap A_2 \cap A_3$ were counted three times in $|A_1| + |A_2| + |A_3|$, and then subtracted three times. Therefore we have not counted them at all. So to get the correct answer we must add the cardinality of $A_1 \cap A_2 \cap A_3 = \{1\}$. Thus $A_1 \cup A_2 \cup A_3$ has cardinality 9.

In general, we have the following rule:

Proposition 8.14. *Suppose that A_1, A_2, and A_3 are finite sets. Then*

$$|A_1 \cup A_2 \cup A_3| = |A_1| + |A_2| + |A_3| - |A_1 \cap A_2| - |A_1 \cap A_3| - |A_2 \cap A_3| + |A_1 \cap A_2 \cap A_3|.$$

But this still hasn't answered the general question. What if we have n sets, where n is some unspecified number? The answer is given by the next result.

Theorem 8.15 (The principle of inclusion-exclusion)**.** *Suppose that we have n finite sets, $A_1, \ldots, A_n$. Then*

$$|A_1 \cup \cdots \cup A_n| = \sum_{1 \le i \le n} |A_i| - \sum_{1 \le i < j \le n} |A_i \cap A_j| + \sum_{1 \le i < j < k \le n} |A_i \cap A_j \cap A_k| + \cdots + (-1)^{s+1} \sum_{1 \le i_1 < \cdots < i_s \le n} |A_{i_1} \cap \cdots \cap A_{i_s}| + \cdots + (-1)^{n+1} |A_1 \cap \cdots \cap A_n|.$$

Remark

The formula in Theorem 8.15 might look complicated, but it is easy enough to understand. To find the cardinality of $A_1 \cup \cdots \cup A_n$, we start by adding together the cardinalities of all the sets. Then we subtract the cardinalities of all possible intersections of two sets. We add the cardinalities of all possible intersections of three sets. Continue in this way, alternating addition and subtraction, until you reach the intersection of all n sets. Because we are alternating between including sets and excluding them in the count, this method is called *inclusion-exclusion*.

We could write the formula in Theorem 8.15 in another way:

$$\left| \bigcup_{i=1}^{n} A_i \right| = \sum_{I \subseteq \{1,\ldots,n\}} (-1)^{|I|+1} \left| \bigcap_{i \in I} A_i \right|.$$

This formula works because -1 raised to the power of an even integer is 1 and -1 raised to the power of an odd integer is -1. So in this formula we are saying that we take every possible subset I of $\{1, \ldots, n\}$. We find the cardinality of the intersection of all subsets A_i where i is in I. We then add this cardinality if $|I|$ is odd and subtract the cardinality if $|I|$ is even.

Example

How many binary strings of length 4 contain a pair of consecutive 1s? For $i = 1, 2, 3$, let A_i be the set of binary strings with length 4 that contain 1s in position i and $i + 1$. Any string with a pair of consecutive 1s belongs to one of A_1, A_2, or A_3. Therefore we would like to find the cardinality of $A_1 \cup A_2 \cup A_3$. First we find the cardinality of A_1. In any string of A_1, positions 1 and 2 are filled with 1s. For each of the other 2 positions, we have two options, 0 or 1. By the multiplication rule, there are $2 \cdot 2 = 2^2 = 4$ strings in A_1. The same argument shows that $|A_2| = |A_3| = 4$ for every i. What about the cardinality of $A_1 \cap A_2$? Any string in this set contains 1s in positions 1, 2, and 3, so we only have a choice for position 4. Therefore $|A_1 \cap A_2| = 2$. The same argument shows that $|A_2 \cap A_3| = 2$. On the other hand, any string in $A_1 \cap A_3$ is made up of 1s in every position, so there is only one such string. Therefore $|A_1 \cap A_3| = 1$. In the same way, $|A_1 \cap A_2 \cap A_3| = 1$. Now

$$\begin{aligned} |A_1 \cup A_2 \cup A_3| &= |A_1| + |A_2| + |A_3| - |A_1 \cap A_2| - |A_1 \cap A_3| - |A_2 \cap A_3| + |A_1 \cap A_2 \cap A_3| \\ &= 4 + 4 + 4 - 2 - 1 - 2 + 1 \\ &= 8 \end{aligned}$$

Can you find all 8 strings?

8.7 Recursive definitions

Let us try to solve a counting problem. Say that n is a non-negative integer. Let $F(n)$ be the number of sequences of odd positive integers that sum to n. Let's try and calculate the first

few values $F(0), F(1), F(2), \ldots$ by counting the number of such sequences.

If $n = 0$, then there are no sequences of odd positive integers that sum to n, so $F(n) = 0$. When $n = 1$, there is only one sequence: 1 by itself. So $F(n) = 1$. Similarly, if $n = 2$, then we have only one sequence: $1, 1$, so $F(n) = 1$. Next, when $n = 3$, we have $1, 1, 1$ and 3, so $F(n) = 2$, and if $n = 4$, then we have $1, 1, 1, 1$ as well as $3, 1$ and $1, 3$, so $F(n) = 3$. We'll do one more. When $n = 5$, we have

$$1, 1, 1, 1, 1 \qquad 3, 1, 1 \qquad 1, 3, 1 \qquad 1, 1, 3 \qquad \text{and} \qquad 5,$$

so $F(n) = 5$. By calculating in this way we can find the following table:

n	0	1	2	3	4	5	6	7	8
$F(n)$	0	1	1	2	3	5	8	13	21

You may recognise this sequence of numbers. It is called the *Fibonacci* sequence. It can be defined by the starting values $F(0) = 0$ and $F(1) = 1$ and the fact that $F(n) = F(n-1) + F(n-2)$ when n is greater than 1. So each number in the sequence is defined by the two numbers before it. This makes the sequence *recursively* defined. Recursive definitions such as this are very common in computer science applications.

Here is the reason that our sequence obeys this recursive definition. Say that we have all the sequences of odd positive integers that sum to $n-2$ and all the sequences that sum to $n-1$. We can obtain sequences of odd positive integers that sum to n by either adding 2 to the last number in a sequence that sums to $n-2$, or we can add another 1 to the end of a sequence that sums to $n-1$. It is easy to see that this recipe generates all the sequences that sum to n, because every such sequence either ends with a 1, or it ends with an odd number greater than 1. In the first case the sequence was generated by adding a 1 to the end of a sequence summing to $n-1$, and in the second case it is generated by adding 2 to the final number in a sequence summing to $n-2$. So this explains why $F(n) = F(n-1) + F(n-2)$.

Summary

Important ideas from Section 8.

- If A is a finite set, then $|A|$ stands for its cardinality, or size, which is the number of elements in A.
- If $A_1, A_2, \ldots, A_m$ are pairwise disjoint finite sets, then the size of $A_1 \cup A_2 \cup \cdots \cup A_m$ is $|A_1| + |A_2| + \cdots + |A_m|$. This is called the addition rule.
- If $A_1, A_2, \ldots, A_m$ are finite sets, then the size of the Cartesian product $A_1 \times A_2 \times \cdots \times A_m$ is $|A_1| \times |A_2| \times \cdots \times |A_m|$. This is called the multiplication rule.
- If A is a finite set, then $|\mathcal{P}(A)| = 2^{|A|}$.
- The number of ways of ordering n objects is $n! = n \cdot (n-1) \cdot (n-2) \cdots 2 \cdot 1$, which we call n factorial.
- The number of ways of making an ordered selection of k distinct elements from a set of n elements is $n!/(n-k)!$.
- The number of ways of choosing a subset of k elements from a set of n elements is $\binom{n}{k} = n!/((n-k)!k!)$.
- The Binomial Theorem says that $(a+b)^n = \sum_{i=0}^{n} \binom{n}{i} a^{n-i} b^i$.

- The pigeonhole principle says that if $f\colon A \to B$ is a function, where $|A| = n$ and $|B| = k$, then there is an element $b \in B$ such that at least $\lfloor n/k \rfloor$ elements in A are mapped to b.
- The inclusion-exclusion principle says that if $A_1, A_2, \ldots, A_n$ are finite sets, then

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{I \subseteq \{1,\ldots,n\}} (-1)^{|I|+1} \left|\bigcap_{i \in I} A_i\right|.$$

9 Number Theory

Number theory is the study of the integers, $\ldots, -2, -1, 0, 1, 2, \ldots$, and their properties. In this section we will think about integers, how to represent them, and how to work with them. Before we begin, remember that the notation $a > b$ or $b < a$ means that a is greater than b, and $a \geq b$ or $b \leq a$ means that a is greater than or equal to b. An integer is positive if it is greater than zero, and negative if it is less than zero.

9.1 Divisibility

Definition 9.1. Let p and q be integers. We say that p *divides* q if q can be written as kp for some integer k. We use the notation $p|q$ to indicate that p divides q, and we write $p \nmid q$ to mean that p does not divide q. If p divides q, we also say that q is a *multiple* of p or that p is a *factor* of q.

Example

Note that the integer p is even if and only if $2|p$. The following statements are all true.

$$3|18, \quad 5 \nmid 12, \quad 1|7, \quad 0|0, \quad 0 \nmid 9, \quad 9|0.$$

Remark

It is tempting to say that $p|q$ holds if $\frac{q}{p}$ is an integer. But this isn't equivalent to the definition we have given. What if $p = 0$?

Here are some basic facts about divisibility.

Proposition 9.2. (i) *If p is any integer, then $p|0$, $1|p$, and $p|p$.*

(ii) *If $p \neq 0$, then $0 \nmid p$.*

(iii) *If $p|a$, then $p|(ab)$, for any integer b.*

(iv) *If $p|a$ and $p|b$, then $p|(a+b)$.*

Proof. We will leave the first three statements as exercises and prove (iv). Assume that $p|a$ and $p|b$. Then a can be written as mp and b can be written as np, for some integers m and n. This means that $a+b = mp+np = (m+n)p$. Since $m+n$ is an integer, it follows that p divides $a+b$. □

Proposition 9.3. *Let A be a set of positive integers. Consider the following relation R on the elements of A. If a and b are positive integers then we declare (a, b) to be in R if and only if a divides b. Then R is a partial order.*

Proof. We need to prove that R is reflexive, anti-symmetric, and transitive. Let a be any integer in A. Since $a = 1 \cdot a$, it follows that a divides itself. Therefore (a, a) is in R, so R is reflexive.

Next we show R is anti-symmetric. Assume that (a, b) and (b, a) are in R. We need to show that $a = b$. Since a divides b, we can write b as ma for some integer m. Similarly, b divides a, so we can write a as nb for some integer n. Now $a = nb = n(ma) = (nm)a$. By dividing both sides of $a = (nm)a$ by a, we see that $nm = 1$. Now m and n are integers, and their product is 1. This means that either $m = n = 1$ or $m = n = -1$. But a and b are positive, so $n \neq -1$, or else $a = -b$. Therefore $m = n = 1$, implying that $a = nb = 1b = b$, and this is what we wanted to prove.

Finally, we need to check that R is transitive. Assume (a, b) and (b, c) are in R. Then $b = ma$ for some integer m and $c = nb$ for some integer n. Now $c = nb = n(ma) = (nm)a$. Since nm is an integer, this means that a divides c, meaning that (a, c) is in R. This shows that R is transitive, and the proof is complete. □

Exercise

Let A be the set $\{1, 2, 3, 4, 5, 6, 9, 10, 12, 18, 20\}$. Define the relation R on A so that aRb if and only if a divides b. Proposition 9.3 shows that R is a partial order, and Figure 35 shows its Hasse diagram.

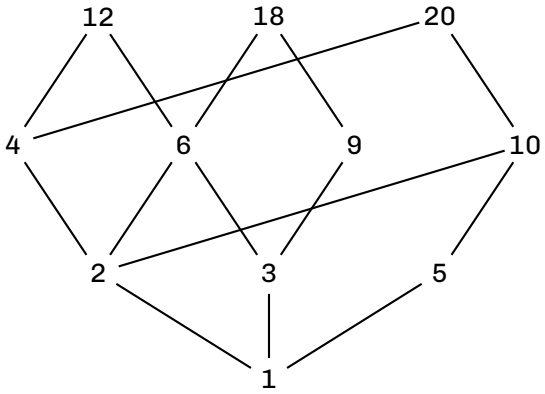


Figure 35: A Hasse diagram showing a divisibility relation.

9.2 Number representations

The way in which we represent numbers is so ingrained that it can be hard to be conscious of it. Let's analyse it in more detail. When we use the notation 2407 we are representing the integer

$$2 \cdot 1000 + 4 \cdot 100 + 0 \cdot 10 + 7 \cdot 1 = 2 \cdot 10^3 + 4 \cdot 10^2 + 0 \cdot 10^1 + 7 \cdot 10^0.$$

In general, any *decimal representation* of a non-negative integer is written in the form $a_n a_{n-1} \cdots a_1 a_0$, and it represents the number

$$a_n 10^n + a_{n-1} 10^{n-1} + \cdots + a_1 10^1 + a_0 10^0.$$

If we want to remind ourselves that we are using decimal notation, we write a subscript 10 after the number. So $(a_n a_{n-1} \cdots a_1 a_0)_{10}$ stands for $a_n 10^n + a_{n-1} 10^{n-1} + \cdots + a_1 10^1 + a_0 10^0$. Note that each of the digits in this representation is an integer between 0 and $10 - 1$.

There is nothing special about 10 in this representation: we only use 10 because we have 10 digits on our hands. We can use other numbers just as well. If b is an integer greater than 1, then $(a_n a_{n-1} \cdots a_1 a_0)_b$ stands for the number

$$a_n b^n + a_{n-1} b^{n-1} + \cdots + a_1 b^1 + a_0 b^0$$

where each of these digits is an integer between 0 and $b - 1$. The number b is said to be the *base* of the representation.

Example

$(4036)_7$ is the number $4 \cdot 7^3 + 0 \cdot 7^2 + 3 \cdot 7^1 + 6 \cdot 7^0 = 4 \cdot 343 + 3 \cdot 7 + 6 = 1399$. This shows that $(4036)_7 = (1399)_{10}$.

In computer science, the most common bases used to represent numbers are 2, 8, 10, and 16. These are known as *binary*, *octal*, *decimal*, and *hexadecimal* representations respectively. Note that in hexadecimal representations, we need digits to represent all the integers between 0 and $16 - 1 = 15$. We use the digits $0, 1, 2, \ldots, 9$ to represent numbers in the usual way, and by convention, we use the letters A, B, C, D, E, and F in the following way.

A	B	C	D	E	F
10	11	12	13	14	15

Example

$$(D49EF)_{16} = 13 \cdot 16^4 + 4 \cdot 16^3 + 9 \cdot 16^2 + 14 \cdot 16^1 + 15 \cdot 16^0 = (870895)_{10}$$
$$(74012)_8 = 7 \cdot 8^4 + 4 \cdot 8^3 + 0 \cdot 8^2 + 1 \cdot 8^1 + 2 \cdot 8^0 = (30730)_{10}$$
$$(1010011)_2 = 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = (83)_{10}$$

It seems obvious that each positive integer is represented in a unique way using base-b representation, but in mathematics we are not satisfied with saying that something seems obvious: we must prove it. We say that a_n is the *most significant* digit and that a_0 is the *least significant* digit in the representation $(a_n a_{n-1} \cdots a_1 a_0)_b$. We always assume that the most significant digit is not zero. (For example, we would not write 0401 when we could write 401.)

Exercise

Let b be an integer greater than 1 and let i be a non-negative integer. Prove that

$$(b-1)b^i + (b-1)b^{i-1} + \cdots + (b-1)b + (b-1) = b^{i+1} - 1.$$

Proposition 9.4. *Let b be an integer greater than 1 and let m be a non-negative integer. Then m has exactly one base-b representation.*

Proof. We will first show that every non-negative integer has a base-b representation, and then we will show that the representation is unique.

Assume that there is a non-negative integer m which does not have a base-b representation. Among all such integers, we will assume that m is the smallest. Then m is not zero, because zero certainly has a base-b representation. Thus $m - 1$ is a non-negative integer which is smaller than m. Therefore it has a base-b representation. Let this representation be

$$m - 1 = a_n b^n + a_{n-1} b^{n-1} + \cdots + a_1 b + a_0$$

where each a_i is a digit between 0 and $b - 1$. If a_0 is less than $b - 1$, then

$$m = a_n b^n + a_{n-1} b^{n-1} + \cdots + a_1 b + (a_0 + 1)$$

is a base-b representation of m, which is a contradiction. So we assume that $a_0 = b - 1$. In fact, we will choose i to be the largest possible index such that $a_0, a_1, \ldots, a_i$ are all equal to $b - 1$. This

means that if i is less than n, then $a_{i+1} \neq b-1$. Now

$$(b-1)b^i + (b-1)b^{i-1} + \cdots (b-1)b + (b-1) + 1 = b^{i+1}$$

by the previous exercise. If $i < n$, then $a_{i+1} \neq b-1$, and therefore a_{i+1} is less than $b-1$. Therefore

$$m = (m-1)+1 = a_n b^{n-1} + a_{n-1}b^{n-1} + \cdots + a_{i+1}b^{i+1} + (b-1)b^i + (b-1)b^{i-1} + \cdots (b-1)b + (b-1) + 1$$
$$= a_n b^{n-1} + a_{n-1}b^{n-1} + \cdots + a_{i+1}b^{i+1} + b^{i+1} = a_n b^{n-1} + a_{n-1}b^{n-1} + \cdots + (a_{i+1}+1)b^{i+1} + 0b^i + \cdots 0$$

so m has a base-b representation, a contradiction. In the other case, $i = n$, so

$$m - 1 = (b-1)b^n + (b-1)b^{n-1} + \cdots + (b-1)b + (b-1) = b^{n+1} - 1$$

(again using the exercise). In this case m has the base-b representation $1b^{n+1} + 0b^n + \cdots + 0$. In any case, m has a representation, which is a contradiction. Therefore every non-negative integer has a base-b representation.

Now we will prove that representations are unique. We again give a proof by contradiction. Assume that $(a_s a_{s-1} \cdots a_1 a_0)_b$ and $(a'_t a'_{t-1} \cdots a'_1 a'_0)_b$ are two different base-b representations of m. The number of digits in these two representations may be different: that is, s and t may be different. By swapping the labels if needed, we can assume that $s \geq t$. If t is less than s, we will pad out the representation $(a'_t a'_{t-1} \cdots a'_1 a'_0)_b$ by placing zero digits at the beginning of the representation. That is, we set all of $a'_s, a'_{s-1}, \ldots, a'_{t+1}$ to be zero (assuming that t really is less than s). This means that $m = a'_s b^s + a'_{s-1}b^{s-1} + \cdots + a'_1 b + a'_0$. So we have two representations of m with the same number of digits: $(a_s a_{s-1} \cdots a_1 a_0)_b$ and $(a'_s a'_{s-1} \cdots a'_1 a'_0)_b$. (Technically, we should not call $(a'_s a'_{s-1} \cdots a'_1 a'_0)_b$ a representation if its first digit is zero, but that doesn't change our argument.)

Claim 9.5. There exists an index i such that $a_i \neq a'_i$.

Proof. If t is less than s, then we padded out $(a'_t a'_{t-1} \cdots a'_1 a'_0)_b$ with zeroes, which means that $a'_s = 0$. But $a_s \neq 0$ since it is a leading digit, so $a_s \neq a'_s$. So the claim is true in the case that t is less than s. Therefore we assume that $s = t$. In this case, there has to be some index i such that $a_i \neq a'_i$, or else the representations $(a_s a_{s-1} \cdots a_1 a_0)_b$ and $(a'_t a'_{t-1} \cdots a'_1 a'_0)_b$ would be exactly the same. So the claim is true in either case. □

Since there is at least one index i such that $a_i \neq a'_i$, we can choose i to be the smallest such index. We have two representations of m:

$$m = a_s b^s + a_{s-1}b^{s-1} + \cdots + a_1 b + a_0 = a'_s b^s + a'_{s-1}b^{s-1} + \cdots + a'_1 b + a'_0.$$

This means that

$$0 = m - m = (a_s - a'_s)b^s + (a_{s-1} - a'_{s-1})b^{s-1} + \cdots + (a_1 - a'_1)b + (a_0 - a'_0).$$

Because of our choice of i, we know that $a_0 = a'_0$, $a_1 = a'_1$, and so on to $a_{i-1} = a'_{i-1}$. Therefore $a_0 - a'_0, a_1 - a'_1, \ldots, a_{i-1} - a'_{i-1}$ are all zero. Therefore we can remove the terms involving these differences from the previous equation. Now we find that

$$0 = (a_s - a'_s)b^s + (a_{s-1} - a'_{s-1})b^{s-1} + \cdots + (a_i - a'_i)b^i.$$

By subtracting $(a_i - a'_i)b^i$ from both sides we obtain

$$(a'_i - a_i)b^i = (a_s - a'_s)b^s + (a_{s-1} - a'_{s-1})b^{s-1} + \cdots + (a_{i+1} - a'_{i+1})b^{i+1}.$$

Now we divide both sides by b^i to produce

$$\begin{aligned} a'_i - a_i &= (a_s - a'_s)b^{s-i} + (a_{s-1} - a'_{s-1})b^{s-i-1} + \cdots + (a_{i+1} - a'_{i+1})b \\ &= b[(a_s - a'_s)b^{s-i-1} + (a_{s-1} - a'_{s-1})b^{s-i-2} + \cdots + (a_{i+1} - a'_{i+1})] \end{aligned}$$

This shows that $a'_i - a_i$ can be expressed as the product of b with an integer. Therefore b divides $a'_i - a_i$. Now a_i and a'_i are digits in $\{0, 1, \ldots, b-1\}$. This means that $a'_i - a_i$ is in the range

$$-(b-1), -(b-2), \ldots, -1, 0, 1, \ldots, b-2, b-1.$$

There is only one number in this range that is divisible by b, and that number is 0. We conclude that $a'_i - a_i = 0$, and therefore $a_i = a'_i$. This contradicts our choice, since we said that $a_i \neq a'_i$. This contradiction completes the proof. □

Consider all the numbers with n-digit base-b representations. The smallest of these is $1b^{n-1} + 0b^{n-2} + \cdots + 0b + 0 = b^{n-1}$. The largest has every digit equal to $b-1$, so is equal to

$$(b-1)b^{n-1} + (b-1)b^{n-2} + \cdots + (b-1)b + (b-1) = b^n - 1.$$

(We have used the previous exercise.) We have shown that every number with an n-digit base-b representation lies in the range between b^{n-1} and $b^n - 1$. This implies the following result.

Proposition 9.6. *Let b be an integer greater than 1. If m is a non-negative integer, and n is the smallest integer such that $b^n > m$, then it takes exactly n digits to represent m in base b.*

Let us remind ourselves of the basics of *logarithmic functions*. A logarithmic function is the inverse of an exponential function. The number $\log_b x$ is the power to which we must raise b in order to obtain x. In other words, $\log_b x = y$ is equivalent to the statement $b^y = x$.

Example

$$\log_2 32 = 5 \qquad \log_{16} 16 = 1 \qquad \log_{10} 1 = 0 \qquad \log_8 512 = 3$$

Here are some useful properties of logarithmic functions.

Proposition 9.7. *Let b be an integer greater than 1. Then*

(i) $\log_b b^n = n$

(ii) $\log_b mn = \log_b m + \log_b n$

(iii) $\log_b m^k = k \log_b m$

(iv) $\log_b(m/n) = \log_b m - \log_b n$

We can obtain one logarithmic function from another by dividing by a constant.

Exercise

Let a and b be integers greater than 1. Prove that

$$\log_b n = \frac{1}{\log_a b}\left(\log_a n\right)$$

Recall from Definition 8.11 that if x is any real number then $\lfloor x \rfloor$ is the floor of x. It is the largest integer which is less than or equal to x. That is, we obtain $\lfloor x \rfloor$ by rounding down x to the nearest integer.

Proposition 9.8. *Let b be an integer greater than 1 and let m be a positive integer. The number of digits in the base-b representation of m is $\lfloor \log_b m \rfloor + 1$.*

Proof. Assume that the base-b representation of m has n digits. Proposition 9.6 shows that n is the smallest integer such that $m < b^n$. This means that $\log_b m < \log_b b^n = n$. (This is true because logarithmic functions are *increasing*, meaning that $x < y$ implies $\log x < \log y$.) Because n is the smallest integer such that $m < b^n$, we do not have $m < b^{n-1}$, and therefore $b^{n-1} \leq m$. Now

$$n - 1 = \log_b b^{n-1} \leq \log_b m.$$

So $\log_b m$ is a number satisfying $n - 1 \leq \log_b < n$. Therefore $\lfloor \log_b m \rfloor = n - 1$ and this gives us what we want. □

9.3 Algorithms for arithmetic

When we think about algorithms that takes the integers as input, we think of that input as being a base-b representation, for some b. Therefore the size of the input is $\lfloor \log_b m \rfloor + 1$, by Proposition 9.8. In computer science, we often think integer representations as being binary, so $b = 2$, but it doesn't matter too much which base we use, because as we proved earlier, $\log_b n$ is equal to $\log_a n$ divided by $\log_a b$. So we can obtain the number of digits needed to represent m in base b by taking the number of digits in base a and then dividing by the constant number $\log_a b$. Therefore we can think of the number of digits in base a and base b as being approximately the same. Therefore we often write $\log m$ to stand for the number of digits in a representation of m, without specifying which base we are using.

We have discussed the importance of finding algorithms with a running time that is a polynomial function of the input size. So we seek algorithms with a running time that is a polynomial function of the number of digits, rather than a running time that is a polynomial function of m itself. In other words, we want algorithms with running times that are at most

$$(\lfloor \log_b m \rfloor + 1)^c$$

where c is a fixed number. This turns out to be equivalent to saying that the running time should be at most $(\log m)^c$, where c is again some fixed number.

The standard arithmetical operations can all be completed by algorithms with running times that are polynomial functions of $\log m$. The addition algorithm we learned as children adds the numbers together digit by digit, using a carry digit. This runs in polynomial time. Let's think about multiplication. We start with Algorithm 3, which calculates the product of $(x_s x_{s-1} \cdots x_1 x_0)_b$ and the single digit y. The output is a representation $(z_s z_{s-1} \cdots z_1 z_0)_b$ or $(z_{s+1} z_s \cdots z_1 z_0)_b$, depending on how many digits are needed to represent xy. Note that y is an integer in $\{0, 1, \ldots, b-1\}$. The algorithm needs to calculate a base-b representation of $x_i y + c$, where x_i is a digit in the representation of x, and c is a carry digit. It is an exercise to show that $x_i y + c$ needs at most two digits in its base-b representation. As part of our pre-processing, we can calculate the representation of $x_i y + c$ for all possible values of x_i, y, and c.

Using Algorithm 3, we can calculate any product at all. Imagine that we want to find the product of $(x_s x_{s-1} \cdots x_1 x_0)_b$ and $(y_t y_{t-1} \cdots y_1 y_0)_b$. For each i, we find the product of

Algorithm 3 Multiplication algorithm (single digit)

Input: A base-b representation of $(x_s x_{s-1} \cdots x_1 x_0)_b$ of x and an integer y in $\{0, 1, \ldots, b-1\}$.
Output: A base-b representation of $z = xy$.

```
c ← 0    //c is the carry digit.
for i = 0 to s do
    (q_1 q_0)_b ← base-b representation of x_i y + c    //Note that we allow q_1 to be 0.
    z_i ← q_0
    c ← q_1
end for
if c > 0 then
    z_{s+1} ← c
end if
```

$(x_s x_{s-1} \cdots x_1 x_0)_b$ and y_i, and then we shift the result by adding i copies of 0 as the least significant digits. At the end of this process we have calculated representations of t integers, and we produce the final result by taking their sum. Once again, this is essentially the same as the algorithm we learn when we are young.

Example

We use Algorithm 3 to calculate the product of $(476)_8$ and $(25)_8$. First we calculate the product of $(476)_8$ and $(5)_8$. We set $(x_s x_{s-1} \cdots x_1 x_0)_b$ to be $(476)_8$ and y to be 5.

i	c	x_i	$x_i y + c$	z_i
0	0	6	$(36)_8$	6
1	3	7	$(46)_8$	6
2	4	4	$(30)_8$	0

When the algorithm terminates we have $c = 3$. So the output is $(3066)_8$.
Next we calculate the product of $(476)_8$ and $(2)_8$. We set $(x_s x_{s-1} \cdots x_1 x_0)_b = (476)_8$ and $y = 2$.

i	c	x_i	$x_i y + c$	z_i
0	0	6	$(14)_8$	4
1	1	7	$(17)_8$	7
2	1	4	$(11)_8$	1

When the algorithm terminates we have $c = 1$, so the result is $(1174)_8$. We add a zero at the end to see that the product of $(476)_8$ and $(20)_8$ is $(11740)_8$. So the product of $(476)_8$ and $(25)_8$ is the sum of $(3066)_8$ and $(11740)_8$. Using the standard addition algorithm, we see that this is equal to $(15026)_8$.

Remark

The algorithm we have presented here is the simplest algorithm for finding the product of two integers, but there are algorithms which are faster, although more complex. This is an active area of research.

Next we think about an algorithm for performing an exponentiation operation. Say that we want to calculate x^y, where x and y are positive integers. One approach would be to multiply

together y copies of x. But this would require looping y times. The running time of this algorithm would be a polynomial function of y, but not of $\log y$. Remember that we think of the input as being base-b representations of x and y, so we want an algorithm with running time that is a polynomial function of $\log x$ and $\log y$. Therefore we need a more sophisticated approach. We will not give the algorithm for a general base, b. Instead, we will only consider the binary case, that is, the case with $b = 2$. The method we present here is called *repeated squaring*.

Say that y is represented by the binary expression $(y_t y_{t-1} \cdots y_1 y_0)_2$, so that

$$y = y_t 2^t + y_{t-1} 2^{t-1} + \cdots + y_2 4 + y_1 2 + y_0.$$

Now

$$\begin{aligned} x^y = x^{y_t 2^t + y_{t-1} 2^{t-1} + \cdots + y_1 2 + y_0} &= \left(x^{y_t 2^t}\right)\left(x^{y_{t-1} 2^{t-1}}\right) \cdots \left(x^{y_1 2}\right)\left(x^{y_0}\right) \\ &= \left(x^{2^t}\right)^{y_t}\left(x^{2^{t-1}}\right)^{y_{t-1}} \cdots \left(x^2\right)^{y_1}\left(x\right)^{y_0} \end{aligned}$$

The sequence $x, x^2, x^4, x^8, \ldots, x^{2^t}$ can be obtained by repeatedly squaring. That is, each term in the sequence is the square of the previous term. Remember that each of the digits y_i is either one or zero. The important point is:

$$\text{If } y_i = 1 \text{ then } \left(x^{2^i}\right)^{y_i} = x^{2^i} \text{ and if } y_i = 0, \text{ then } \left(x^{2^i}\right)^{y_i} = 1.$$

This shows that to calculate x^y, we can keep a running product z as we proceed through the sequence $x, x^2, x^4, x^8, \ldots, x^{2^t}$. When we process x^{2^i} we look at the digit y_i. If it is equal to one then we multiply z by x^{2^i} and otherwise we leave z unchanged. This is exactly the process shown in Algorithm 4.

Algorithm 4 Algorithm for repeated squaring

Input: Base-2 representations of x and $y = (y_t y_{t-1} \cdots y_1 y_0)_2$.
Output: A representation of $z = x^y$.

```
1: z ← 1
2: p ← x
3: for i = 0 to t do
4:     if y_i = 1 then
5:         z ← z · p
6:     end if
7:     p ← p^2
8: end for
```

9.4 Quotients and remainders

Theorem 9.9. *Let m and n be positive integers. There are unique integers q and r such that $m = qn + r$ and $0 \leq r < n$.*

The number q is called the *quotient* and r is called the *remainder*. When we find the quotient and remainder, we are asking how many times n goes into m and what is left over.

Example

If $m = 10$ and $n = 4$, then q would be 2 and r would be 2, since $10 = 2 \cdot 4 + 2$ and $0 \leq 2 < 4$.
If $m = 25$ and $n = 7$, then $q = 3$ and $r = 4$, since $25 = 3 \cdot 7 + 4$ and $0 \leq 4 < 7$.

Proof of Theorem 9.9. The proof is in two parts. First we prove that q and r exist, and then we prove that they are unique. We give an inductive proof of the existence of q and r by choosing a smallest-possible counterexample. Assume that there are positive integers m and n such that q and r do not exist. From all these integers, we choose m and n so that m is as small as possible.

If m is less than n, then we could simply set q to be 0 and set r to be m. In this case $0 \leq r = m < n$, and $qn + r = 0n + m = m$. But we said that m could not be expressed as $qn + r$ with $0 \leq r < n$, so this is impossible. Therefore $m \geq n$. If $m = n$, then we can set q to be 1, and set r to be 0. In this case $0 \leq r < n$, and $qn + r = 1n + 0 = n = m$. Again, this is impossible, so $m > n$.

Now we know that $m - n > 0$, since $m > n$. Therefore $m - n$ is a positive integer. But m was chosen to be the smallest positive integer for which q and r do not exist, and $m - n$ is smaller than m. Therefore we can find the quotient and remainder for $m - n$. In other words, $m - n$ can be expressed as $q'n + r'$ for some integers q' and r', where $0 \leq r' < n$. Now we see that

$$m = (m - n) + n = (q'n + r') + n = (q'n + n) + r' = (q' + 1)n + r'.$$

If we set q to be $q' + 1$ and r to be r' then $m = qn + r$ and $0 \leq r < n$. But this contradicts our assumption that q and r do not exist for m. We created this impossible situation by assuming that q and r do not exist for some positive integer m. Therefore q and r must exist for every choice of m and n. This completes the proof of existence.

Next we must prove that q and r are unique. Assume that m can be expressed as $qn + r$ and $q'n + r'$, where q, q', r, and r' are all integers, and r and r' both belong to the range $0, 1, \ldots, n - 1$. If we can show that $q = q'$ and $r = r'$, then we will have shown that there is only one choice for q and r. Now

$$0 = m - m = (qn + r) - (q'n + r') = (q - q')n + (r - r').$$

But $0 = (q-q')n+(r-r')$ implies $r'-r = (q-q')n$. Therefore n divides $r'-r$. Since r and r' are both in the range $0, 1, \ldots, n-1$ we see that $r'-r$ is in the range $-(n-1), -(n-2), \ldots, -1, 0, 1, \ldots, n-2, n-1$. The only number in this range that is divisible by n is 0. Therefore $r' - r = 0$, and hence $r = r'$. Now $0 = (q - q')n + (r - r') = (q - q')n$. Dividing both sides by n we obtain $q - q' = 0$, so $q = q'$ and now we are done. □

Example

Let m be 78, and let n be 9. We can find the quotient and remainder by repeatedly subtracting 9 from 78.

$$78 - 9 = 69 \qquad 69 - 9 = 60 \qquad 60 - 9 = 51 \qquad 51 - 9 = 42 \qquad 42 - 9 = 33$$

$$33 - 9 = 24 \qquad 24 - 9 = 15 \qquad 15 - 9 = 6.$$

We stop when we produce a number in the range between 0 and $9 - 1 = 8$. This number is the remainder. So in this example, the remainder is 6. The number of times we subtracted 9 is 8, so this is the quotient. We deduce that $78 = 8 \times 9 + 6$.

In the previous example we repeatedly subtracted n from m to find the remainder and quotient. This process has a running time which is polynomial in m, but *not* polynomial in $\log m$. As we have discussed, we do not regard this as satisfactory. We need a smarter algorithm.

Algorithm 5 Algorithm to find the quotient and remainder

Input: Base-b representations of $m = (m_s m_{s-1} \cdots m_1 m_0)_b$ and $n = (n_t n_{t-1} \cdots n_1 n_0)_b$ ($m \geq n$).
Output: q and r such that $m = qn + r$ and $0 \leq r < n$.

```
1:  r ← m
2:  i ← s − t
3:  q ← 0
4:  while r ≥ n do
5:      while r ≥ nb^i do
6:          r ← r − nb^i
7:          q ← q + b^i
8:      end while
9:      if i > 0 then
10:         i ← i − 1
11:     end if
12: end while
```

Algorithm 5 is essentially a long-division algorithm. We set i to be $s - t$, where s and t are the number of digits in m and n respectively. Instead of subtracting copies of n from m, we subtract copies of nb^i. We do so until we reach a number that is smaller than nb^i, and then we reduce i by 1 and repeat. Each time we subtract nb^i we add b^i to q to keep track of the number of copies of n we are subtracting. Each repetition of the inner **while** loop on line 5 reduces the most significant digit of r by at least 1, so this loop can repeat at most b times. Each repetition of the outer **while** loop on line 4 reduces i by 1, so this loop runs at most $s - t$ times. Therefore the total number of loops is $b(s - t)$. Remember that b is a fixed number: it does not depend on the input. From this discussion we see that the algorithm has a running time that is polynomial in both s and t, as desired.

Example

We use Algorithm 5 to find the quotient and remainder when $m = (5019)_{10}$ and $n = (19)_{10}$.

i	r	nb^i	$r - nb^i$	q
2	$(5019)_{10}$	$(1900)_{10}$	$(3119)_{10}$	$(100)_{10}$
	$(3119)_{10}$	$(1900)_{10}$	$(1219)_{10}$	$(200)_{10}$
1	$(1219)_{10}$	$(190)_{10}$	$(1029)_{10}$	$(210)_{10}$
	$(1029)_{10}$	$(190)_{10}$	$(839)_{10}$	$(220)_{10}$
	$(839)_{10}$	$(190)_{10}$	$(649)_{10}$	$(230)_{10}$
	$(649)_{10}$	$(190)_{10}$	$(459)_{10}$	$(240)_{10}$
	$(459)_{10}$	$(190)_{10}$	$(269)_{10}$	$(250)_{10}$
	$(269)_{10}$	$(190)_{10}$	$(79)_{10}$	$(260)_{10}$
0	$(79)_{10}$	$(19)_{10}$	$(60)_{10}$	$(261)_{10}$
	$(60)_{10}$	$(19)_{10}$	$(41)_{10}$	$(262)_{10}$
	$(41)_{10}$	$(19)_{10}$	$(22)_{10}$	$(263)_{10}$
	$(22)_{10}$	$(19)_{10}$	$(3)_{10}$	$(264)_{10}$

We terminate at this point because r is less than n. We see that the quotient is 264 and the remainder is 3, so $5019 = 264 \cdot 19 + 3$.

By using Algorithm 5, we can develop a base-conversion algorithm. Say that we want to produce the base-b representation of the positive integer m. We find the quotient and remainder $m = q_0 b + r_0$ where $0 \leq r_0 < b$. Now r_0 will be the least significant digit in the representation of m. Next we find the quotient and remainder for q_0, so $q_0 = q_1 b + r_1$ where $0 \leq r_1 < b$. Now r_1 is the second-least significant digit. We continue in this way until we reach a quotient which is less than b. Say that this occurs with the quotient q_t. Thus $0 \leq q_t < b$. In fact, q_t cannot be zero, or else we would have $q_{t-1} = q_t b + r_t = 0b + r_t = r_t$ and this shows that q_{t-1} is between 0 and $b - 1$. This is impossible because we have said q_t is the first quotient to be less than b. So $q_t \neq 0$ and

$$\begin{aligned} m &= q_0 b + r_0 \\ &= (q_1 b + r_1) b + r_0 \\ &= ((q_2 b + r_2) b + r_1) b + r_0 \\ &\quad \vdots \\ &= (\cdots (q_t b + r_t) b + r_{t-1}) b + r_{t-2}) \cdots + r_1) b + r_0 \\ &= q_t b^{t+1} + r_t b^t + r_{t-1} b^{t-1} + \cdots + r_1 b + r_0 \end{aligned}$$

This shows that the base-b representation of m is $(q_t r_t r_{t-1} \cdots r_1 r_0)_b$.

Example

We will find the octal representation of $(51025)_{10}$.

$$\begin{aligned} 51025 &= 6378 \cdot 8 + 1 \\ 6378 &= 797 \cdot 8 + 2 \\ 797 &= 99 \cdot 8 + 5 \\ 99 &= 12 \cdot 8 + 3 \\ 12 &= 1 \cdot 8 + 4 \end{aligned}$$

We stop here because the final quotient is 1, which is less than 8. This calculation shows that $(51025)_{10} = (143521)_8$.

9.5 Modular arithmetic

In the next result we introduce a very important equivalence relation.

Proposition 9.10. *Let n be a positive integer. Define a relation, R, on the set of integers as follows: if a and b are integers, then (a, b) is in R if and only if n divides $a - b$. Then R is an equivalence relation.*

Proof. We need to prove that R is reflexive, symmetric, and transitive. Let a be an arbitrary integer. Then $a - a = 0$, and n divides 0 since $0 = 0 \cdot n$. Therefore (a, a) is in R, so R is reflexive.

Now assume that (a, b) is in R. Then n divides $a - b$, so $a - b$ can be written as kn for some integer k. Therefore $b - a = -(a - b) = -(kn) = (-k)n$. Since $(b - a)$ can be written as n times an integer $(-k)$, it follows that n divides $b - a$. Thus (b, a) is in R. This shows that R is symmetric.

Finally, we assume that (a, b) and (b, c) are in R. Then n divides both $a - b$ and $b - c$. Proposition 9.2 (iv) says that n divides the sum of these two numbers, so n divides $(a - b) + (b - c) = a - c$. Therefore (a, c) is in R. This shows R is transitive, and the proof is complete. □

If n divides $a - b$ then we say that a and b are *equivalent modulo* n. We will write this as

$$a \equiv b \pmod{n}.$$

Example

Let a and b be two integers. Then a and b are equivalent modulo 2 if and only if 2 divides $a - b$. This is the case if and only if a and b are both even, or both odd. The equivalence classes are the set of even integers and the set of odd integers.

The next result shows that if b and c are equivalent modulo n, then adding b to an integer is the same as adding c, at least up to equivalence modulo n. The same is true for multiplication.

Proposition 9.11. *Let n be a positive integer. Let a, b, and c be integers. If $b \equiv c \pmod{n}$, then $a + b \equiv a + c \pmod{n}$ and $ab \equiv ac \pmod{n}$.*

Proof. Since b and c are equivalent modulo n, we know that n divides $b - c$. So we can write $b - c$ as kn for some integer k. Therefore $b = c + kn$. Now $(a + b) - (a + c) = b - c = kn$, so n divides $(a + b) - (a + c)$. Therefore $a + b$ and $a + c$ are equivalent modulo n. Next, $ab - ac = a(c + kn) - ac = ac - akn - ac = (ak)n$. Therefore n divides $ab - ac$, so ab and ac are equivalent modulo n. □

Because of Proposition 9.11, we can now carry out addition and multiplication of equivalence classes. Say that n is a positive integer. Let a and b be integers, and let $[a]$ and $[b]$ be the equivalence classes of a and b, under equivalence modulo n. We want to define the sum $[a] + [b]$. This is a strange thing to do, because $[a]$ and $[b]$ are not numbers: they are sets of numbers. So how do we add two sets? In this case, we will *define* $[a] + [b]$ to be $[a + b]$. That is, the sum of $[a]$ and $[b]$ is the equivalence class that contains $a + b$. So adding together two equivalence classes has produced another equivalence class. There is a potential problem with this definition, because there are many different ways of representing the classes $[a]$ and $[b]$. If c is any other number in $[b]$ then $[b] = [c]$ (we know this because of Proposition 7.9). So $[a] + [b]$ should be the same as $[a] + [c]$. But our definition says that the first one is equal to $[a + b]$ and the second one is $[a + c]$. This is where Proposition 9.11 comes to the rescue. It tells us that $a + b$ and $a + c$ are equivalent modulo n, and therefore $[a + b]$ is the same equivalence class as $[a + c]$. It doesn't matter whether we represent this equivalence class with $a + b$ or $a + c$.

In the same way, we can define the product of $[a]$ and $[b]$ to be $[ab]$. Proposition 9.11 tells us that it does not matter which representatives we choose from the equivalence classes: the result will always be the same.

Example

Let n be 2. Two integers are equivalent modulo n if and only if both are even, or both are odd. The equivalence class containing 3 is denoted by $[3]$. It is the set of all odd numbers. We define $[3] + [5]$ to be $[3 + 5] = [8]$. This is the set of all even numbers. It does not matter which representative we choose from $[5]$. We could have chosen $[7]$ instead because $[3 + 7] = [10]$ is the same equivalence class as $[8]$: both $[8]$ and $[10]$ are equal to the set of even numbers. In the same way $[3] \cdot [2] = [3 \cdot 2] = [6]$ is the same equivalence class as $[3] \cdot [4] = [3 \cdot 4] = [12]$: either way the result is the class of even numbers.

The discussion we have just had says that when we want to add or multiply equivalence classes modulo n, it does not matter which representative we choose. But we may have a preference for particularly nice representatives. The next result tells us that there is a unique representative belonging to the set $\{0, 1, 2, \ldots, n-1\}$.

Proposition 9.12. *Let n be a positive integer and let a be an integer. There is a unique number in $\{0, 1, 2, \ldots, n-1\}$ that is equivalent to a modulo n.*

Proof. Theorem 9.9 says that there are unique integers q and r such that $a = qn + r$ and $0 \leq r < n$. Therefore $a - r = qn$, so n divides $a - r$, meaning that a and r are equivalent modulo n. Furthermore, r satisfies $0 \leq r < n$, so it is in the range $0, 1, 2, \ldots, n-1$. So we have shown that there exists a number in this range that is equivalent to a. Next we must show that it is unique. So let r' be a number in $0, 1, 2, \ldots, n-1$ that is equivalent to a. Since a is equivalent to both r and r', the symmetry and transitivity of the modulo equivalence relation mean that r and r' are equivalent modulo n. So n divides $r - r'$. But $r - r'$ is in the range

$$-(n-1), -(n-2), \ldots, -1, 0, 1, \ldots, (n-2), (n-1).$$

The only number in this range that is divisible by n is 0, so $r - r' = 0$, and hence $r = r'$. This proves uniqueness. □

Let n be a positive integer. Proposition 9.12 implies that each equivalence class modulo n has a unique representative in the range $0, 1, 2 \ldots, n-1$. These representatives are called *residues*. If a is an integer and r is the unique integer in $0, 1, 2, \ldots, n-1$ such that a and r are equivalent modulo n, we say that r is the *residue of a modulo n*. We write $a \bmod n$ to denote the residue of a modulo n.

Remark

Notice the difference between the notations $a \bmod n$ and $a \equiv b \pmod{n}$. The first notation denotes a *number*: namely the unique number in $\{0, 1, 2, \ldots, n-1\}$ that is equivalent to a modulo n. (The notation $\mathrm{rem}(a, n)$ is sometimes used to denote this number. Some programming languages use `a%n`.) The notation $a \equiv b \pmod{n}$ denotes an *equivalence*. It says that n divides the number $a - b$.

Example

$$100 \bmod 9 = 1 \qquad 16 \equiv 23 \pmod{7} \qquad 1 \bmod 2 = 1 \qquad 23 \bmod 5 = 3.$$

Let n be a positive integer. We will usually represent equivalence classes modulo n using the residues $\{0, 1, \ldots, n-1\}$. So adding and multiplying equivalence classes is really the same as adding and multiplying residues and then taking the remainder of the result. For example, if we are working modulo 6, then adding the residues 4 and 5 gives 9, and when we divide 9 by 6 we get the remainder 3. Therefore $(4+5) \bmod 6 = 3$. In the same way, $(5 \cdot 5) \bmod 6 = 1$, because $5 \cdot 5 = 25$, and when we divide 25 by 6, we get a remainder of 1, so $(5 \cdot 5) \bmod 6 = 1$.

Example

$$(12 + 7) \bmod 16 = 3 \qquad (11 \cdot 4) \bmod 12 = 8 \qquad (3 + 5) \bmod 8 = 0$$

This type of arithmetic is called *arithmetic modulo n*. When we carry out addition and multiplication modulo n, we can use the usual algorithms, and then find the remainder of the result using Algorithm 5.

Very frequently in computer science applications (especially in *cryptography*, the science of sending secret messages), we need to find the result of raising a residue to a very large power. When we are performing exponentiation in modular arithmetic we do things slightly differently. Say we want to find $2^{342} \bmod 13$. Now 2^{342} is an astronomically large number. We do not want to find all of its digits and then reduce to its residue modulo 13. Instead we use Algorithm 6, which is a modified version of Algorithm 4. Whenever we carry out a multiplication in this algorithm, on line 5 or line 7, we immediately reduce the result modulo n.

Algorithm 6 Algorithm for repeated squaring modulo n

Input: Base-2 representations of x and $y = (y_t y_{t-1} \cdots y_1 y_0)_2$, residues modulo n.
Output: A representation of $z = x^y \bmod n$.

```
1: z ← 1
2: p ← x mod n
3: for i = 0 to t do
4:     if y_i = 1 then
5:         z ← (z · p) mod n
6:     end if
7:     p ← p^2 mod n
8: end for
```

Example

We use the modified version of Algorithm 4 to calculate the value of $2^{342} \bmod 13$. First of all we want the binary representations of $x = 2$ and $y = 342$. These are $(10)_2 = (2)_{10}$ and $(101010110)_2 = (342)_{10}$. We set $z \leftarrow 1$ and $p \leftarrow x = 2$. As the algorithm runs, i is set to the values 0 through 8, since 342 has nine binary digits. Initially, z is set to 1 and p is set to $(10)_2$. The following table shows the values of y_i, p, and z at the end of each **for** loop (after we execute line 7). Remember that whenever we carry out a multiplication, we find the remainder modulo 13.

i	y_i	z	p
0	0	$(1)_2$	$(100)_2$
1	1	$(100)_2$	$(11)_2$
2	1	$(1100)_2$	$(1001)_2$
3	0	$(1100)_2$	$(11)_2$
4	1	$(1010)_2$	$(1001)_2$
5	0	$(1010)_2$	$(11)_2$
6	1	$(100)_2$	$(1001)_2$
7	0	$(100)_2$	$(11)_2$
8	1	$(1100)_2$	$(1001)_2$

The output is the value of z at the end of the **for** loop. So the algorithm has shown that $2^{342} \bmod 13$ is equal to $(1100)_2 = (12)_{10}$.

9.6 Greatest common divisors

Definition 9.13. Let m and n be non-negative integers. If d is an integer such that $d|m$ and $d|n$, then we say that d is a *common divisor* of m and n. The *greatest common divisor* of m and n is the largest integer that is a common divisor. We denote the greatest common divisor of m and n by $\gcd(m, n)$.

Example

The common divisors of 18 and 30 are ± 1, ± 2, ± 3, and ± 6. Therefore $\gcd(18, 30) = 6$.

Exercise

Use statement (i) in Proposition 9.2 to show that if m is a non-negative integer, then $\gcd(m, 0) = m$.

How do we find greatest common divisors? We could try listing all the divisors, and comparing them. But this is extremely difficult for large numbers. The most efficient way is to use a procedure that was described around 300 BCE by Euclid. The next result helps us understand why Euclid's algorithm works.

Proposition 9.14. *Let m and n be positive integers where $m \geq n$. If $r = m \bmod n$, then* $\gcd(m, n) = \gcd(n, r)$.

Proof. Let q be the quotient of m divided by n, so that $m = qn + r$ and $0 \leq r < n$. Let $d_1 = \gcd(m, n)$ and let $d_2 = \gcd(n, r)$. Since $d_2|n$, Proposition 9.2 (iii) tells us that $d_2|(qn)$. Also, $d_2|r$, so Proposition 9.2 (iv) implies that $d_2|(qn + r)$. Therefore d_2 divides $m = qn + r$. As d_2 also divides n it is a common divisor of m and n. As d_1 is the greatest common divisor of m and n, this means that $d_2 \leq d_1$.

On the other hand, $d_1|n$, so $d_1|(-qn)$. Since d_1 also divides m, it divides $m+(-qn) = m-qn = r$. Thus d_1 divides n and r. Since d_2 is the greatest common divisor of n and r, this implies $d_1 \leq d_2$. As $d_2 \leq d_1$ and $d_1 \leq d_2$, we deduce that $d_1 = d_2$, which is what we wanted to prove. □

This gives us a method for finding the greatest common divisor of two numbers, which we illustrate with an example. Assume that we want to find the greatest common divisor of 287 and 91. Our first step is to find the quotient and remainder when we divide 287 by 91. We see that $3 \cdot 91 + 14$. Proposition 9.14 tells us that $\gcd(287, 91) = \gcd(91, 14)$, so we have reduced finding $\gcd(287, 91)$ to the easier problem of finding $\gcd(91, 14)$. Now we can repeat this process. We write 91 as $6 \cdot 14 + 7$. Proposition 9.14 says that $\gcd(287, 91) = \gcd(91, 14) = \gcd(14, 7)$. We repeat the process and we see that $14 = 2 \cdot 7 + 0$, so the greatest common divisor we are looking for is $\gcd(7, 0)$. The previous exercise shows that $\gcd(7, 0) = 7$, so we have solved the problem, and now we know that $\gcd(287, 91) = 7$.

This process is illustrated by the diagram in Figure 36. The arrows show that the remainder in each step of the algorithm becomes the divisor in the next step. We halt the algorithm when we find a remainder of zero.

We think of the Euclidean algorithm in this way: we generate a sequence of numbers $r_0, r_1, r_2, \ldots$. We initialise by setting r_0 to m and r_1 to n (we assume that $m \geq n$). Each subsequent r_i is the residue of r_{i-2} modulo r_{i-1}. The residues must decrease at each step, so eventually the sequence will reach zero. Let r_p be the first residue that is equal to zero. In this case r_{p-1} is the greatest common divisor of m and n. The pseudocode for this procedure is in Algorithm 7.

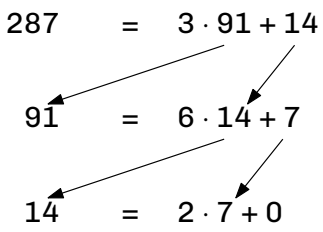


Figure 36: Using the Euclidean algorithm to find gcd(287, 91).

We use the variables r_{old} and r_{new} to store the values of r_{i-1} and r_i, as i increases. In order to update them simultaneously we use the temporary variable t.

Algorithm 7 The Euclidean algorithm

Input: Non-negative integers m and n where $m \geq n$.
Output: The greatest common divisor $r_{\text{old}} = \gcd(m, n)$.

```
r_old ← m
r_new ← n
while r_new > 0 do
    t ← r_new
    r_new ← r_old mod t
    r_old ← t
end while
return r_old
```

We want to be sure that the running time of the Euclidean algorithm is a polynomial function of the input size. Remember that the size of the input is not m or n, but rather it is $\log m + \log n$, since the integers m and n are given by their binary representations (or possibly some other base-b representation). The next result helps explain why the Euclidean algorithm has a polynomial running time.

Proposition 9.15. *Let m and n be non-negative integers such that $m \geq n$. If $r = m \bmod n$ then $r < m/2$.*

Proof. Since r is the residue of m modulo n, the definition means that r satisfies $0 \leq r < n$. Either $n \leq m/2$ is true or it is not. If $n \leq m/2$, then $r < n$ implies $r < m/2$ and we are done. So we assume that $m \geq n > m/2$. This means that $q = 1$. To see this, note that $q = 0$ would imply $m = 0n + r = r$ and now $0 \leq r < n$ implies $m < n$, contradicting $m \geq n$. Furthermore, if $q \geq 2$ then

$$m = qn + r \geq qn \geq 2n > 2(m/2) = m$$

and this implies $m > m$ which is impossible. So $q = 1$ is the only option. This means that $m = n+r$. Therefore

$$r = m - n < m - (m/2) = m/2.$$

So $r < m/2$ in either case. □

Proposition 9.16. *Let m and n be non-negative integers satisfying $m \geq n$. Define the sequence $r_0, r_1, r_2, \ldots$ by $r_0 = m$, $r_1 = n$, and $r_i = r_{i-2} \bmod r_{i-1}$. Let q be the smallest integer such that $2^q > m$. There is some integer $p \leq 2q$ such that $r_p = 0$.*

Proof. We repeatedly use Proposition 9.15. It tells us that $r_2 < r_0/2 = m/2$. By applying the same result we see that $r_4 < r_2/2 = m/4$ and $r_6 < m/8$. In general, $r_{2k} < m/2^k$. In particular, this means that

$$r_{2q} < \frac{m}{2^q} < \frac{m}{m} = 1.$$

So r_{2q} is a non-negative integer that is less than 1. Therefore $r_{2q} = 0$. The result follows. □

A binary representation of m has q digits, where q is the smallest integer such that $2^q > m$. Proposition 9.16 shows that the number of loops in the Euclidean algorithm is at most $2q$. So the number of loops is a polynomial function of the number of binary digits in m, and this is enough to demonstrate that the Euclidean algorithm has a running time that is a polynomial function of the input size.

We take a small diversion and describe the Euclidean algorithm as a *recursive* algorithm. Algorithm 8 shows an alternative piece of pseudocode. It describes a function, GCD, which takes as input non-negative integers m and n where $m \geq n$. Notice that when we execute the pseudocode for GCD, we are required to call the function GCD (on line 6). This may seem circular, but because the integers that we use as inputs to the function become smaller and smaller, we will eventually reach a point where we want to evaluate GCD$(m, 0)$. At this point the algorithm tells us that the output is m. This algorithm is called recursive because it describes a function by calling on the function itself. This style of algorithm will be discussed in more detail later in the module.

Algorithm 8 The Euclidean algorithm as a recursive procedure

Input: Non-negative integers m and n where $m \geq n$.
Output: The greatest common divisor $\gcd(m, n)$.

```
1: procedure GCD(m, n)
2:     if n = 0 then
3:         return m
4:     else
5:         r ← m mod n
6:         return GCD(n, r)
7:     end if
8: end procedure
```

There is an important application of the Euclidean algorithm that we now discuss.

Proposition 9.17. *Let m and n be positive integers. There exist integers x and y such that $xm + yn = \gcd(m, n)$.*

Proof. We give a proof by induction. Let us assume that there is some pair m, n for which we cannot express $\gcd(m, n)$ as an integer combination $xm+yn$. Assume that $m \geq n$. We will assume that amongst all the possible counterexample pairs with $m \geq n$, we have chosen m and n so that m is as small as possible. If $m = n$ then $\gcd(m, n) = m = 1m + 0n$, so m, n does not provide a counterexample after all. Therefore we have to assume that $m > n$. Now there are integers q and r such that $m = qn + r$ and $0 \leq r < n$. Because n is smaller than m, the pair n, r cannot provide a counterexample. Therefore there are integers x' and y' such that $\gcd(n, r) = x'n + y'r$. Proposition 9.14 says that

$$\gcd(m, n) = \gcd(n, r) = x'n + y'r = x'n + y'(m - qn) = y'm + (x' - y'q)n.$$

But now we can set x to be y' and y to be $x' - y'q$. We see that m, n do not give us a counterexample after all. This is a contradiction, and we must reject the idea that there are any counterexamples. Therefore the proposition is proved. □

The process we use to find integers x and y such that $xm + yn = \gcd(m, n)$ is called the *extended Euclidean algorithm*. Algorithm 9 shows the pseudocode for the extended Euclidean algorithm. It is similar to the pseudocode in Algorithm 7, but it has been augmented with the variables x_{old} and y_{old}. They are continuously updated so that $r_{\text{old}} = x_{\text{old}}m + y_{\text{old}}n$ is correct after every loop of the algorithm. A similar process happens with the variables x_{new} and y_{new}. When the algorithm terminates, r_{old} is equal to $\gcd(m, n)$, and this greatest common divisor can be expressed as $x_{\text{old}}m + y_{\text{old}}n$.

Algorithm 9 The extended Euclidean algorithm

Input: Non-negative integers m and n where $m \geq n$.
Output: The greatest common divisor $r_{\text{old}} = \gcd(m, n)$ and integers x_{old} and y_{old} such that $\gcd(m, n) = x_{\text{old}}m + y_{\text{old}}n$.

$r_{\text{old}} \leftarrow m$
$r_{\text{new}} \leftarrow n$
$x_{\text{old}} \leftarrow 1$
$x_{\text{new}} \leftarrow 0$
$y_{\text{old}} \leftarrow 0$
$y_{\text{new}} \leftarrow 1$
while $r_{\text{new}} > 0$ **do**
    $t \leftarrow r_{\text{new}}$
    $(q, r) \leftarrow$ integers such that $r_{\text{old}} = qt + r$ and $0 \leq r < t$
    $r_{\text{new}} \leftarrow r$
    $r_{\text{old}} \leftarrow t$
    $t_x \leftarrow x_{\text{new}}$
    $x_{\text{new}} \leftarrow x_{\text{old}} - qt_x$
    $x_{\text{old}} \leftarrow t_x$
    $t_y \leftarrow y_{\text{new}}$
    $y_{\text{new}} \leftarrow y_{\text{old}} - qt_y$
    $y_{\text{old}} \leftarrow t_y$
end while
return $(r_{\text{old}}, x_{\text{old}}, y_{\text{old}})$

Example

If we want to apply the extended Euclidean algorithm using paper and pen, we would probably not use the pseudocode in Algorithm 9. Instead, we implement the Euclidean algorithm as usual, and then backtrack through the equations we generate to find the coefficients we need to express the greatest common divisor.
Say that we want to express $\gcd(623, 222)$ as a combination $623x + 222y$. We start with the usual Euclidean algorithm.

$$623 = 2 \cdot 222 + 179 \quad (1)$$
$$222 = 1 \cdot 179 + 43 \quad (2)$$

$$179 = 4 \cdot 43 + 7 \tag{3}$$
$$43 = 6 \cdot 7 + 1 \tag{4}$$
$$7 = 7 \cdot 1 + 0 \tag{5}$$

These calculations show that $\gcd(623, 222) = 1$. By rewriting equation (4), we see that $1 = 43 - 6 \cdot 7$. Now we can rewrite (3) to express 7 as $179 - 4 \cdot 43$ and substitute this in. We obtain

$$1 = 43 - 6(179 - 4 \cdot 43) = 25 \cdot 43 - 6 \cdot 179.$$

Next we rewrite (2) to express 43 as $222 - 179$ and substitute this in to obtain

$$1 = 25(222 - 179) - 6 \cdot 179 = 25 \cdot 222 - 31 \cdot 179.$$

Finally, we rewrite (1) to express 179 as $623 - 2 \cdot 222$ and we end the process.

$$1 = 25 \cdot 222 - 31(623 - 2 \cdot 222) = 87 \cdot 222 - 31 \cdot 623.$$

We see that if $x = -31$ and $y = 87$, then $623x + 222y = \gcd(623, 222) = 1$.

9.7 Multiplicative inverses

Let n be a positive integer. Let a and b be residues modulo n. Recall that this means that a and b are integers in $\{0, 1, \ldots, n-1\}$. If $ab \bmod n = 1$ then we say that a and b are *multiplicative inverses* of each other (modulo n). If a multiplicative inverse for a exists, then we will write it as a^{-1}.

Example

The multiplicative inverse of 5 modulo 11 is 9, because $5 \times 9 = 45$ and the residue of 45 modulo 11 is 1.

In the next result we explain when multiplicative inverses exist. If $\gcd(n, a) = 1$, then we say that n and a are *coprime*.

Proposition 9.18. *Let n be a positive integer and let a be a residue modulo n. Then a has a multiplicative inverse modulo n if and only if n and a are coprime.*

Proof. Assume that n and a are coprime, so that $\gcd(n, a) = 1$. Proposition 9.17 tells us that there are integers x and y such that $xn + ya = 1$. We claim that the residue of y modulo n is the multiplicative inverse of a. Let q and r be the quotient and remainder of y after dividing by n, so $y = qn + r$ and $0 \leq r < n$, which means that r is the residue of y modulo m. Now

$$ar - 1 = a(y - qn) - 1 = (ya - 1) - qna = (-xn) - qna = (-x - qa)n.$$

This shows that n divides $ar - 1$, so ar and 1 are equivalent modulo n. Therefore 1 is the residue of ra modulo n and r is the inverse of a, as we claimed.

For the other direction of the proof, we assume that a has an inverse b, so that b is a residue modulo n, and $ab \bmod n = 1$. Therefore ab and 1 are equivalent modulo n, so n divides $ab - 1$. We write $ab - 1$ as kn. Let d be the greatest common divisor of n and a. Since d divides both n and a we see that d divides $ab - kn = 1$. So d is a positive integer and it divides 1. This means that $d = 1$, so n and a are coprime, as we wanted. □

This last result shows us how to find multiplicative inverses: say that n is a positive integer and that a is a residue modulo n. To find the multiplicative inverse of a, we perform the extended Euclidean algorithm on n and a. If $\gcd(n, a) > 1$ then no inverse exists. Otherwise the extended Euclidean algorithm gives us integers x and y such that $xn + ya = 1$. We reduce y to its residue modulo n, and the result is the inverse of a.

Example

What is the multiplicative inverse of 22 modulo 27? We start with the Euclidean algorithm.

$$\begin{aligned} 27 &= 1 \cdot 22 + 5 \\ 22 &= 4 \cdot 5 + 2 \\ 5 &= 2 \cdot 2 + 1 \\ 2 &= 2 \cdot 2 + 0 \end{aligned}$$

We backtrack through these equations and find that

$$\begin{aligned} 1 &= 5 - 2 \cdot 2 \\ &= 5 - 2(22 - 4 \cdot 5) = 9 \cdot 5 - 2 \cdot 22 \\ &= 9(27 - 22) - 2 \cdot 22 = 9 \cdot 27 - 11 \cdot 22 \end{aligned}$$

Now we can find the multiplicative inverse of 22 by finding the residue of -11 modulo 27. We can express -11 as $(-1) \cdot 27 + 16$. Therefore its residue is 16, and this is the inverse of 22 modulo 27. We check that $16 \cdot 22 = 352 = 13 \cdot 27 + 1$, so $(22 \cdot 16) \bmod 27 = 1$.

Modular inverses are used in many computer science applications, especially in cryptography. We will talk about a related application: the generation of *pseudorandom* numbers. Random numbers are useful in many contexts, but it is very hard to use a computer to generate true randomness. Instead, we develop algorithms that generate numbers that *appear* random. Here is one way of doing this: Choose n to be a large prime number and choose numbers a and c such that $0 < a, c < n$. Because n is prime, every number between 1 and $n - 1$ is coprime with n. This means that every residue modulo n has a multiplicative inverse, except for zero. We choose the *seed number* x_0 satisfying $0 \leq x_0 < n$ and generate a sequence of numbers using the following definition:

$$x_{i+1} = \begin{cases} (ax_i^{-1} + c) \bmod n & \text{if } x_i \neq 0 \\ c & \text{if } x_i = 0 \end{cases}$$

For a very small example, choose n to be 7, choose $a = 2$ and $c = 3$. If $x_0 = 2$ is the seed value, then $x_0^{-1} = 4 \bmod 7$, so $x_1 = (2 \cdot 4 + 3) \bmod 7 = 4$. The inverse of 4 modulo 7 is 2, so $x_2 = (2 \cdot 2 + 3) \bmod 7 = 0$. Since $x_2 = 0$, the next number is $x_3 = c = 3$, and so on. The first few numbers in the sequence are $2, 4, 0, 3, 6, 1, 5, 2, 4, 0, 3, 6, \ldots$. The sequence will repeat periodically after this.

For a larger example, say that $n = 113$, $a = 28$, and $c = 7$. Then the pseudorandom sequence that starts with the seed $x_0 = 57$ begins in this way:

$$57, 63, 20, 31, 48, 17, 2, 21, 46, 42, 83, 104, 29, 82, 79, 66, 4, 14, 9, 98, 88, 33, 1, 35, 53, 112, 92, 81, 105, 60, 15, 39, 28, 8, 67, 85, 6, 87, 32, 22, 111, 106, 3, 54, 41, 38, 91, 16, 37, 78, 74, \ldots$$

This sequence certainly has the appearance of a random sequence, although we would not want to depend on it for high-security cryptographic applications.

Summary

Important ideas from Section 9.

- If p and q are integers, then $p|q$ means that there is an integer k such that $q = kp$.
- Let $b > 1$ be an integer. A base-b representation is a sequence $(a_n a_{n-1} \cdots a_1 a_0)_b$ where each a_i is in the range $0, 1, \ldots, b-1$. This stands for the number $a_n b^n + a_{n-1} b^{n-1} + \cdots + a_1 b + a_0$.
- Let m be a positive integer. The number of digits in a base-b representation of m is the smallest integer n such that $b^n > m$. This is equal to $\lfloor \log_b m \rfloor + 1$.
- Let m and n be positive integers. There are unique integers q and r such that $m = qn + r$ and $0 \leq r < n$. We say q is the quotient and r is the remainder after dividing m by n.
- If r is the remainder after dividing m by n, then we say that r is the residue of m modulo n, and we write $r = m \bmod n$.
- Let a, b, and n be positive integers. We write $a \equiv b \pmod{n}$ if n divides $a - b$.
- Let m and n be positive integers. A common divisor of m and n is a number d such that $d|m$ and $d|n$. The greatest common divisor is written $gcd(m, n)$.
- There are integers x and y such that $\gcd(m, n) = xm + yn$.
- Let n be a positive integer. Let a and b be residues modulo n. If $ab \bmod n$ then a and b are inverses modulo n.